Assignment 2

June 5, 2025

CC4051 Machine Learning - FCUP

05.06.2025

By Camila Alves, Miguel Goncalves, and Nina Lichtenberger (Group B)

1 Introduction

Intensive Care Units (ICUs) generate vast amounts of complex, high-frequency data that reflect the dynamic and often critical condition of patients. Accurately predicting the **Length of Stay (LOS)** in the ICU is essential for effective resource allocation, staff planning, and improving patient outcomes. However, LOS prediction remains challenging due to the heterogeneity of patient profiles and the non-linear relationships among clinical variables. In this project, we leverage the **MIMIC-III database**, a rich source of de-identified ICU data, to develop machine learning models that estimate LOS for patients diagnosed with **heart failure (ICD-9: 4280)**. By transforming raw time series data into structured, tabular features and applying various machine learning algorithms, we aim to identify which clinical factors are most predictive of LOS and assess the comparative performance of different modeling approaches. Through this work, we seek not only to build an accurate predictive model but also to gain interpretable insights that could inform ICU clinical practice. This work is structured in the following way: in the first section, we will preprocess various tables of the database to craft a dataframe that can be used for prediction purposes. Then, we will visualize the data of an individual patient to gain deeper insight into a common ICU stay. In section 3, we will conduct a statistical analysis comprising a general data exploration as well as a deep dive into key variables. Lastly, we will build and finetune several models and analyze their performance, residuals, as well as feature importance.

2 Data Preprocessing

2.1 Building the Base Dataframe for LOS Prediction and Individual Patient Observations

2.1.1 Textual Summary

We started this project by loading and filtering the raw data. We loaded only the relevant columns from different tables in MIMIC-III. Columns that were unlikely to help with predicting ICU Length of Stay (LOS) or were difficult to process (such as unstructured notes) were skipped to simplify the initial analysis, though they could still be added at the end if model performance was not satisfactory. In our work, we focused on patients with ICD9 code 4280 (Congestive Heart Failure) and data from Metavision. The excluded tables included:

1. **datetimevents:** This table records the dates of specific clinical events (e.g., first dialysis). While potentially useful, these temporal markers are challenging to preprocess in a way that machine learning models can effectively utilize.
2. **labevents and microbiologyevents:** These tables contain laboratory and microbiology results, respectively. However, we excluded them because the entries were not explicitly linked to an `ICUSTAY_ID`, making it difficult to associate them with specific ICU stays—a key requirement for our analysis.
3. **inputevents_cv:** This table documents medication inputs recorded via the CareVue system. For consistency and simplicity, we used only the Metavision (`inputevents_mv`) data, as it provides a more structured and comprehensive representation of administered treatments. In this way, we ensured a uniform data format. Considering the high number of instances, we could afford to exclude the CareVue data.
4. **prescriptions:** While this table includes medication orders, we excluded it in favor of the inputevents tables, which captures what was actually administered to patients rather than what was merely prescribed. This ensured our analysis reflected real-world treatment delivery.

We then created the dataframe with basic information (no events) by merging patient data across different tables. This dataframe was later used for LOS prediction as well as for the visualization of individual patient stays. In an initial data exploration, we performed basic checks to understand the structure and content of the dataframe. We looked at missing values, unique counts, and descriptive statistics to determine what cleaning was needed and which features might be useful. Insights from these statistics informed our decisions on data scaling and feature engineering.

The only numerical variable contained in the non-processed data was the target variable, LOS. We observed that the LOS had a wide range (from 0.0004 days to 87 days) with a mean of 4.7 and a standard deviation of 5.89. This indicated a high variance when comparing the standard deviation to the mean.

We also cleaned categorical columns to remove values that were either too rare, noisy, or not useful for prediction. Ethnicity was cleaned and standardized using partial matching, which helped reduce category fragmentation. Cleaning in admission location and type removed confusing or irrelevant admission sources and dropped rows with too many missing values to maintain data quality. We also simplified admission types for better grouping.

Afterwards, we implemented feature engineering to create two useful features: the age of the patient at the time of admission and the time between hospital admission and ICU admission (in hours). Negative values were handled appropriately (set to 0), and the features were cleaned for modeling.

One ICU stay can have different diagnoses. We aimed to use the additional diagnoses that patients received apart from Congestive Heart Failure as a feature. Some medical conditions might increase or decrease the LOS. To avoid the curse of dimensionality and prevent sparse and noisy data, we retained only diagnoses that appeared frequently in the dataset. Rare ICD9 codes were not used as features, as they were likely to represent unique rather than generalizable cases.

Furthermore, a patient can have more than one ICU stay per hospital admission, which could indicate more severe cases. This was considered an important feature for predicting LOS, so we also computed it.

The outcome of this section is a cleaned and engineered dataframe for patients with heart failure (ICD9 4280) for the next steps (modeling). In the following sections, we include time-series data

in the form of events in the dataframe. This data has to be handled differently for the two tasks of 1. visualizing individual patient observations and 2. predicting LOS, but the dataframe created in this section serves as the base for both of these tasks.

2.1.2 Code

[ ]:
```
import pandas as pd

# For all datasets, we only keep columns with predictive information
admissions = pd.read_csv('ADMISSIONS.csv')
admissions = admissions[['INSURANCE', 'LANGUAGE', 'RELIGION', 'MARITAL_STATUS',␣
↪'ETHNICITY', 'ADMISSION_LOCATION', 'ADMISSION_TYPE', 'HADM_ID',␣
↪'SUBJECT_ID', 'ADMITTIME']]
diagnoses = pd.read_csv('DIAGNOSES_ICD.csv')
diagnoses = diagnoses[['ICD9_CODE', 'HADM_ID']]
icustays = pd.read_csv('ICUSTAYS.csv')
icustays = icustays[['HADM_ID', 'LOS', 'ICUSTAY_ID', 'INTIME','DBSOURCE']]
patients = pd.read_csv('PATIENTS.csv')
patients = patients[['SUBJECT_ID', 'GENDER', 'DOB']]
```

[ ]:
```
# Building base df
df = icustays.copy()
df = df.merge(admissions, on='HADM_ID', how='left')
df = df.merge(patients, on='SUBJECT_ID', how='left')
df = df.merge(diagnoses, on='HADM_ID', how='left')
df['ICUSTAY_ID'].duplicated().any()
```

[ ]:
```
np.True_
```

[ ]:
```
df[df['ICUSTAY_ID'].duplicated(keep=False)] # we have duplicate rows for␣
↪certain stays bc the stays had multiple diagnoses
```

[ ]:
```
        HADM_ID     LOS  ICUSTAY_ID               INTIME    DBSOURCE  \
0        110404  3.2490      280836  2198-02-14 23:27:38     carevue
1        110404  3.2490      280836  2198-02-14 23:27:38     carevue
2        110404  3.2490      280836  2198-02-14 23:27:38     carevue
3        110404  3.2490      280836  2198-02-14 23:27:38     carevue
4        110404  3.2490      280836  2198-02-14 23:27:38     carevue
…          …      …          …                    …          …
705916   156386  4.5022      275346  2157-05-19 02:54:54  metavision
705917   156386  4.5022      275346  2157-05-19 02:54:54  metavision
705918   156386  4.5022      275346  2157-05-19 02:54:54  metavision
705919   156386  4.5022      275346  2157-05-19 02:54:54  metavision
705920   156386  4.5022      275346  2157-05-19 02:54:54  metavision

       INSURANCE LANGUAGE  RELIGION MARITAL_STATUS           ETHNICITY  \
0       Medicare      NaN  CATHOLIC      SEPARATED  HISPANIC OR LATINO
```

```
1       Medicare      NaN  CATHOLIC      SEPARATED  HISPANIC OR LATINO
2       Medicare      NaN  CATHOLIC      SEPARATED  HISPANIC OR LATINO
3       Medicare      NaN  CATHOLIC      SEPARATED  HISPANIC OR LATINO
4       Medicare      NaN  CATHOLIC      SEPARATED  HISPANIC OR LATINO
…          …      …        …            …                  …
705916   Private     ENGL  CATHOLIC        MARRIED               WHITE
705917   Private     ENGL  CATHOLIC        MARRIED               WHITE
705918   Private     ENGL  CATHOLIC        MARRIED               WHITE
705919   Private     ENGL  CATHOLIC        MARRIED               WHITE
705920   Private     ENGL  CATHOLIC        MARRIED               WHITE

          ADMISSION_LOCATION ADMISSION_TYPE  SUBJECT_ID            ADMITTIME  \
0       EMERGENCY ROOM ADMIT      EMERGENCY         268  2198-02-11 13:40:00
1       EMERGENCY ROOM ADMIT      EMERGENCY         268  2198-02-11 13:40:00
2       EMERGENCY ROOM ADMIT      EMERGENCY         268  2198-02-11 13:40:00
3       EMERGENCY ROOM ADMIT      EMERGENCY         268  2198-02-11 13:40:00
4       EMERGENCY ROOM ADMIT      EMERGENCY         268  2198-02-11 13:40:00
…                     …            …         …                  …
705916  EMERGENCY ROOM ADMIT      EMERGENCY       94956  2157-05-19 02:54:00
705917  EMERGENCY ROOM ADMIT      EMERGENCY       94956  2157-05-19 02:54:00
705918  EMERGENCY ROOM ADMIT      EMERGENCY       94956  2157-05-19 02:54:00
705919  EMERGENCY ROOM ADMIT      EMERGENCY       94956  2157-05-19 02:54:00
705920  EMERGENCY ROOM ADMIT      EMERGENCY       94956  2157-05-19 02:54:00

       GENDER                  DOB ICD9_CODE
0           F  2132-02-21 00:00:00     48241
1           F  2132-02-21 00:00:00      4280
2           F  2132-02-21 00:00:00     51881
3           F  2132-02-21 00:00:00      5845
4           F  2132-02-21 00:00:00      4275
…       …                  …       …
705916      M  2123-03-01 00:00:00     25001
705917      M  2123-03-01 00:00:00     V4585
705918      M  2123-03-01 00:00:00     87342
705919      M  2123-03-01 00:00:00      4019
705920      M  2123-03-01 00:00:00      2720

[705604 rows x 17 columns]
```

[ ]:
```python
# Find ICUSTAY_IDs where the patient had our ICD9 code of interest
ids_to_keep = df.loc[df['ICD9_CODE'] == '4280', 'ICUSTAY_ID'].unique()
# Keep all rows for those ICU stays
df = df[df['ICUSTAY_ID'].isin(ids_to_keep)]
```

[ ]:
```python
# Find ICUSTAY_IDs where the patient has our dbsource
ids_to_keep = df.loc[df['DBSOURCE'] == 'metavision', 'ICUSTAY_ID'].unique()
# Keep all rows for those ICU stays
```

```
df = df[df['ICUSTAY_ID'].isin(ids_to_keep)]
```

[ ]:
```
import pandas as pd

# --- BASIC INFO ---
print("="*50)
print("1. BASIC INFO")
print("="*50)
print(df.info())
print("\nShape of DataFrame:", df.shape)

# --- FIRST FEW ROWS ---
print("\n", "="*50)
print("2. FIRST FEW ROWS")
print("="*50)
print(df.head())

# --- DESCRIPTIVE STATS ---
print("\n", "="*50)
print("3. DESCRIPTIVE STATISTICS (Numerical Variables)")
print("="*50)
print(df.describe())

# --- NA VALUES ---
print("\n", "="*50)
print("4. MISSING VALUES PER COLUMN")
print("="*50)
na_counts = df.isna().sum()
na_perc = (na_counts / len(df)) * 100
na_summary = pd.DataFrame({'Missing Values': na_counts, 'Percent Missing':␣
↪na_perc})
print(na_summary[na_summary['Missing Values'] > 0].sort_values('Percent␣
↪Missing', ascending=False))

# --- UNIQUE VALUES ---
print("\n", "="*50)
print("5. UNIQUE VALUES PER COLUMN")
print("="*50)
for col in df.columns:
    unique_vals = df[col].nunique()
    print(f"{col}: {unique_vals} unique values")

# --- CATEGORICAL VARIABLES: VALUE COUNTS ---
print("\n", "="*50)
print("6. VALUE COUNTS FOR CATEGORICAL VARIABLES")
print("="*50)
for col in df.columns:
```

```python
    if df[col].dtype in ['object', 'category']:
        print(f"\nColumn: {col}")
        print(df[col].value_counts(dropna=False))

# --- NUMERICAL VARIABLES: QUICK SUMMARY ---
print("\n", "="*50)
print("7. SUMMARY OF NUMERICAL VARIABLES")
print("="*50)
for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        print(f"\nColumn: {col}")
        print(f"Min: {df[col].min()}, Max: {df[col].max()}, Mean: {df[col].
 ↪mean()}, Std: {df[col].std()}")
```

```
==================================================
1. BASIC INFO
==================================================
<class 'pandas.core.frame.DataFrame'>
Index: 115883 entries, 405 to 705874
Data columns (total 17 columns):
 #   Column              Non-Null Count   Dtype
---  ------              --------------   -----
 0   HADM_ID             115883 non-null  int64
 1   LOS                 115883 non-null  float64
 2   ICUSTAY_ID          115883 non-null  int64
 3   INTIME              115883 non-null  object
 4   DBSOURCE            115883 non-null  object
 5   INSURANCE           115883 non-null  object
 6   LANGUAGE            114965 non-null  object
 7   RELIGION            115883 non-null  object
 8   MARITAL_STATUS      113271 non-null  object
 9   ETHNICITY           115883 non-null  object
 10  ADMISSION_LOCATION  115883 non-null  object
 11  ADMISSION_TYPE      115883 non-null  object
 12  SUBJECT_ID          115883 non-null  int64
 13  ADMITTIME           115883 non-null  object
 14  GENDER              115883 non-null  object
 15  DOB                 115883 non-null  object
 16  ICD9_CODE           115883 non-null  object
dtypes: float64(1), int64(3), object(13)
memory usage: 15.9+ MB
None

Shape of DataFrame: (115883, 17)

 ==================================================
2. FIRST FEW ROWS
```

```
==================================================
     HADM_ID    LOS  ICUSTAY_ID               INTIME    DBSOURCE INSURANCE  \
405   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
406   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
407   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
408   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
409   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare

    LANGUAGE  RELIGION MARITAL_STATUS ETHNICITY         ADMISSION_LOCATION  \
405     PORT  CATHOLIC        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM
406     PORT  CATHOLIC        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM
407     PORT  CATHOLIC        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM
408     PORT  CATHOLIC        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM
409     PORT  CATHOLIC        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM

    ADMISSION_TYPE  SUBJECT_ID            ADMITTIME GENDER  \
405      EMERGENCY         305  2129-08-20 18:03:00      F
406      EMERGENCY         305  2129-08-20 18:03:00      F
407      EMERGENCY         305  2129-08-20 18:03:00      F
408      EMERGENCY         305  2129-08-20 18:03:00      F
409      EMERGENCY         305  2129-08-20 18:03:00      F

                     DOB ICD9_CODE
405  2052-10-24 00:00:00      0389
406  2052-10-24 00:00:00     42823
407  2052-10-24 00:00:00     78552
408  2052-10-24 00:00:00      5849
409  2052-10-24 00:00:00     34831

 ==================================================
3. DESCRIPTIVE STATISTICS (Numerical Variables)
==================================================
             HADM_ID            LOS     ICUSTAY_ID     SUBJECT_ID
count  115883.000000  115883.000000  115883.000000  115883.000000
mean   150569.203714       4.700539  250037.164502   56920.567106
std     28865.671675       5.886910   28855.057602   26984.877532
min    100018.000000       0.000400  200001.000000      34.000000
25%    125351.000000       1.425100  225571.000000   32605.000000
50%    151045.000000       2.768100  249900.000000   59225.000000
75%    175345.000000       5.288400  275184.000000   79075.000000
max    199972.000000      86.848400  299957.000000   99995.000000

 ==================================================
4. MISSING VALUES PER COLUMN
==================================================
                Missing Values  Percent Missing
MARITAL_STATUS            2612         2.253998
LANGUAGE                   918         0.792178
```

```
 ==================================================
5. UNIQUE VALUES PER COLUMN
==================================================
HADM_ID: 5497 unique values
LOS: 5731 unique values
ICUSTAY_ID: 6013 unique values
INTIME: 6013 unique values
DBSOURCE: 1 unique values
INSURANCE: 5 unique values
LANGUAGE: 38 unique values
RELIGION: 16 unique values
MARITAL_STATUS: 7 unique values
ETHNICITY: 34 unique values
ADMISSION_LOCATION: 6 unique values
ADMISSION_TYPE: 3 unique values
SUBJECT_ID: 4216 unique values
ADMITTIME: 5496 unique values
GENDER: 2 unique values
DOB: 4070 unique values
ICD9_CODE: 3091 unique values

 ==================================================
6. VALUE COUNTS FOR CATEGORICAL VARIABLES
==================================================

Column: INTIME
INTIME
2200-04-09 20:22:32    39
2121-09-08 10:42:25    39
2194-09-15 20:24:54    39
2108-05-02 02:02:57    39
2170-04-12 19:20:15    39
                       ..
2184-02-05 11:27:18     5
2155-07-30 11:50:17     5
2118-09-06 10:59:23     4
2143-11-12 14:41:13     4
2189-11-03 10:22:08     4
Name: count, Length: 6013, dtype: int64

Column: DBSOURCE
DBSOURCE
metavision    115883
Name: count, dtype: int64

Column: INSURANCE
INSURANCE
```

```
Medicare      90302
Private       17894
Medicaid       5986
Government     1391
Self Pay        310
Name: count, dtype: int64

Column: LANGUAGE
LANGUAGE
ENGL    103119
RUSS      3449
SPAN      2510
PTUN      1520
NaN        918
CANT       656
ITAL       517
PORT       481
CAPE       422
HAIT       313
ARAB       264
GREE       245
MAND       233
VIET       186
PERS       123
POLI       121
CAMB       106
*ARM        85
AMER        60
*BUR        60
BENG        50
*CDI        49
SOMA        48
ALBA        42
**TO        38
*GUJ        35
FREN        34
HIND        29
TAGA        21
*TEL        20
*TOI        20
URDU        18
*TOY        17
KORE        15
*ARA        15
*FIL        15
*CRE        14
THAI        10
*CHI         5
```

```
Name: count, dtype: int64

Column: RELIGION
RELIGION
CATHOLIC                  45365
NOT SPECIFIED             21789
PROTESTANT QUAKER         18757
JEWISH                    15115
UNOBTAINABLE               5289
OTHER                      3646
EPISCOPALIAN               1478
GREEK ORTHODOX             1252
MUSLIM                      759
CHRISTIAN SCIENTIST         690
BUDDHIST                    587
ROMANIAN EAST. ORTH         353
7TH DAY ADVENTIST           290
JEHOVAH'S WITNESS           212
UNITARIAN-UNIVERSALIST      180
HINDU                       121
Name: count, dtype: int64

Column: MARITAL_STATUS
MARITAL_STATUS
MARRIED              51760
SINGLE               26141
WIDOWED              24651
DIVORCED              8892
NaN                   2612
SEPARATED             1355
UNKNOWN (DEFAULT)      385
LIFE PARTNER            87
Name: count, dtype: int64

Column: ETHNICITY
ETHNICITY
WHITE                                                 83770
BLACK/AFRICAN AMERICAN                                14184
UNKNOWN/NOT SPECIFIED                                  2807
OTHER                                                  2750
HISPANIC OR LATINO                                     2655
UNABLE TO OBTAIN                                       2276
WHITE - RUSSIAN                                        1124
ASIAN                                                   983
HISPANIC/LATINO - PUERTO RICAN                          753
PATIENT DECLINED TO ANSWER                              700
ASIAN - ASIAN INDIAN                                    664
ASIAN - CHINESE                                         515
```

```
PORTUGUESE                                                    459
BLACK/CAPE VERDEAN                                            401
BLACK/HAITIAN                                                 220
ASIAN - FILIPINO                                              210
HISPANIC/LATINO - DOMINICAN                                   187
WHITE - OTHER EUROPEAN                                        181
MIDDLE EASTERN                                                142
WHITE - BRAZILIAN                                             115
ASIAN - CAMBODIAN                                             106
MULTI RACE ETHNICITY                                          102
ASIAN - VIETNAMESE                                             98
AMERICAN INDIAN/ALASKA NATIVE                                  94
BLACK/AFRICAN                                                  87
WHITE - EASTERN EUROPEAN                                       76
HISPANIC/LATINO - CUBAN                                        55
HISPANIC/LATINO - GUATEMALAN                                   34
HISPANIC/LATINO - CENTRAL AMERICAN (OTHER)                     33
ASIAN - KOREAN                                                 28
HISPANIC/LATINO - SALVADORAN                                   28
AMERICAN INDIAN/ALASKA NATIVE FEDERALLY RECOGNIZED TRIBE       22
ASIAN - OTHER                                                  14
ASIAN - THAI                                                   10
Name: count, dtype: int64

Column: ADMISSION_LOCATION
ADMISSION_LOCATION
CLINIC REFERRAL/PREMATURE    47060
EMERGENCY ROOM ADMIT         34428
TRANSFER FROM HOSP/EXTRAM    20257
PHYS REFERRAL/NORMAL DELI    13021
TRANSFER FROM SKILLED NUR      766
TRANSFER FROM OTHER HEALT      351
Name: count, dtype: int64

Column: ADMISSION_TYPE
ADMISSION_TYPE
EMERGENCY    104403
ELECTIVE       9721
URGENT         1759
Name: count, dtype: int64

Column: ADMITTIME
ADMITTIME
2176-02-27 17:39:00    156
2156-10-23 11:47:00    156
2185-01-03 20:52:00    155
2144-09-19 05:09:00    152
2112-07-02 21:32:00    148
```

```
                        …
2172-06-25 14:22:00      5
2160-07-07 21:42:00      5
2143-11-12 14:39:00      4
2118-09-06 07:15:00      4
2189-10-29 17:11:00      4
Name: count, Length: 5496, dtype: int64

Column: GENDER
GENDER
M    61929
F    53954
Name: count, dtype: int64

Column: DOB
DOB
2120-10-31 00:00:00    461
2089-12-13 00:00:00    306
2055-03-02 00:00:00    265
2045-04-02 00:00:00    259
2111-04-12 00:00:00    257
                        …
2090-01-17 00:00:00      5
2167-04-22 00:00:00      5
2056-10-30 00:00:00      4
2157-02-10 00:00:00      4
2090-08-05 00:00:00      4
Name: count, Length: 4070, dtype: int64

Column: ICD9_CODE
ICD9_CODE
4280     6013
42731    2888
41401    2232
4019     2195
2724     2139
         …
80842       1
47825       1
44581       1
5304        1
83104       1
Name: count, Length: 3091, dtype: int64

 ====================================================
7. SUMMARY OF NUMERICAL VARIABLES
====================================================
```

```
Column: HADM_ID
Min: 100018, Max: 199972, Mean: 150569.20371409092, Std: 28865.67167514634

Column: LOS
Min: 0.0004, Max: 86.8484, Mean: 4.7005385492263745, Std: 5.886909831385161

Column: ICUSTAY_ID
Min: 200001, Max: 299957, Mean: 250037.16450212715, Std: 28855.057601593693

Column: SUBJECT_ID
Min: 34, Max: 99995, Mean: 56920.56710647808, Std: 26984.877531631708
```

[ ]:
```python
# Get rid of rows which have values in non-representative amounts
import pandas as pd

# Drop rows where insurance is 'government' or 'self pay'
df = df[~df['INSURANCE'].isin(['Government', 'Self Pay'])]

# Drop 'language' and 'religion' columns because they have too many unique␣
↪values and should not have predictive value
df = df.drop(columns=['LANGUAGE', 'RELIGION'])

# Handle 'MARITAL_STATUS' issues
# Drop NaN and rare values
df = df[~df['MARITAL_STATUS'].isin(['SEPARATED', 'UNKNOWN', 'LIFE PARTNER',␣
↪'UNKNOWN (DEFAULT)'])]

# Handle 'ETHNICITY' values
# Function to map ethnicities with partial matching
def map_ethnicity(ethnicity_str):
    if pd.isna(ethnicity_str):
        return "UNKNOWN"

    ethnicity_upper = str(ethnicity_str).upper()

    if 'WHITE' in ethnicity_upper or 'PORTUGUESE' in ethnicity_upper:
        return 'WHITE'
    elif 'BLACK' in ethnicity_upper or 'AFRICAN' in ethnicity_upper:
        return 'BLACK/AFRICAN AMERICAN'
    elif 'ASIAN' in ethnicity_upper:
        return 'ASIAN'
    elif 'HISPANIC' in ethnicity_upper or 'LATINO' in ethnicity_upper:
        return 'HISPANIC/LATINO'
    else:
        return 'OTHER'

# Apply the function to ethnicity column
```

```python
df['ETHNICITY'] = df['ETHNICITY'].apply(map_ethnicity)

# Drop rows with NAs or mapped as 'OTHER'
df = df[df['ETHNICITY'] != 'UNKNOWN']  # Drop rows with NA in mapped ethnicity
df = df[df['ETHNICITY'] != 'OTHER']  # Drop rows mapped as 'OTHER'

# Print the count to confirm
print(f"Original dataframe size: {len(df)}")
print(f"Cleaned dataframe size: {len(df)}")

# Handle 'ADMISSION_LOCATION'
# Drop rows with unwanted values
df = df[~df['ADMISSION_LOCATION'].isin([
    'TRANSFER FROM SKILLED NUR', 'TRANSFER FROM OTHER HEALT',
    'HMO REFERRAL/SICK', '** INFO NOT AVAILABLE **'
])]

# Handle 'ADMISSION_TYPE'
# Drop 'NEWBORN' and change 'URGENT' to 'EMERGENCY'
df = df[~df['ADMISSION_TYPE'].isin(['NEWBORN'])]
df['ADMISSION_TYPE'] = df['ADMISSION_TYPE'].replace('URGENT', 'EMERGENCY')

# Drop all remaining NAs
df=df.dropna()

# Final Check: Show the cleaned dataframe summary
print("="*50)
print("Data Cleaning Summary")
print("="*50)
print(df.info())
print("\nShape after cleaning:", df.shape)
```

```
Original dataframe size: 104146
Cleaned dataframe size: 104146
==================================================
Data Cleaning Summary
==================================================
<class 'pandas.core.frame.DataFrame'>
Index: 102167 entries, 405 to 705874
Data columns (total 15 columns):
 #   Column              Non-Null Count   Dtype
---  ------              --------------   -----
 0   HADM_ID             102167 non-null  int64
 1   LOS                 102167 non-null  float64
 2   ICUSTAY_ID          102167 non-null  int64
 3   INTIME              102167 non-null  object
 4   DBSOURCE            102167 non-null  object
```

```
 5   INSURANCE           102167 non-null  object
 6   MARITAL_STATUS      102167 non-null  object
 7   ETHNICITY           102167 non-null  object
 8   ADMISSION_LOCATION  102167 non-null  object
 9   ADMISSION_TYPE      102167 non-null  object
 10  SUBJECT_ID          102167 non-null  int64
 11  ADMITTIME           102167 non-null  object
 12  GENDER              102167 non-null  object
 13  DOB                 102167 non-null  object
 14  ICD9_CODE           102167 non-null  object
dtypes: float64(1), int64(3), object(11)
memory usage: 12.5+ MB
None

Shape after cleaning: (102167, 15)
```

[ ]:
```python
import pandas as pd
pd.options.mode.chained_assignment = None

# Convert object columns to datetime format
df['DOB'] = pd.to_datetime(df['DOB'])
df['INTIME'] = pd.to_datetime(df['INTIME'])
df['ADMITTIME'] = pd.to_datetime(df['ADMITTIME'])

# Calculate age at admission in years
df['AGE'] = df['INTIME'].dt.year - df['DOB'].dt.year
df['TIME_BETW_ADMISSION_ICU'] = (df['INTIME'] - df['ADMITTIME']).dt.
 ↪total_seconds() / 3600

# Drop DOB
df = df.drop(columns=['DOB'])
```

[ ]:
```python
# Print ICD9_CODE values occurring more than 1000 times and their counts
icd9_counts = df['ICD9_CODE'].value_counts()
frequent_icd9_codes = icd9_counts[icd9_counts > 1000]
print("ICD9 Codes occurring more than 1000 times:\n", frequent_icd9_codes)
print("\nNumber of such ICD9 codes:", len(frequent_icd9_codes))


# Print summary statistics for 'TIME_BETW_ADMISSION_ICU'
print("\nSummary statistics for TIME_BETW_ADMISSION_ICU:\n")
print(df['TIME_BETW_ADMISSION_ICU'].describe())
```

```
ICD9 Codes occurring more than 1000 times:
 ICD9_CODE
4280     5288
42731    2561
41401    1977
```

```
4019     1900
2724     1893
5849     1892
25000    1394
40390    1349
51881    1218
V5861    1128
5859     1103
Name: count, dtype: int64

Number of such ICD9 codes: 11

Summary statistics for TIME_BETW_ADMISSION_ICU:

count    102167.000000
mean         61.501572
std         157.756793
min         -35.765278
25%           0.022222
50%           0.081944
75%          50.679444
max        2530.475000
Name: TIME_BETW_ADMISSION_ICU, dtype: float64
```

[ ]:
```
# Only keep ICD9_Codes that have representative amounts
# Get frequent codes
icd9_counts = df['ICD9_CODE'].value_counts()
frequent_icd9_codes = icd9_counts[icd9_counts > 1000].index.astype(str)  #␣
↪Ensure string type

# Filter DataFrame
df = df[df['ICD9_CODE'].astype(str).isin(frequent_icd9_codes)]

# Round 'TIME_BETW_ADMISSION_ICU' to one decimal place
df['TIME_BETW_ADMISSION_ICU'] = df['TIME_BETW_ADMISSION_ICU'].round(1)

# Transform negative values in 'TIME_BETW_ADMISSION_ICU' to 0  (likely due to␣
↪reporting errors)
df['TIME_BETW_ADMISSION_ICU'] = df['TIME_BETW_ADMISSION_ICU'].apply(lambda x: 0␣
↪if x < 0 else x)
```

[ ]:
```
df['ICUSTAY_ID'].nunique()
```

[ ]: 5288

[ ]:
```
# Step 1: Sort to prioritize ICD9_CODE = 4280
df_sorted = df.sort_values(by=['ICUSTAY_ID', 'ICD9_CODE'],
```

```
                                key=lambda x: x != '4280')  # Puts 4280 first per␣
↪group

# Step 2: Keep first row per ICUSTAY_ID
df_base = df_sorted.drop_duplicates('ICUSTAY_ID', keep='first')

# Step 3: Create indicator columns for all frequent ICD9 codes
for code in frequent_icd9_codes:
    col_name = f"{code}_DIAGNOSIS"
    # Check if each ICUSTAY_ID has this diagnosis anywhere in original df
    has_diagnosis = df.groupby('ICUSTAY_ID')['ICD9_CODE'].apply(lambda x:␣
↪str(code) in x.values)
    df_base[col_name] = df_base['ICUSTAY_ID'].map(has_diagnosis)

# Step 4: Reset index (optional)
df_final = df_base.reset_index(drop=True)

# Assign new df to df
df = df_final
df = df.drop(columns=['4280_DIAGNOSIS'])
```

[ ]:
```
# Group by HADM_ID and count unique ICUSTAY_IDs
hadm_with_multiple_icu = df.groupby('HADM_ID')['ICUSTAY_ID'].nunique()

# Filter for HADM_IDs with >1 ICUSTAY_ID
hadm_multiple_stays = hadm_with_multiple_icu[hadm_with_multiple_icu > 1]

# Count the number of such HADM_IDs
num_hadm_multiple_stays = len(hadm_multiple_stays)

print(f"Number of HADM_IDs with >1 ICUSTAY_ID: {num_hadm_multiple_stays}")
```

```
Number of HADM_IDs with >1 ICUSTAY_ID: 387
```

[ ]:
```
# Step 1: Sort by HADM_ID and TIME_BETW_ADMISSION_ICU (ascending)
df_sorted = df.sort_values(['HADM_ID', 'TIME_BETW_ADMISSION_ICU'])

# Step 2: Assign sequential numbers within each HADM_ID group
df_sorted['NUMBER_ICUSTAY'] = df_sorted.groupby('HADM_ID').cumcount() + 1

# Step 3: Restore original order
df = df_sorted.sort_index()
```

[ ]:
```
df.to_csv('base_df.csv', index=False)
```

2.2 Build Dataframe for LOS Prediction

2.2.1 Textual Summary

We now focus on preparing a comprehensive dataframe for training a model to predict ICU Length of Stay (LOS). This process involves merging event-based time-series data from the tables chartevents, inputevents, and procedureevents with our base dataframe and crafting relevant features for each ICU stay.

For chartevents, we had 555 unique ITEMIDs (corresponding to measurements). To decide on which measurements to use for our analysis, we focussed on two goals. First, have a low number of NAs. Second, get measurements with predictive value. Therefore, we first printed the 10 most frequently recorded measurements. For example, **Heart Rate** (220045) and **Blood Pressure** readings dominated, which makes sense as they are core to ICU monitoring. Also **Oxygen saturation (SpO2)** and **Respiratory Rate** appear frequently, reflecting the respiratory focus of many ICU stays. Then, we generated a ranking of the top 10 measurements based on which are the **most widely available across ICU stays** — not just the most frequent overall. This helps us choose predictors that generalize well across patients. Otherwise, we might end up with vital signs that were recorded very often for single patients but never for other patients. The top features here include: - **Heart Rate** - **Respiratory Rate** - **SpO2** - **Alarm thresholds for HR and O2 saturation**

For our final decision, we combined the two rankings from before and computed the correlation of values for each of the measurements with the LOS. The 5 measurements with the highest absolute correlation coefficients were used as features for our prediction models. These included Heart Rate, Arterial Blood Pressure systolic, Central Venous Pressure, Non Invasive Blood Pressure mean, and Arterial Blood Pressure mean.

To get a better understanding of our variables, we visualized how frequently these items were recorded as well as at which time (in days since admission).Some variables, such as heart rate, were recorded very frequently, while others only appeared more sparsely during a stay. We could conclude that all the plots were right-skewed, being measurements with a long right tail (some ICU stays with a high count). Furthermore, we could see that most measurements were concentrated in the early ICU days, reflecting how clinicians prioritize vital signs during initial stabilization.

To reduce the complexity, we then aggregated the measurements on a daily basis. If one measurement was taken more than once during a day, we kept the median as the value for the day. Furthermore, we had to specify a window size for our features as we want to use the values for predicting LOS. If our model was to be used in practice, for example, we could not necessarily wait until the 10th day to predict how long the patient will stay (the patient might even be discharged by then!). We decided to keep the measurements within the first three days of the ICU stay as features in our dataframe as the average LOS is 4.7 days. In the model building process we will see if it could also be sufficient to just use data from the first or the first two days.

Aggregating on a daily basis allowed us to create features useful for classical ML models from the time series data. The features have the following format: ITEM ABBREVIATION_MEASUREMENT TIME, with the value indicating the value of the respective measurement on the specified day. However, not all variables are recorded at every time point for each ICU stay. This results in a high number of missing values, especially for less frequently measured variables or at later ICU days. It will be crucial to address this issue in the model building stage.

The same procedure was followed for the inputevents table which records the administered medication. The only differences in the processing were the following:

- The medication was aggregated on a daily basis, but not by median but by the full amount administered over the whole day. This accounts for differences in administration, which are not as indicative as the actual amount.
- NAs could easily be filled with 0 as 0 is meaningful in this case (0ml of the medication were administered).
- We kept the 10 input events with the highest correlation (instead of the top 5 as for chart events) because the correlation between input events and LOS was higher than for chart events.
- Some medication was administered over a longer time horizon than a day, wherefore this could skew our format based on "ITEM ABBREVIATION_DAY OF ADMINISTRATION". As this was, however, only true for a very small percentage of cases (2%) - we decided to ignore this slight inconsistency.

The two medications with the strongest correlations with LOS were Midazolam (Versed) (r = 0.14) and Potassium Chloride (KCl) (r = -0.13). Midazolam is a sedative commonly used in mechanically ventilated or agitated patients, who often require prolonged ICU stays due to critical illness. The positive correlation suggests that patients receiving Midazolam tend to stay longer, likely because they are in more severe condition (e.g., requiring sedation for intubation or managing delirium). For KCl, the negative correlation implies that patients receiving KCl supplementation have shorter ICU stays. This could be because KCl is often given for electrolyte imbalances (e.g., hypokalemia), which, when corrected, may lead to faster stabilization and discharge compared to patients with more complex conditions.

For procedure events, we also followed the same process as for chart events with the following caveats:

- Some procedure events were measured in time. For their values we could, therefore, easily compute the Pearson correlation coefficient with LOS to establish importance as before. Some events were, however, boolean values. For those, we calculated the point-biserial correlation.
- As for input events, we kept the top 10 highest correlated procedures instead of the top 5 (as for chart events) because the correlation between procedure events and LOS was higher than for chart events and LOS.
- Similar to input events, we could fill NAs with 0 (for both boolean and time-measured procedures) and used the sum as an aggregation function for time-measured procedures to transform to a daily feature.

The two most promising procedures (highest correlation with LOS) were chest X-Ray (r = 0.29) and Blood Cultures (r = 0.23). This might be because frequent imaging may reflect more severe or uncertain clinical cases, and blood cultures are often associated with suspected sepsis, which tends to prolong ICU stays.

The result of this section is the full dataframe that can be used for LOS prediction and contains information about the most important charted vital signs, administered drugs, as well as procedures during the first three days of the ICU stay.

2.2.2 Code

[ ]:
```
import pandas as pd
df=pd.read_csv('base_df.csv')
```

[ ]:
```
import pandas as pd
# We filtered chartevents locally to only include the rows with the ICUSTAY_IDs␣
↪from our base df due to memory restrictions on Google Colab
chartevents = pd.read_csv('CHARTEVENTS_ICU.csv', usecols=['ITEMID','VALUE',␣
↪'ICUSTAY_ID', 'CHARTTIME'])
```

```
<ipython-input-47-7c3e91e08e52>:3: DtypeWarning: Columns (4) have mixed types.
Specify dtype option on import or set low_memory=False.
  chartevents = pd.read_csv('CHARTEVENTS_ICU.csv', usecols=['ITEMID', 'VALUE',
'ICUSTAY_ID', 'CHARTTIME'])
```

[ ]:
```
# Merge df
df_LOS = df.merge(chartevents, on='ICUSTAY_ID', how='left')
df_LOS.head()
```

[ ]:
```
   HADM_ID    LOS  ICUSTAY_ID               INTIME    DBSOURCE INSURANCE  \
0   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
1   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
2   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
3   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
4   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare

  MARITAL_STATUS ETHNICITY         ADMISSION_LOCATION ADMISSION_TYPE  …  \
0        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
1        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
2        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
3        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
4        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …

   5849_DIAGNOSIS 25000_DIAGNOSIS 40390_DIAGNOSIS  51881_DIAGNOSIS  \
0            True            True           False            False
1            True            True           False            False
2            True            True           False            False
3            True            True           False            False
4            True            True           False            False

   V5861_DIAGNOSIS  5859_DIAGNOSIS  NUMBER_ICUSTAY    ITEMID  \
0            False            True               1  224562.0
1            False            True               1  224846.0
2            False            True               1  226253.0
3            False            True               1  220045.0
4            False            True               1  220179.0
```

```
              CHARTTIME  VALUE
0  2129-09-05 20:00:00    2.0
1  2129-09-05 20:00:00    2.0
2  2129-09-05 20:00:00   85.0
3  2129-09-05 21:00:00   95.0
4  2129-09-05 21:00:00  104.0

[5 rows x 30 columns]
```

[ ]:
```
df_LOS['ITEMID'].nunique()
```

[ ]: 555

[ ]:
```
d_items = pd.read_csv('D_ITEMS.csv')
```

[ ]:
```
value_counts = df_LOS['ITEMID'].value_counts()
num_frequent_items = (value_counts > 5000).sum()
print(f"Number of unique ITEMID values occurring more than 1000 times:␣
↪{num_frequent_items}")
```

```
Number of unique ITEMID values occurring more than 1000 times: 130
```

[ ]:
```
# Group by ICUSTAY_ID and get sets of ITEMID values
itemids_per_stay = df_LOS.groupby('ICUSTAY_ID')['ITEMID'].apply(set)

# Find the intersection of all sets - i.e., ITEMIDs that appear in every␣
↪ICUSTAY_ID
common_itemids = set.intersection(*itemids_per_stay)

print(f"ITEMIDs common to all ICUSTAY_IDs: {common_itemids}")
```

```
ITEMIDs common to all ICUSTAY_IDs: set()
```

[ ]:
```
top10_itemids = df_LOS['ITEMID'].value_counts().head(10)
print(top10_itemids)
top10_itemids_list = top10_itemids.index.tolist()

# Filter d_items where ITEMID is in the top list
matching_rows = d_items[d_items['ITEMID'].isin(top10_itemids_list)]

print(matching_rows)
```

```
ITEMID
220045.0    707148
220210.0    701224
220277.0    680282
220181.0    335654
220179.0    334512
```

```
220180.0    334331
220052.0    280954
220050.0    279216
220051.0    279152
220074.0    128804
Name: count, dtype: int64
       ROW_ID  ITEMID                                 LABEL ABBREVIATION  \
11498   12712  220045                            Heart Rate           HR
11502   12716  220050       Arterial Blood Pressure systolic         ABPs
11503   12717  220051      Arterial Blood Pressure diastolic         ABPd
11504   12718  220052           Arterial Blood Pressure mean         ABPm
11515   12729  220074                Central Venous Pressure          CVP
11520   12734  220179   Non Invasive Blood Pressure systolic         NBPs
11521   12735  220180  Non Invasive Blood Pressure diastolic         NBPd
11522   12736  220181       Non Invasive Blood Pressure mean         NBPm
11524   12738  220210                       Respiratory Rate           RR
12355   12746  220277            O2 saturation pulseoxymetry         SpO2

         DBSOURCE      LINKSTO             CATEGORY  UNITNAME PARAM_TYPE  \
11498  metavision  chartevents  Routine Vital Signs       bpm    Numeric
11502  metavision  chartevents  Routine Vital Signs      mmHg    Numeric
11503  metavision  chartevents  Routine Vital Signs      mmHg    Numeric
11504  metavision  chartevents  Routine Vital Signs      mmHg    Numeric
11515  metavision  chartevents         Hemodynamics      mmHg    Numeric
11520  metavision  chartevents  Routine Vital Signs      mmHg    Numeric
11521  metavision  chartevents  Routine Vital Signs      mmHg    Numeric
11522  metavision  chartevents  Routine Vital Signs      mmHg    Numeric
11524  metavision  chartevents          Respiratory  insp/min    Numeric
12355  metavision  chartevents          Respiratory         %    Numeric

       CONCEPTID
11498        NaN
11502        NaN
11503        NaN
11504        NaN
11515        NaN
11520        NaN
11521        NaN
11522        NaN
11524        NaN
12355        NaN
```

[ ]:
```python
# Drop duplicates so each ITEMID-ICUSTAY_ID pair is only counted once
unique_pairs = df_LOS[['ICUSTAY_ID', 'ITEMID']].drop_duplicates()

# Count how many unique ICUSTAY_IDs each ITEMID appears in
itemid_counts = unique_pairs['ITEMID'].value_counts()
```

```python
# Get the top 10
top10_itemids_unique = itemid_counts.head(10)
print(top10_itemids_unique)

top10_itemids_unique_list = top10_itemids_unique.index.tolist()

# Filter d_items where ITEMID is in the top list
matching_rows = d_items[d_items['ITEMID'].isin(top10_itemids_unique_list)]
print(matching_rows)
```

```
ITEMID
220045.0    5215
220210.0    5214
220277.0    5212
223770.0    5209
220046.0    5209
220047.0    5209
223769.0    5207
224162.0    5205
224161.0    5205
226253.0    5180
Name: count, dtype: int64
       ROW_ID  ITEMID                                     LABEL  \
9311    13047  224161                         Resp Alarm - High
9312    13048  224162                          Resp Alarm - Low
11498   12712  220045                                Heart Rate
11499   12713  220046                   Heart rate Alarm - High
11500   12714  220047                    Heart Rate Alarm - Low
11524   12738  220210                          Respiratory Rate
11926   14442  226253                          SpO2 Desat Limit
12355   12746  220277                O2 saturation pulseoxymetry
12374   12765  223769  O2 Saturation Pulseoxymetry Alarm - High
12375   12766  223770   O2 Saturation Pulseoxymetry Alarm - Low

            ABBREVIATION    DBSOURCE      LINKSTO             CATEGORY  \
9311   Resp Alarm - High  metavision  chartevents               Alarms
9312    Resp Alarm - Low  metavision  chartevents               Alarms
11498                 HR  metavision  chartevents  Routine Vital Signs
11499    HR Alarm - High  metavision  chartevents               Alarms
11500     HR Alarm - Low  metavision  chartevents               Alarms
11524                 RR  metavision  chartevents          Respiratory
11926   SpO2 Desat Limit  metavision  chartevents               Alarms
12355               SpO2  metavision  chartevents          Respiratory
12374  SpO2 Alarm - High  metavision  chartevents               Alarms
12375   SpO2 Alarm - Low  metavision  chartevents               Alarms
```

```
       UNITNAME PARAM_TYPE  CONCEPTID
9311   insp/min    Numeric        NaN
9312   insp/min    Numeric        NaN
11498       bpm    Numeric        NaN
11499       bpm    Numeric        NaN
11500       bpm    Numeric        NaN
11524  insp/min    Numeric        NaN
11926         %    Numeric        NaN
12355         %    Numeric        NaN
12374         %    Numeric        NaN
12375         %    Numeric        NaN
```

[ ]:
```
import pandas as pd

# Combine both lists and get unique ITEMIDs (in case there's overlap)
all_itemids = list(set(top10_itemids_unique_list + top10_itemids_list))

correlation_results = []

for itemid in all_itemids:
    # Step 1: Filter rows for the current ITEMID
    df_filtered = df_LOS[df_LOS['ITEMID'] == itemid]

    # Step 2: Compute correlation between VALUE and LOS
    correlation = df_filtered['VALUE'].corr(df_filtered['LOS'])

    # Store the result (including raw correlation and absolute value for␣
 ↪sorting)
    correlation_results.append((itemid, correlation, abs(correlation)))

# Step 3: Sort by ABSOLUTE correlation in descending order
correlation_results.sort(key=lambda x: x[2], reverse=True)  # x[2] is the␣
 ↪absolute correlation

# Get additional info from d_items
# Create a DataFrame for easier merging
corr_df = pd.DataFrame(
    [(itemid, corr) for itemid, corr, _ in correlation_results],  # Drop the␣
 ↪absolute value for display
    columns=['ITEMID', 'Correlation']
)

# Merge with d_items to get LABEL, ABBREVIATION, and UNITNAME
result_df = corr_df.merge(
    d_items[['ITEMID', 'LABEL', 'ABBREVIATION', 'UNITNAME']],
    on='ITEMID',
    how='left'
```

```
)

# Print the results in a formatted way
print("ITEMID\tCorrelation\tLABEL\tABBREVIATION\tUNITNAME")
print("--------------------------------------------------")
for _, row in result_df.iterrows():
    print(
        f"{row['ITEMID']}\t"
        f"{row['Correlation']:.4f}\t"
        f"{row['LABEL']}\t"
        f"{row['ABBREVIATION']}\t"
        f"{row['UNITNAME']}"
    )
```

```
ITEMID  Correlation     LABEL   ABBREVIATION    UNITNAME
--------------------------------------------------
220045.0        0.1996  Heart Rate      HR      bpm
220050.0        -0.0571 Arterial Blood Pressure systolic        ABPs    mmHg
220074.0        0.0548  Central Venous Pressure CVP     mmHg
220181.0        -0.0162 Non Invasive Blood Pressure mean        NBPm    mmHg
220052.0        -0.0126 Arterial Blood Pressure mean    ABPm    mmHg
220047.0        0.0083  Heart Rate Alarm - Low  HR Alarm - Low  bpm
224161.0        0.0061  Resp Alarm - High       Resp Alarm - High       insp/min
226253.0        -0.0045 SpO2 Desat Limit        SpO2 Desat Limit        %
223770.0        -0.0042 O2 Saturation Pulseoxymetry Alarm - Low SpO2 Alarm - Low
%
220046.0        -0.0033 Heart rate Alarm - High HR Alarm - High bpm
220179.0        -0.0030 Non Invasive Blood Pressure systolic    NBPs    mmHg
224162.0        0.0026  Resp Alarm - Low        Resp Alarm - Low        insp/min
220051.0        -0.0020 Arterial Blood Pressure diastolic       ABPd    mmHg
220277.0        0.0014  O2 saturation pulseoxymetry     SpO2    %
220180.0        0.0014  Non Invasive Blood Pressure diastolic   NBPd    mmHg
223769.0        -0.0008 O2 Saturation Pulseoxymetry Alarm - High        SpO2
Alarm - High       %
220210.0        -0.0005 Respiratory Rate        RR      insp/min
```

[ ]:
```
# --- Extract top 5 ITEMIDs ---
top5_itemids = [itemid for itemid, _, _ in correlation_results[:5]]
print("Top 5 ITEMIDs by absolute correlation:", top5_itemids)

# --- Filter df_LOS to keep only top5 ITEMIDs or NaN ---
df_LOS = df_LOS[df_LOS['ITEMID'].isin(top5_itemids) | df_LOS['ITEMID'].isna()]

# --- (Optional) Print summary of filtered data ---
df_LOS.head()
```

```
Top 5 ITEMIDs by absolute correlation: [220045.0, 220050.0, 220074.0, 220181.0,
220052.0]
```

[ ]:
```
    HADM_ID    LOS  ICUSTAY_ID               INTIME    DBSOURCE INSURANCE  \
3    194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
6    194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
9    194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
12   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare
17   194340  2.437      217232  2129-09-03 12:31:31  metavision  Medicare

   MARITAL_STATUS ETHNICITY         ADMISSION_LOCATION ADMISSION_TYPE  …  \
3         MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
6         MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
9         MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
12        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
17        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …

    5849_DIAGNOSIS 25000_DIAGNOSIS 40390_DIAGNOSIS  51881_DIAGNOSIS  \
3             True            True           False            False
6             True            True           False            False
9             True            True           False            False
12            True            True           False            False
17            True            True           False            False

    V5861_DIAGNOSIS  5859_DIAGNOSIS  NUMBER_ICUSTAY    ITEMID  \
3             False            True               1  220045.0
6             False            True               1  220181.0
9             False            True               1  220045.0
12            False            True               1  220181.0
17            False            True               1  220045.0

              CHARTTIME  VALUE
3   2129-09-05 21:00:00   95.0
6   2129-09-05 21:00:00   58.0
9   2129-09-05 22:00:00  105.0
12  2129-09-05 22:00:00   64.0
17  2129-09-03 12:31:00  104.0

[5 rows x 30 columns]
```

[ ]:
```python
df_LOS['CHARTTIME'] = pd.to_datetime(df_LOS['CHARTTIME'])
df_LOS['INTIME'] = pd.to_datetime(df_LOS['INTIME'])
df_LOS['CHARTTIME_DAYS'] = round((df_LOS['CHARTTIME'] - df_LOS['INTIME']).dt.
 ↪total_seconds() / (3600*24),0)
df_LOS[["CHARTTIME_DAYS","CHARTTIME","INTIME"]]
```

[ ]:
```
         CHARTTIME_DAYS           CHARTTIME              INTIME
3                   2.0 2129-09-05 21:00:00 2129-09-03 12:31:31
6                   2.0 2129-09-05 21:00:00 2129-09-03 12:31:31
9                   2.0 2129-09-05 22:00:00 2129-09-03 12:31:31
```

```
12                  2.0 2129-09-05 22:00:00 2129-09-03 12:31:31
17                 -0.0 2129-09-03 12:31:00 2129-09-03 12:31:31
…                  …                   …                   …
8304914             1.0 2179-12-26 14:00:00 2179-12-25 19:07:29
8304918             1.0 2179-12-26 15:00:00 2179-12-25 19:07:29
8304919             1.0 2179-12-26 15:00:00 2179-12-25 19:07:29
8304922             1.0 2179-12-26 16:00:00 2179-12-25 19:07:29
8304925             1.0 2179-12-26 16:00:00 2179-12-25 19:07:29

[1731836 rows x 3 columns]
```

[ ]:
```python
print("\nSummary statistics for CHARTTIME_DAYS:\n")
print(df_LOS['CHARTTIME_DAYS'].describe())
df_LOS.drop(columns=['CHARTTIME'], inplace=True)
```

```
Summary statistics for CHARTTIME_DAYS:

count    1.731776e+06
mean     5.482534e+00
std      6.914606e+00
min      0.000000e+00
25%      1.000000e+00
50%      3.000000e+00
75%      7.000000e+00
max      5.300000e+01
Name: CHARTTIME_DAYS, dtype: float64
```

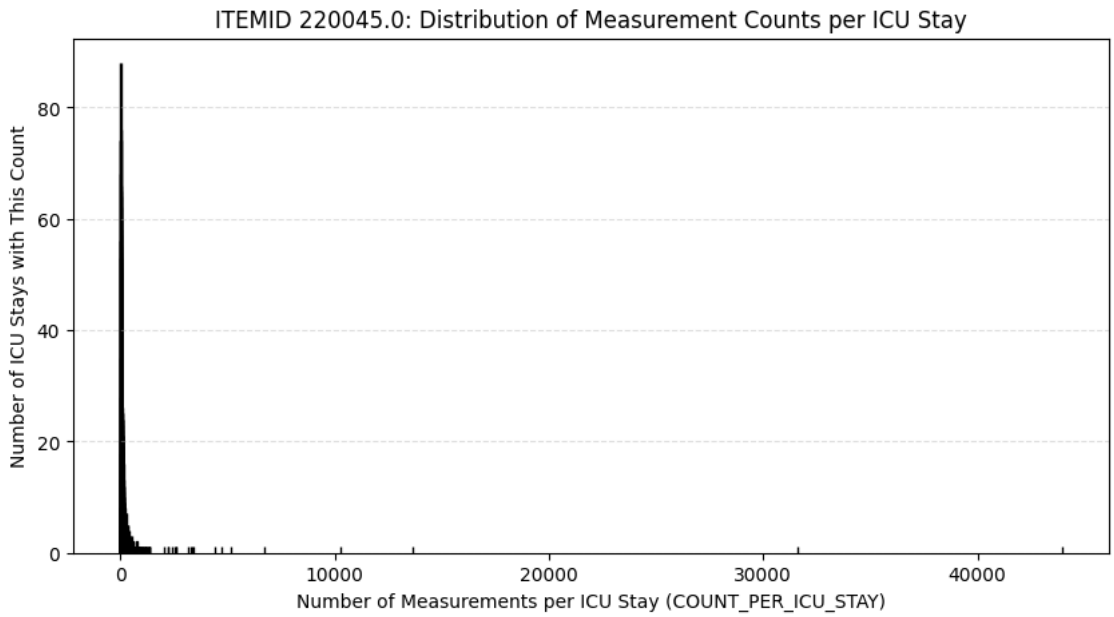
[ ]:
```python
df_LOS.head()
```

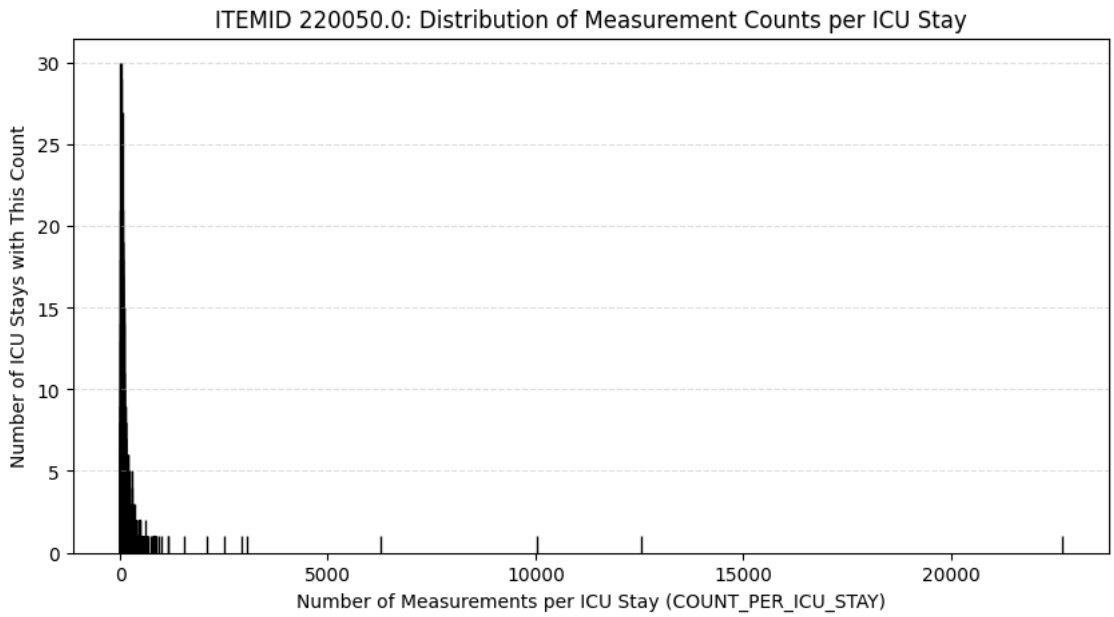
[ ]:
```
    HADM_ID    LOS  ICUSTAY_ID              INTIME    DBSOURCE INSURANCE  \
3    194340  2.437      217232 2129-09-03 12:31:31  metavision  Medicare
6    194340  2.437      217232 2129-09-03 12:31:31  metavision  Medicare
9    194340  2.437      217232 2129-09-03 12:31:31  metavision  Medicare
12   194340  2.437      217232 2129-09-03 12:31:31  metavision  Medicare
17   194340  2.437      217232 2129-09-03 12:31:31  metavision  Medicare

   MARITAL_STATUS ETHNICITY         ADMISSION_LOCATION ADMISSION_TYPE  …  \
3         MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
6         MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
9         MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
12        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …
17        MARRIED     WHITE  TRANSFER FROM HOSP/EXTRAM      EMERGENCY  …

    5849_DIAGNOSIS 25000_DIAGNOSIS 40390_DIAGNOSIS  51881_DIAGNOSIS  \
3             True            True           False            False
6             True            True           False            False
```

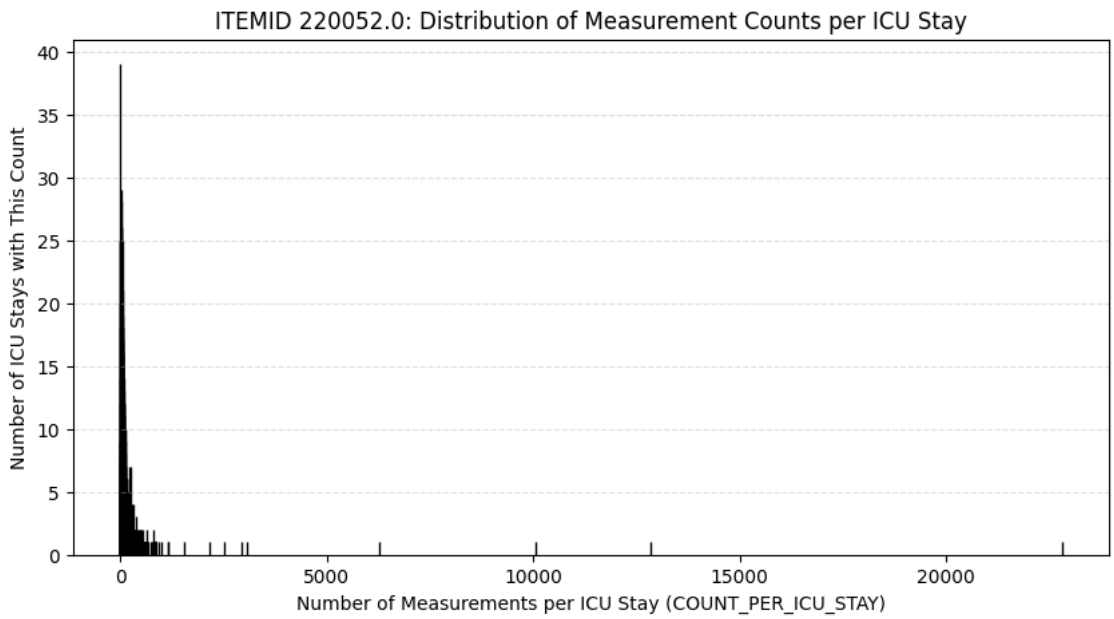
```
9             True            True           False            False
12            True            True           False            False
17            True            True           False            False

    V5861_DIAGNOSIS  5859_DIAGNOSIS  NUMBER_ICUSTAY    ITEMID  VALUE  \
3             False            True               1  220045.0   95.0
6             False            True               1  220181.0   58.0
9             False            True               1  220045.0  105.0
12            False            True               1  220181.0   64.0
17            False            True               1  220045.0  104.0

    CHARTTIME_DAYS
3              2.0
6              2.0
9              2.0
12             2.0
17            -0.0

[5 rows x 30 columns]
```

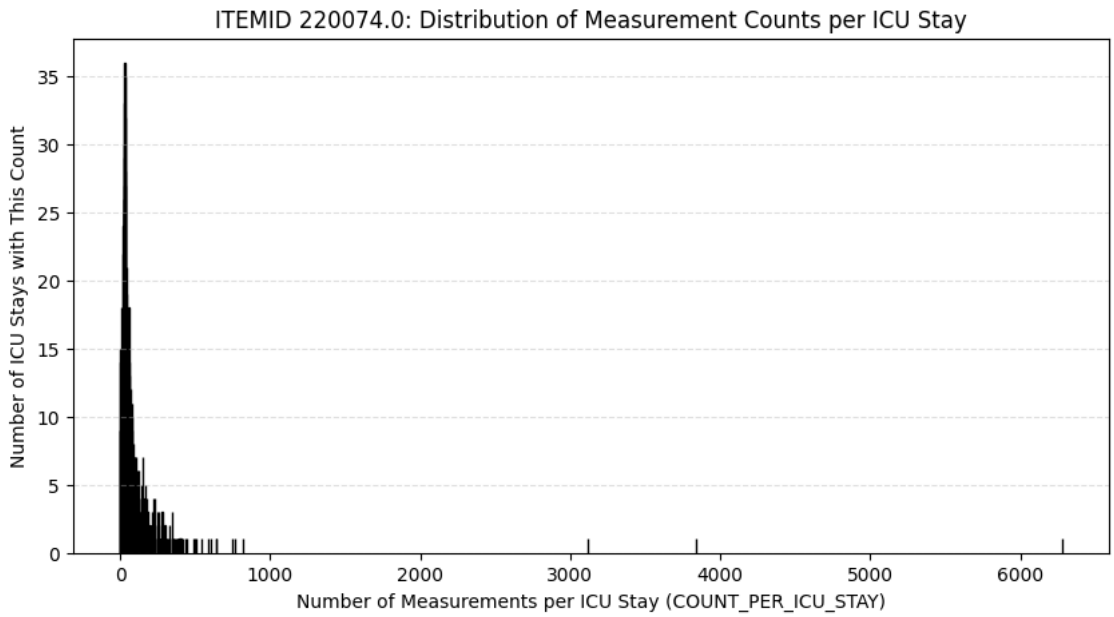
[ ]:
```python
import pandas as pd
import matplotlib.pyplot as plt

# Generate the result DataFrame
itemid_icu_counts = (
    df_LOS.groupby(['ITEMID', 'ICUSTAY_ID'])
    .size()
    .reset_index(name='COUNT_PER_ICU_STAY')
)
result = (
    itemid_icu_counts.groupby(['ITEMID', 'COUNT_PER_ICU_STAY'])
    .size()
    .reset_index(name='NUMBER_OF_ICU_STAYS_WITH_THIS_COUNT')
    .sort_values(['ITEMID', 'COUNT_PER_ICU_STAY'])
)

# Plot histograms for each ITEMID
for itemid in result['ITEMID'].unique():
    # Filter data for the current ITEMID
    subset = result[result['ITEMID'] == itemid]

    # Create the plot
    plt.figure(figsize=(10, 5))
    plt.bar(
        subset['COUNT_PER_ICU_STAY'],
        subset['NUMBER_OF_ICU_STAYS_WITH_THIS_COUNT'],
        color='skyblue',
```

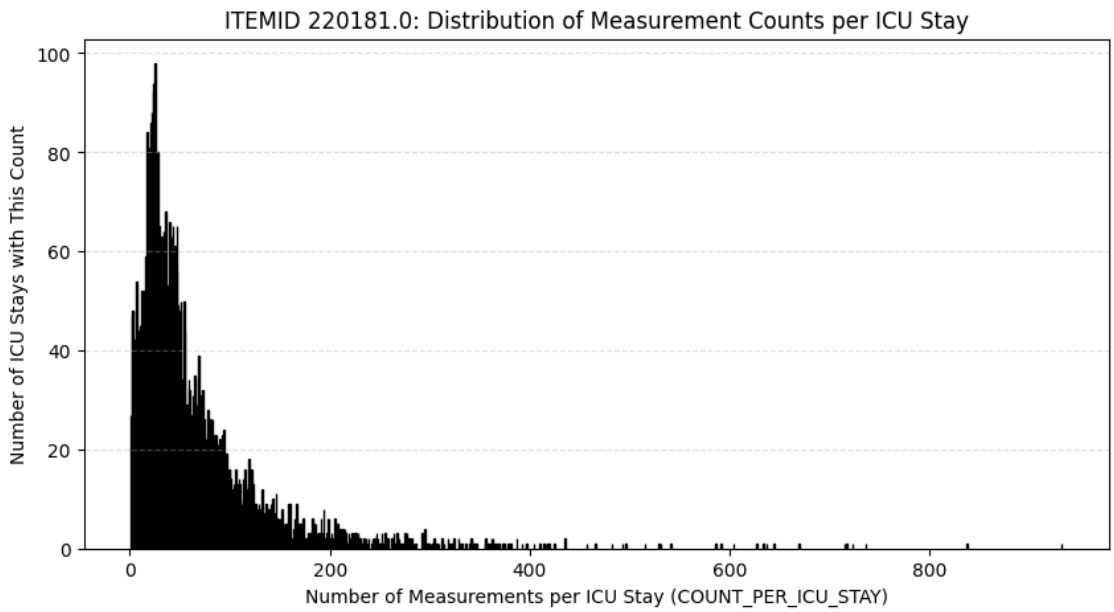
```
        edgecolor='black'
    )

    # Customize the plot
    plt.title(f'ITEMID {itemid}: Distribution of Measurement Counts per ICU␣
↪Stay')
    plt.xlabel('Number of Measurements per ICU Stay (COUNT_PER_ICU_STAY)')
    plt.ylabel('Number of ICU Stays with This Count')
    plt.grid(axis='y', linestyle='--', alpha=0.4)

    # Show plot
    plt.show()
```




ITEMID 220050.0: Distribution of Measurement Counts per ICU Stay
Number of ICU Stays with This Count
Number of Measurements per ICU Stay (COUNT_PER_ICU_STAY)
0
5
10
15
20
25
30
0
5000
10000
15000
20000

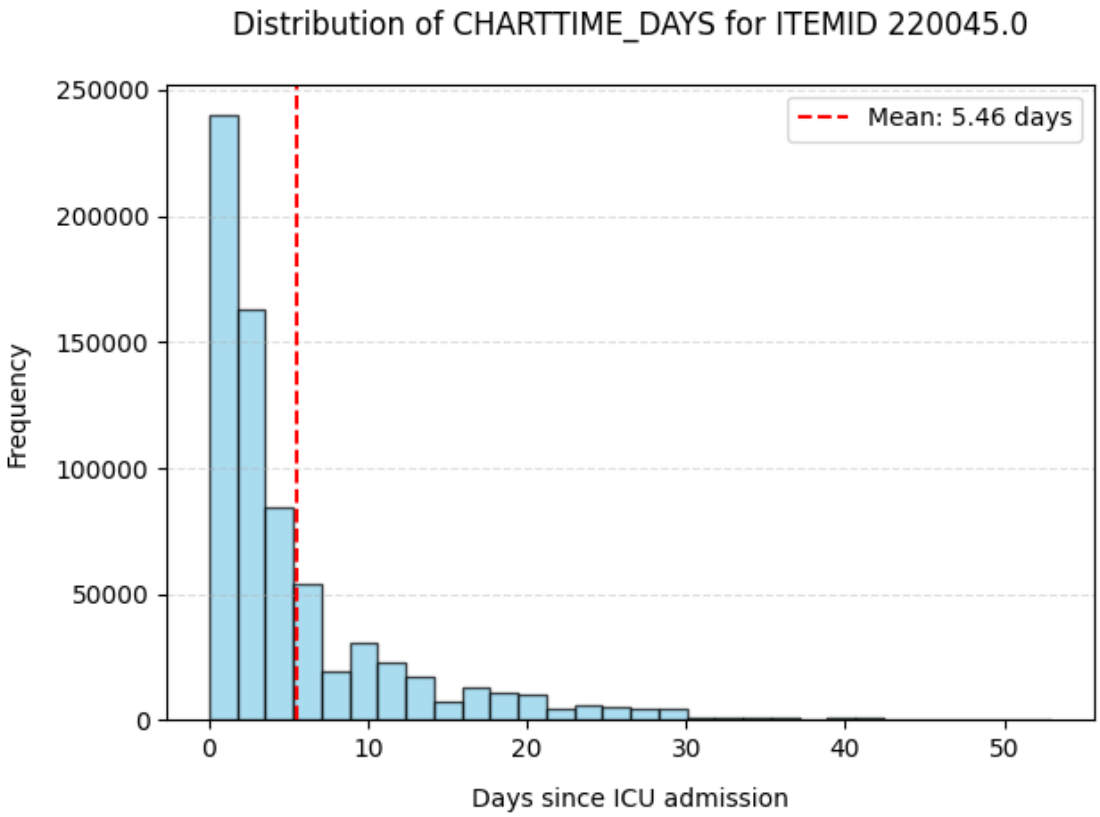

ITEMID 220052.0: Distribution of Measurement Counts per ICU Stay
Number of ICU Stays with This Count
Number of Measurements per ICU Stay (COUNT_PER_ICU_STAY)
0
5
10
15
20
25
30
35
40
0
5000
10000
15000
20000

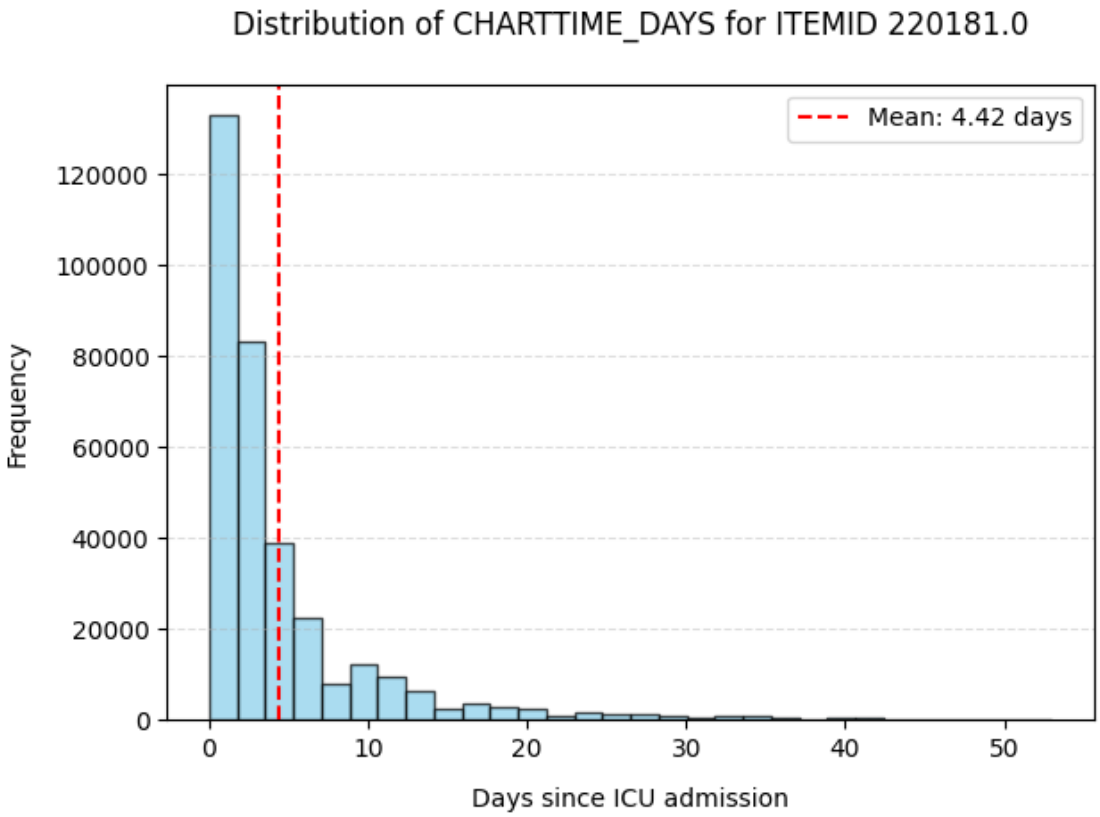


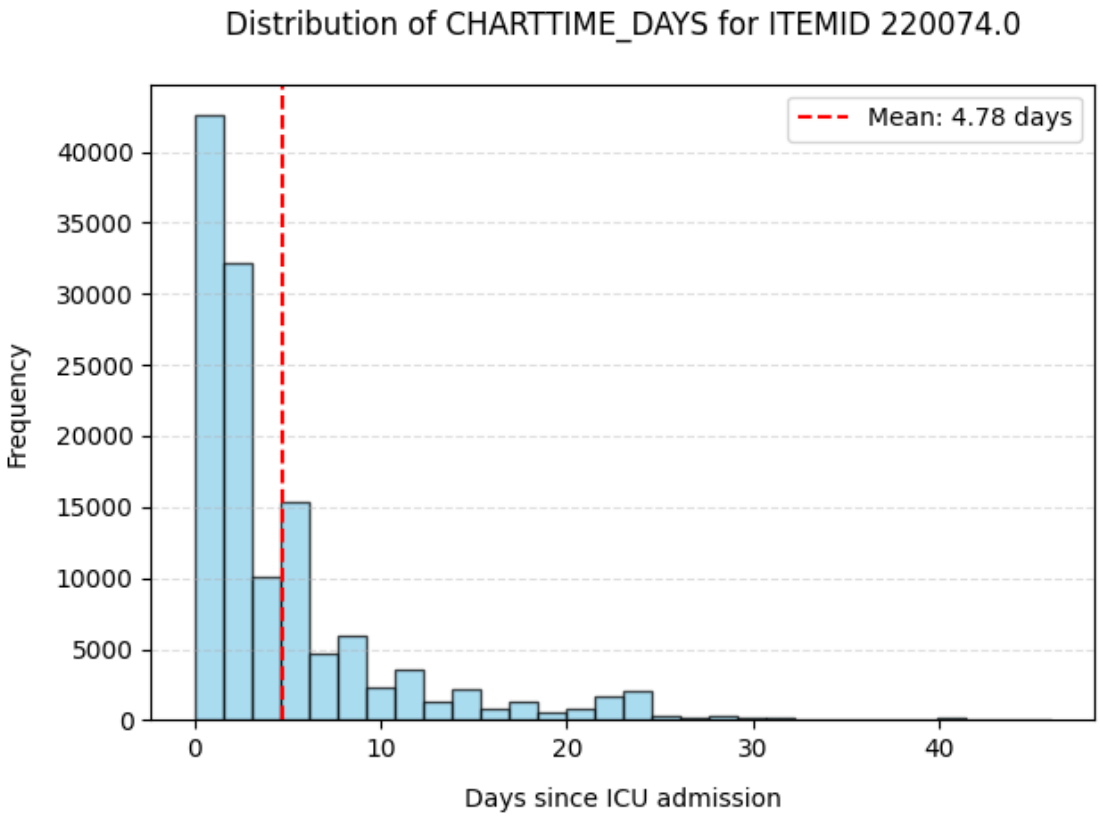


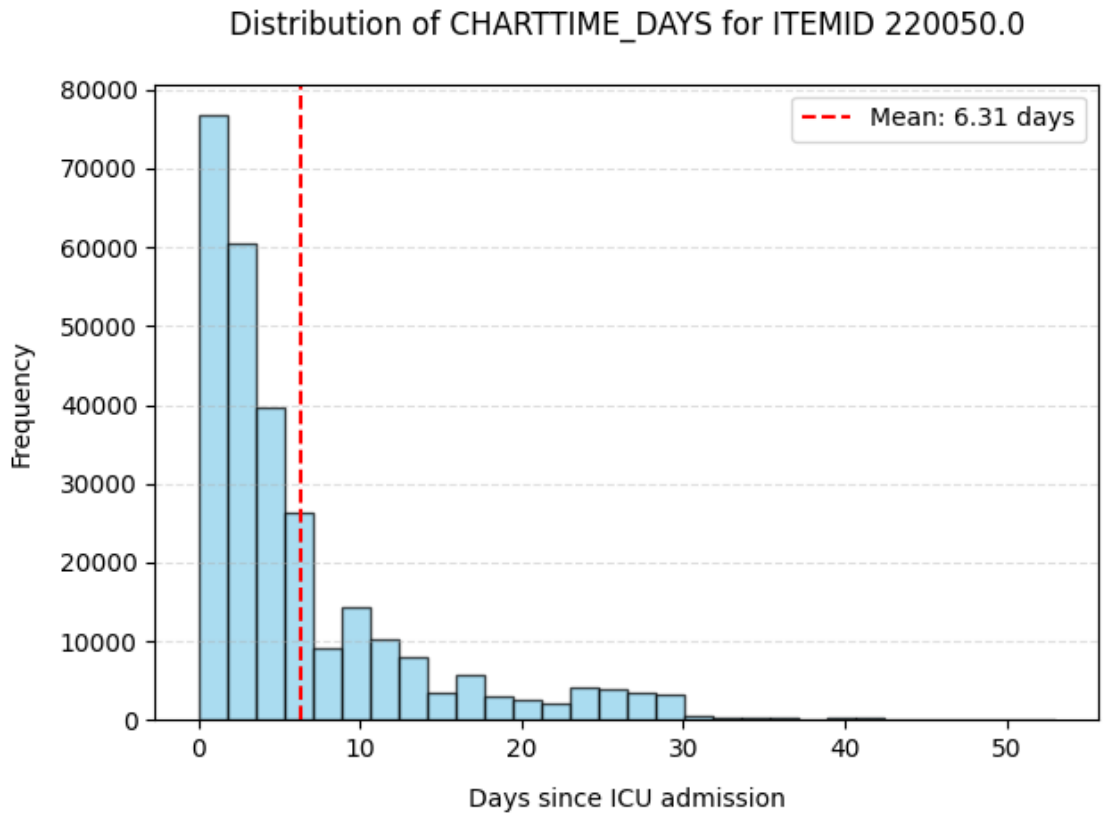
[ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

# Get all unique ITEMIDs
unique_itemids = df_LOS['ITEMID'].unique()
```

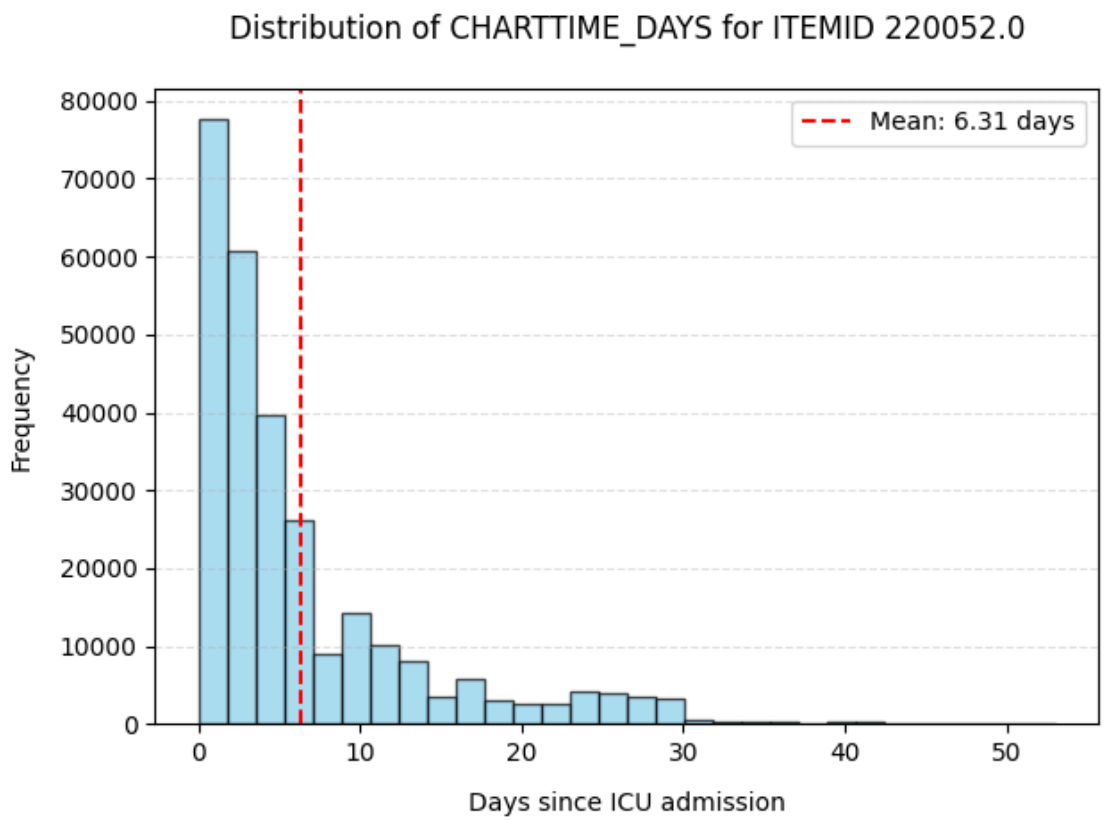
```python
# Create histograms for each ITEMID
for itemid in unique_itemids:
    # Filter data for current ITEMID
    item_data = df_LOS[df_LOS['ITEMID'] == itemid]['CHARTTIME_DAYS']

    # Skip if no data
    if len(item_data) == 0:
        continue

    # Create figure
    fig, ax = plt.subplots()

    # Plot histogram
    n, bins, patches = ax.hist(item_data, bins=30, color='skyblue',␣
 ↪edgecolor='black', alpha=0.7)

    # Add labels and title
    ax.set_title(f'Distribution of CHARTTIME_DAYS for ITEMID {itemid}', pad=20)
    ax.set_xlabel('Days since ICU admission', labelpad=10)
    ax.set_ylabel('Frequency', labelpad=10)

    # Add grid
    ax.grid(axis='y', linestyle='--', alpha=0.4)

    # Add mean line
    mean_val = item_data.mean()
    ax.axvline(mean_val, color='red', linestyle='dashed', linewidth=1.5,
               label=f'Mean: {mean_val:.2f} days')

    # Add legend
    ax.legend()

    # Adjust layout
    plt.tight_layout()

    # Show plot
    plt.show()
```


Distribution of CHARTTIME_DAYS for ITEMID 220045.0
Mean: 5.46 days
Frequency
250000
200000
150000
100000
50000
0
0
10
20
30
40
50
Days since ICU admission


Distribution of CHARTTIME_DAYS for ITEMID 220181.0
Mean: 4.42 days
Frequency
120000
100000
80000
60000
40000
20000
0
0
10
20
30
40
50
Days since ICU admission


Distribution of CHARTTIME_DAYS for ITEMID 220074.0
Mean: 4.78 days
Frequency
40000
35000
30000
25000
20000
15000
10000
5000
0
0
10
20
30
40
Days since ICU admission


Distribution of CHARTTIME_DAYS for ITEMID 220050.0
Mean: 6.31 days
Frequency
80000
70000
60000
50000
40000
30000
20000
10000
0
0
10
20
30
40
50
Days since ICU admission



[ ]:
```
print(df_LOS['LOS'].describe())
```

```
count    1.731836e+06
mean     1.171524e+01
std      1.062480e+01
min      4.000000e-04
25%      3.435700e+00
50%      8.070100e+00
75%      1.661940e+01
max      5.325210e+01
Name: LOS, dtype: float64
```

[ ]:
```
import pandas as pd

# Convert VALUE to numeric (handling non-numeric values if needed)
df_LOS['VALUE'] = pd.to_numeric(df_LOS['VALUE'], errors='coerce')

# Create a dictionary for aggregation - median for VALUE, first for all other␣
↪columns
agg_dict = {'VALUE': 'median'}
other_columns = [col for col in df_LOS.columns
```

```
                if col not in ['ICUSTAY_ID', 'ITEMID', 'CHARTTIME_DAYS',␣
 ↪'VALUE']]
agg_dict.update({col: 'first' for col in other_columns})

# Perform the grouped aggregation
df_LOS = df_LOS.groupby(
    ['ICUSTAY_ID', 'ITEMID', 'CHARTTIME_DAYS'],
    as_index=False
).agg(agg_dict)
```

[ ]:
```
df_LOS["CHARTTIME_DAYS"].unique()
```

[ ]:
```
array([-0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10., 11., 12.,
       13., 14., 15., 16., 17., 18., 19., 20., 21., 22., 23., 24., 25.,
       26., 27., 28., 29., 30., 31., 32., 33., 34., 35., 36., 37., 38.,
       39., 40., 41., 42., 43., 44., 45., 46., 47., 48., 49., 50., 51.,
       52., 53.])
```

[ ]:
```
df_LOS = df_LOS[df_LOS['CHARTTIME_DAYS'] <= 3]
```

[ ]:
```
# Step 1: Create a combined column for ITEMID + CHARTTIME_DAYS
df_LOS['ITEMID_CHARTTIME'] = df_LOS['ITEMID'].astype(str) + '_' +␣
 ↪df_LOS['CHARTTIME_DAYS'].astype(str)

# Step 2: Pivot the table to get one row per ICUSTAY_ID and columns for each␣
 ↪ITEMID_CHARTTIME
df_pivoted = df_LOS.pivot_table(
    index='ICUSTAY_ID',  # Keep ICUSTAY_ID as the index (one row per ID)
    columns='ITEMID_CHARTTIME',  # New columns will be ITEMID_DAY combinations
    values='VALUE',  # Fill cells with the VALUE
    aggfunc='first'  # Take the first value if duplicates exist (or 'median' if␣
 ↪needed)
).reset_index()  # Reset index to make ICUSTAY_ID a column again

# Step 3: (Optional) Rename columns if needed (e.g., remove any multi-indexing)
df_pivoted.columns.name = None  # Remove the 'ITEMID_CHARTTIME' label from␣
 ↪columns
```

[ ]:
```
df_pivoted.head()
```

[ ]:
```
   ICUSTAY_ID  220045.0_-0.0  220045.0_1.0  220045.0_2.0  220045.0_3.0  \
0      200001          103.0          94.0          81.0          71.0
1      200011           70.0          73.0          76.5           NaN
2      200024          115.0           NaN           NaN           NaN
3      200028           77.0          95.5          82.0          72.0
4      200038           77.0          77.5           NaN           NaN
```

```
   220050.0_-0.0  220050.0_1.0  220050.0_2.0  220050.0_3.0  220052.0_-0.0  \
0            NaN         113.5         114.0         107.5            NaN
1            NaN           NaN           NaN           NaN            NaN
2           93.5           NaN           NaN           NaN           79.0
3          106.5         120.0         138.0           NaN           64.0
4            NaN           NaN           NaN           NaN            NaN

   …  220052.0_2.0  220052.0_3.0  220074.0_-0.0  220074.0_1.0  220074.0_2.0  \
0  …          70.0          69.5            NaN           NaN           NaN
1  …           NaN           NaN            NaN           NaN           NaN
2  …           NaN           NaN            NaN           NaN           NaN
3  …          81.0           NaN           13.0           5.5           NaN
4  …           NaN           NaN            NaN           NaN           NaN

   220074.0_3.0  220181.0_-0.0  220181.0_1.0  220181.0_2.0  220181.0_3.0
0           NaN           70.0          65.5           NaN          62.5
1           NaN           76.0          90.0          92.0           NaN
2           NaN           81.0           NaN           NaN           NaN
3           NaN           59.0           NaN          70.0          76.0
4           NaN           89.0          91.0           NaN           NaN

[5 rows x 21 columns]
```

[ ]:
```
import numpy as np

# Get all columns that might have -0.0 or 0.0 variants
cols = df_pivoted.columns

# Dictionary to track columns to drop (those ending with -0.0)
columns_to_drop = []

# Iterate through columns to find matching pairs
for col in cols:
    if col.endswith('-0.0'):
        # Find the corresponding column without -0.0
        base_col = col[:-4] + '0.0'  # Replace '-0.0' with '0.0'

        if base_col in df_pivoted.columns:
            # Compute median, preferring non-NaN values
            df_pivoted[base_col] = np.where(
                df_pivoted[col].isna(),
                df_pivoted[base_col],  # If col is NaN, keep base_col
                np.where(
                    df_pivoted[base_col].isna(),
                    df_pivoted[col],  # If base_col is NaN, keep col
                    (df_pivoted[col] + df_pivoted[base_col]) / 2  # Median of␣
 ↪two values
```

```
                )
            )
            # Mark the -0.0 column for removal
            columns_to_drop.append(col)

# Drop the redundant columns (-0.0 versions)
df_pivoted = df_pivoted.drop(columns=columns_to_drop)
```

[ ]:
```
# Create new column names by removing the last two characters
# except for "ICUSTAY_ID"
new_columns = {}

for col in df_pivoted.columns:
    if col != "ICUSTAY_ID":
        # Remove the last two characters from the column name
        new_columns[col] = col[:-2]
    else:
        # Keep "ICUSTAY_ID" unchanged
        new_columns[col] = col

# Rename the columns
df_pivoted = df_pivoted.rename(columns=new_columns)

# Display the first few rows to confirm changes
print(df_pivoted.head())
```

```
   ICUSTAY_ID  220045.0_-0  220045.0_1  220045.0_2  220045.0_3  220050.0_-0  \
0      200001        103.0        94.0        81.0        71.0          NaN
1      200011         70.0        73.0        76.5         NaN          NaN
2      200024        115.0         NaN         NaN         NaN         93.5
3      200028         77.0        95.5        82.0        72.0        106.5
4      200038         77.0        77.5         NaN         NaN          NaN

   220050.0_1  220050.0_2  220050.0_3  220052.0_-0  …  220052.0_2  \
0       113.5       114.0       107.5          NaN  …        70.0
1         NaN         NaN         NaN          NaN  …         NaN
2         NaN         NaN         NaN         79.0  …         NaN
3       120.0       138.0         NaN         64.0  …        81.0
4         NaN         NaN         NaN          NaN  …         NaN

   220052.0_3  220074.0_-0  220074.0_1  220074.0_2  220074.0_3  220181.0_-0  \
0        69.5          NaN         NaN         NaN         NaN         70.0
1         NaN          NaN         NaN         NaN         NaN         76.0
2         NaN          NaN         NaN         NaN         NaN         81.0
3         NaN         13.0         5.5         NaN         NaN         59.0
4         NaN          NaN         NaN         NaN         NaN         89.0
```

```
   220181.0_1  220181.0_2  220181.0_3
0        65.5         NaN        62.5
1        90.0        92.0         NaN
2         NaN         NaN         NaN
3         NaN        70.0        76.0
4        91.0         NaN         NaN

[5 rows x 21 columns]
```

[ ]:
```
# Create a dictionary mapping ITEMIDs to their abbreviations from d_items
itemid_to_abbrev = {}
for _, row in d_items.iterrows():
    itemid_to_abbrev[str(row['ITEMID'])] = row['ABBREVIATION']

# Create new column names by replacing the first 8 characters with the␣
↪corresponding abbreviation
new_columns = {}

for col in df_pivoted.columns:
    if col != "ICUSTAY_ID":
        # Extract the first 8 characters (which represent the ITEMID)
        itemid_str = col[:8]

        # Remove the decimal point if present and convert to match the integer␣
↪format in d_items
        if '.' in itemid_str:
            itemid_str = itemid_str.split('.')[0]

        # Look for the abbreviation in the mapping
        if itemid_str in itemid_to_abbrev:
            # Replace the first 8 characters with the abbreviation
            new_columns[col] = d_items.loc[d_items['ITEMID'] ==␣
↪int(itemid_str), 'ABBREVIATION'].values[0] + col[8:]
        else:
            # Keep the original column name if no match is found
            new_columns[col] = col
    else:
        # Keep "ICUSTAY_ID" unchanged
        new_columns[col] = col

# Create a new dictionary with "-0" replaced by "0"
updated_columns = {}
for old_col, new_col in new_columns.items():
    if "_-0" in new_col:
        updated_columns[old_col] = new_col.replace("_-0", "_0")
    else:
        updated_columns[old_col] = new_col
```

[ ]:
```
# Rename the columns
df_pivoted = df_pivoted.rename(columns=updated_columns)

# Drop unprocessed ITEM columns from df_LOS
df_LOS.
 ↪drop(columns=["ITEMID","VALUE","CHARTTIME_DAYS","ITEMID_CHARTTIME"],inplace=True)
df_LOS = df_LOS.drop_duplicates()

# Merge df with the processed ITEM columns
df_LOS = df_LOS.merge(df_pivoted, on='ICUSTAY_ID', how='left')
```

[ ]:
```
# Returns True if duplicates exist in ICUSTAY_ID
has_duplicates = df_LOS['ICUSTAY_ID'].duplicated().any()
print(f"Has duplicates in ICUSTAY_ID: {has_duplicates}")
```

```
Has duplicates in ICUSTAY_ID: False
```

[ ]:
```
df_LOS.head()
```

[ ]:
```
   ICUSTAY_ID  HADM_ID     LOS              INTIME    DBSOURCE INSURANCE  \
0      200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
1      200011   121562  1.6330 2188-08-06 01:39:24  metavision  Medicare
2      200024   179633  0.3812 2127-03-03 16:09:07  metavision  Medicare
3      200028   181955  2.9038 2133-10-29 17:13:50  metavision  Medicare
4      200038   175436  1.0588 2143-10-24 20:35:24  metavision  Medicare

  MARITAL_STATUS               ETHNICITY         ADMISSION_LOCATION  \
0         SINGLE                   ASIAN  CLINIC REFERRAL/PREMATURE
1         SINGLE                   WHITE  CLINIC REFERRAL/PREMATURE
2        MARRIED  BLACK/AFRICAN AMERICAN       EMERGENCY ROOM ADMIT
3        MARRIED                   WHITE  PHYS REFERRAL/NORMAL DELI
4        MARRIED                   WHITE       EMERGENCY ROOM ADMIT

  ADMISSION_TYPE  …  ABPm_2 ABPm_3 CVP_0  CVP_1  CVP_2  CVP_3  NBPm_0  \
0      EMERGENCY  …    70.0   69.5   NaN    NaN    NaN    NaN    70.0
1      EMERGENCY  …     NaN    NaN   NaN    NaN    NaN    NaN    76.0
2      EMERGENCY  …     NaN    NaN   NaN    NaN    NaN    NaN    81.0
3       ELECTIVE  …    81.0    NaN  13.0    5.5    NaN    NaN    59.0
4      EMERGENCY  …     NaN    NaN   NaN    NaN    NaN    NaN    89.0

   NBPm_1  NBPm_2  NBPm_3
0    65.5     NaN    62.5
1    90.0    92.0     NaN
2     NaN     NaN     NaN
3     NaN    70.0    76.0
4    91.0     NaN     NaN
```

```
[5 rows x 47 columns]
```

[ ]:
```python
df_LOS.to_csv('base_df_LOS_chartevents.csv', index=False)
```

[ ]:
```python
import pandas as pd
# We filtered inputevents locally to only include the rows with the ICUSTAY_IDs␣
↪from our base df due to memory restrictions on Google Colab
df_LOS = pd.read_csv('base_df_LOS_chartevents.csv')
inputevents = pd.read_csv('INPUTEVENTS_MV.
↪csv',usecols=['ICUSTAY_ID','STARTTIME', 'ENDTIME', 'ITEMID',␣
↪'AMOUNT','RATE'])
d_items = pd.read_csv('D_ITEMS.csv')
```

[ ]:
```python
# Merge df
df_LOS = df_LOS.merge(inputevents, on='ICUSTAY_ID', how='left')
df_LOS.head()
```

[ ]:
```
   ICUSTAY_ID  HADM_ID     LOS               INTIME    DBSOURCE INSURANCE  \
0      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare
1      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare
2      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare
3      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare
4      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare

  MARITAL_STATUS ETHNICITY         ADMISSION_LOCATION ADMISSION_TYPE  …  \
0         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
1         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
2         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
3         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
4         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …

   CVP_3 NBPm_0 NBPm_1  NBPm_2  NBPm_3            STARTTIME  \
0    NaN   70.0   65.5     NaN    62.5  2181-11-26 12:00:00
1    NaN   70.0   65.5     NaN    62.5  2181-11-26 12:00:00
2    NaN   70.0   65.5     NaN    62.5  2181-11-26 11:33:00
3    NaN   70.0   65.5     NaN    62.5  2181-11-27 01:57:00
4    NaN   70.0   65.5     NaN    62.5  2181-11-27 01:57:00

               ENDTIME    ITEMID      AMOUNT       RATE
0  2181-11-26 18:00:00  228339.0  360.000000   1.000000
1  2181-11-26 18:00:00  220949.0  299.999992  49.999999
2  2181-11-26 11:34:00  221347.0    0.000000        NaN
3  2181-11-27 10:05:00  228339.0  244.000010   0.500000
4  2181-11-27 10:05:00  220949.0  203.333330  25.000000

[5 rows x 52 columns]
```

[ ]:
```
df_LOS['STARTTIME'] = pd.to_datetime(df_LOS['STARTTIME'])
df_LOS['ENDTIME'] = pd.to_datetime(df_LOS['ENDTIME'])
df_LOS['ADMINISTERED_TIME'] = round((df_LOS['ENDTIME'] - df_LOS['STARTTIME']).
 ↪dt.total_seconds() / (3600),2)
df_LOS[["STARTTIME","ENDTIME","ADMINISTERED_TIME"]]
```

[ ]:
```
                 STARTTIME             ENDTIME  ADMINISTERED_TIME
0      2181-11-26 12:00:00 2181-11-26 18:00:00               6.00
1      2181-11-26 12:00:00 2181-11-26 18:00:00               6.00
2      2181-11-26 11:33:00 2181-11-26 11:34:00               0.02
3      2181-11-27 01:57:00 2181-11-27 10:05:00               8.13
4      2181-11-27 01:57:00 2181-11-27 10:05:00               8.13
…                    …                   …                  …
839041 2132-10-13 18:04:00 2132-10-13 19:04:00               1.00
839042 2132-10-13 17:13:00 2132-10-13 18:04:00               0.85
839043 2132-10-13 17:13:00 2132-10-13 18:04:00               0.85
839044 2132-10-14 00:00:00 2132-10-14 00:01:00               0.02
839045 2132-10-14 09:30:00 2132-10-14 09:31:00               0.02

[839046 rows x 3 columns]
```

[ ]:
```
df_LOS["ADMINISTERED_TIME"].describe()
```

[ ]:
```
count    839037.000000
mean          3.234311
std          20.087274
min         -47.430000
25%           0.020000
50%           0.500000
75%           3.000000
max        8379.000000
Name: ADMINISTERED_TIME, dtype: float64
```

[ ]:
```
# Calculate the percentage of values > 24
percentage = (df_LOS["ADMINISTERED_TIME"] > 24).mean() * 100

print(f"{percentage:.2f}% of values are higher than 24")
```

```
2.23% of values are higher than 24
```

[ ]:
```
df_LOS.to_csv("state_1805.csv",index=False)
```

[ ]:
```
import pandas as pd
df_LOS = pd.read_csv("state_1805.csv")
```

[ ]:
```
df_LOS.head()
```

[ ]:
```
   ICUSTAY_ID  HADM_ID     LOS               INTIME    DBSOURCE INSURANCE  \
0      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare
1      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare
2      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare
3      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare
4      200001   152234  3.0786  2181-11-25 19:06:12  metavision  Medicare

  MARITAL_STATUS ETHNICITY         ADMISSION_LOCATION ADMISSION_TYPE  …  \
0         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
1         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
2         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
3         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
4         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …

   NBPm_0 NBPm_1 NBPm_2  NBPm_3            STARTTIME              ENDTIME  \
0    70.0   65.5    NaN    62.5  2181-11-26 12:00:00  2181-11-26 18:00:00
1    70.0   65.5    NaN    62.5  2181-11-26 12:00:00  2181-11-26 18:00:00
2    70.0   65.5    NaN    62.5  2181-11-26 11:33:00  2181-11-26 11:34:00
3    70.0   65.5    NaN    62.5  2181-11-27 01:57:00  2181-11-27 10:05:00
4    70.0   65.5    NaN    62.5  2181-11-27 01:57:00  2181-11-27 10:05:00

     ITEMID      AMOUNT       RATE  ADMINISTERED_TIME
0  228339.0  360.000000   1.000000               6.00
1  220949.0  299.999992  49.999999               6.00
2  221347.0    0.000000        NaN               0.02
3  228339.0  244.000010   0.500000               8.13
4  220949.0  203.333330  25.000000               8.13

[5 rows x 53 columns]
```

[ ]:
```python
df_LOS['STARTTIME'] = pd.to_datetime(df_LOS['STARTTIME'])
df_LOS['INTIME'] = pd.to_datetime(df_LOS['INTIME'])
df_LOS['STARTTIME-INTIME'] = round((df_LOS['STARTTIME'] - df_LOS['INTIME']).dt.
 ↪total_seconds() / (3600*24),0)
```

[ ]:
```python
df_LOS[["STARTTIME","INTIME","STARTTIME-INTIME"]]
```

[ ]:
```
                  STARTTIME              INTIME  STARTTIME-INTIME
0      2181-11-26 12:00:00 2181-11-25 19:06:12               1.0
1      2181-11-26 12:00:00 2181-11-25 19:06:12               1.0
2      2181-11-26 11:33:00 2181-11-25 19:06:12               1.0
3      2181-11-27 01:57:00 2181-11-25 19:06:12               1.0
4      2181-11-27 01:57:00 2181-11-25 19:06:12               1.0
…                    …                  …                  …
839041 2132-10-13 18:04:00 2132-10-13 10:41:17               0.0
839042 2132-10-13 17:13:00 2132-10-13 10:41:17               0.0
839043 2132-10-13 17:13:00 2132-10-13 10:41:17               0.0
```

```
839044 2132-10-14 00:00:00 2132-10-13 10:41:17                1.0
839045 2132-10-14 09:30:00 2132-10-13 10:41:17                1.0

[839046 rows x 3 columns]
```

[ ]:
```python
df_LOS["STARTTIME-INTIME"].describe()
```

[ ]:
```
count    839037.000000
mean          5.202860
std           6.571429
min           0.000000
25%           1.000000
50%           3.000000
75%           7.000000
max          53.000000
Name: STARTTIME-INTIME, dtype: float64
```

[ ]:
```python
df_LOS = df_LOS[df_LOS['STARTTIME-INTIME'] <= 3]
```

[ ]:
```python
top10_itemids = df_LOS['ITEMID'].value_counts().head(10)
print(top10_itemids)
top10_itemids_list = top10_itemids.index.tolist()

# Filter d_items where ITEMID is in the top list
matching_rows = d_items[d_items['ITEMID'].isin(top10_itemids_list)]

print(matching_rows)
```

```
ITEMID
225158.0    80324
220949.0    59800
226452.0    28399
225943.0    25008
222168.0    17816
221906.0    17287
223258.0    16011
221749.0    14666
221794.0    10528
221668.0     9059
Name: count, dtype: int64
       ROW_ID  ITEMID                LABEL         ABBREVIATION    DBSOURCE  \
8892    12973  221668  Midazolam (Versed)  Midazolam (Versed)  metavision
9547    12978  221749       Phenylephrine       Phenylephrine  metavision
9548    12979  221794  Furosemide (Lasix)  Furosemide (Lasix)  metavision
9553    12984  221906      Norepinephrine      Norepinephrine  metavision
9564    12995  222168            Propofol            Propofol  metavision
9572    13003  223258   Insulin - Regular   Insulin - Regular  metavision
```

```
11675   14375  225943            Solution            Solution  metavision
12075   12865  220949         Dextrose 5%         Dextrose 5%  metavision
12126   13758  225158           NaCl 0.9%           NaCl 0.9%  metavision
12279   14471  226452           PO Intake           PO Intake  metavision

              LINKSTO       CATEGORY UNITNAME PARAM_TYPE  CONCEPTID
8892   inputevents_mv    Medications       mg   Solution        NaN
9547   inputevents_mv    Medications       mg   Solution        NaN
9548   inputevents_mv    Medications       mg   Solution        NaN
9553   inputevents_mv    Medications       mg   Solution        NaN
9564   inputevents_mv    Medications       mg   Solution        NaN
9572   inputevents_mv    Medications    units   Solution        NaN
11675  inputevents_mv  Fluids/Intake       mL   Solution        NaN
12075  inputevents_mv  Fluids/Intake       mL   Solution        NaN
12126  inputevents_mv  Fluids/Intake       mL   Solution        NaN
12279  inputevents_mv  Fluids/Intake       mL   Solution        NaN
```

[ ]:
```python
# Drop duplicates so each ITEMID-ICUSTAY_ID pair is only counted once
unique_pairs = df_LOS[['ICUSTAY_ID', 'ITEMID']].drop_duplicates()

# Count how many unique ICUSTAY_IDs each ITEMID appears in
itemid_counts = unique_pairs['ITEMID'].value_counts()

# Get the top 10
top10_itemids_unique = itemid_counts.head(10)
print(top10_itemids_unique)

top10_itemids_unique_list = top10_itemids_unique.index.tolist()

# Filter d_items where ITEMID is in the top list
matching_rows = d_items[d_items['ITEMID'].isin(top10_itemids_unique_list)]
print(matching_rows)
```

```
ITEMID
225158.0    4490
220949.0    4191
226452.0    3854
221794.0    2505
225943.0    2418
225975.0    2311
225798.0    2180
222011.0    2113
225166.0    1826
223258.0    1661
Name: count, dtype: int64
       ROW_ID  ITEMID                          LABEL  \
8676    13765  225166             Potassium Chloride
9548    12979  221794             Furosemide (Lasix)
```

```
9555    12986  222011             Magnesium Sulfate
9572    13003  223258             Insulin - Regular
10218   13982  225798                    Vancomycin
11149   14224  225975  Heparin Sodium (Prophylaxis)
11675   14375  225943                      Solution
12075   12865  220949                   Dextrose 5%
12126   13758  225158                     NaCl 0.9%
12279   14471  226452                     PO Intake

                      ABBREVIATION    DBSOURCE         LINKSTO  \
8676       Potassium Chloride - KCL  metavision  inputevents_mv
9548             Furosemide (Lasix)  metavision  inputevents_mv
9555              Magnesium Sulfate  metavision  inputevents_mv
9572              Insulin - Regular  metavision  inputevents_mv
10218                    Vancomycin  metavision  inputevents_mv
11149  Heparin Sodium (Prophylaxis)  metavision  inputevents_mv
11675                      Solution  metavision  inputevents_mv
12075                   Dextrose 5%  metavision  inputevents_mv
12126                     NaCl 0.9%  metavision  inputevents_mv
12279                     PO Intake  metavision  inputevents_mv

            CATEGORY UNITNAME PARAM_TYPE  CONCEPTID
8676     Medications      mEq   Solution        NaN
9548     Medications       mg   Solution        NaN
9555     Medications    grams   Solution        NaN
9572     Medications    units   Solution        NaN
10218    Antibiotics     dose   Solution        NaN
11149    Medications     dose   Solution        NaN
11675  Fluids/Intake       mL   Solution        NaN
12075  Fluids/Intake       mL   Solution        NaN
12126  Fluids/Intake       mL   Solution        NaN
12279  Fluids/Intake       mL   Solution        NaN
```

[ ]:
```python
import pandas as pd

# Combine both lists and get unique ITEMIDs (in case there's overlap)
all_itemids = list(set(top10_itemids_unique_list + top10_itemids_list))

correlation_results = []

for itemid in all_itemids:
    # Step 1: Filter rows for the current ITEMID
    df_filtered = df_LOS[df_LOS['ITEMID'] == itemid]

    # Step 2: Compute correlation between AMOUNT and LOS
    correlation = df_filtered['AMOUNT'].corr(df_filtered['LOS'])
```

```python
    # Store the result (including raw correlation and absolute value for␣
 ↪sorting)
    correlation_results.append((itemid, correlation, abs(correlation)))

# Step 3: Sort by ABSOLUTE correlation in descending order
correlation_results.sort(key=lambda x: x[2], reverse=True)  # x[2] is the␣
 ↪absolute correlation

# Get additional info from d_items
# Create a DataFrame for easier merging
corr_df = pd.DataFrame(
    [(itemid, corr) for itemid, corr, _ in correlation_results],  # Drop the␣
 ↪absolute value for display
    columns=['ITEMID', 'Correlation']
)

# Merge with d_items to get LABEL, ABBREVIATION, and UNITNAME
result_df = corr_df.merge(
    d_items[['ITEMID', 'LABEL', 'ABBREVIATION', 'UNITNAME']],
    on='ITEMID',
    how='left'
)

# Print the results in a formatted way
print("ITEMID\tCorrelation\tLABEL\tABBREVIATION\tUNITNAME")
print("--------------------------------------------------")
for _, row in result_df.iterrows():
    print(
        f"{row['ITEMID']}\t"
        f"{row['Correlation']:.4f}\t"
        f"{row['LABEL']}\t"
        f"{row['ABBREVIATION']}\t"
        f"{row['UNITNAME']}"
    )
```

```
ITEMID  Correlation     LABEL   ABBREVIATION    UNITNAME
--------------------------------------------------
221668.0        0.1364  Midazolam (Versed)      Midazolam (Versed)      mg
225166.0        -0.1293 Potassium Chloride      Potassium Chloride - KCL
mEq
222168.0        0.0721  Propofol        Propofol        mg
221794.0        0.0492  Furosemide (Lasix)      Furosemide (Lasix)      mg
225158.0        -0.0465 NaCl 0.9%       NaCl 0.9%       mL
223258.0        0.0407  Insulin - Regular       Insulin - Regular       units
225798.0        -0.0326 Vancomycin      Vancomycin      dose
221749.0        0.0320  Phenylephrine   Phenylephrine   mg
220949.0        0.0316  Dextrose 5%     Dextrose 5%     mL
```

```
221906.0        0.0266  Norepinephrine  Norepinephrine  mg
225943.0        0.0266  Solution        Solution        mL
222011.0        -0.0152 Magnesium Sulfate       Magnesium Sulfate       grams
225975.0        0.0120  Heparin Sodium (Prophylaxis)    Heparin Sodium
(Prophylaxis)    dose
226452.0        -0.0063 PO Intake       PO Intake       mL
```

[ ]:
```
# --- Extract top 10 ITEMIDs (10 instead of 5 like in chartevents bc higher␣
↪correlations for input) ---
top10_itemids = [itemid for itemid, _, _ in correlation_results[:10]]
print("Top 10 ITEMIDs by absolute correlation:", top10_itemids)

# --- Filter df_LOS to keep only top10 ITEMIDs or NaN ---
df_LOS = df_LOS[df_LOS['ITEMID'].isin(top10_itemids) | df_LOS['ITEMID'].isna()]

# --- Print summary of filtered data ---
df_LOS.head()
```

```
Top 10 ITEMIDs by absolute correlation: [221668.0, 225166.0, 222168.0, 221794.0,
225158.0, 223258.0, 225798.0, 221749.0, 220949.0, 221906.0]
```

[ ]:
```
    ICUSTAY_ID  HADM_ID     LOS              INTIME    DBSOURCE INSURANCE  \
1       200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
4       200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
6       200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
9       200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
11      200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare

   MARITAL_STATUS ETHNICITY         ADMISSION_LOCATION ADMISSION_TYPE  …  \
1          SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
4          SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
6          SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
9          SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …
11         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY  …

    NBPm_1 NBPm_2 NBPm_3           STARTTIME              ENDTIME    ITEMID  \
1     65.5    NaN   62.5 2181-11-26 12:00:00  2181-11-26 18:00:00  220949.0
4     65.5    NaN   62.5 2181-11-27 01:57:00  2181-11-27 10:05:00  220949.0
6     65.5    NaN   62.5 2181-11-27 12:05:00  2181-11-27 12:06:00  220949.0
9     65.5    NaN   62.5 2181-11-26 16:15:00  2181-11-26 17:49:00  220949.0
11    65.5    NaN   62.5 2181-11-26 18:00:00  2181-11-27 01:57:00  220949.0

        AMOUNT       RATE  ADMINISTERED_TIME  STARTTIME-INTIME
1   299.999992  49.999999               6.00               1.0
4   203.333330  25.000000               8.13               1.0
6   100.000000        NaN               0.02               2.0
9    16.456583  10.504202               1.57               1.0
```

```
11  200.000009  25.157234                7.95                1.0

[5 rows x 54 columns]
```

[ ]:
```python
# Drop columns we don't need
df_LOS.
 ↪drop(columns=['STARTTIME','ENDTIME','ADMINISTERED_TIME','RATE'],inplace=True)

# Convert AMOUNT to numeric (handling non-numeric values if needed)
df_LOS['AMOUNT'] = pd.to_numeric(df_LOS['AMOUNT'], errors='coerce')

# Create a dictionary for aggregation - sum for AMOUNT, first for all other␣
 ↪columns
agg_dict = {'AMOUNT': 'sum'}
other_columns = [col for col in df_LOS.columns
                if col not in ['ICUSTAY_ID', 'ITEMID', 'STARTTIME-INTIME',␣
 ↪'AMOUNT']]
agg_dict.update({col: 'first' for col in other_columns})

# Perform the grouped aggregation
df_LOS = df_LOS.groupby(
    ['ICUSTAY_ID', 'ITEMID', 'STARTTIME-INTIME'],
    as_index=False
).agg(agg_dict)
```

```
<ipython-input-45-715079f760f2>:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_LOS.drop(columns=['STARTTIME','ENDTIME','ADMINISTERED_TIME','RATE'],inplace
=True)
<ipython-input-45-715079f760f2>:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_LOS['AMOUNT'] = pd.to_numeric(df_LOS['AMOUNT'], errors='coerce')
```

[ ]:
```python
df_LOS.head()
```

[ ]:
```
   ICUSTAY_ID    ITEMID  STARTTIME-INTIME       AMOUNT  HADM_ID     LOS  \
0      200001  220949.0               1.0  1020.206577   152234  3.0786
1      200001  220949.0               2.0   100.000000   152234  3.0786
2      200001  225158.0               0.0   200.000006   152234  3.0786
3      200001  225158.0               2.0   250.000000   152234  3.0786
```

```
4      200011  221794.0                0.0    20.000001   121562  1.6330

                INTIME    DBSOURCE INSURANCE MARITAL_STATUS  … ABPm_2 ABPm_3  \
0 2181-11-25 19:06:12  metavision  Medicare         SINGLE  …   70.0   69.5
1 2181-11-25 19:06:12  metavision  Medicare         SINGLE  …   70.0   69.5
2 2181-11-25 19:06:12  metavision  Medicare         SINGLE  …   70.0   69.5
3 2181-11-25 19:06:12  metavision  Medicare         SINGLE  …   70.0   69.5
4 2188-08-06 01:39:24  metavision  Medicare         SINGLE  …    NaN    NaN

  CVP_0  CVP_1 CVP_2 CVP_3  NBPm_0  NBPm_1  NBPm_2  NBPm_3
0   NaN    NaN   NaN   NaN    70.0    65.5     NaN    62.5
1   NaN    NaN   NaN   NaN    70.0    65.5     NaN    62.5
2   NaN    NaN   NaN   NaN    70.0    65.5     NaN    62.5
3   NaN    NaN   NaN   NaN    70.0    65.5     NaN    62.5
4   NaN    NaN   NaN   NaN    76.0    90.0    92.0     NaN

[5 rows x 50 columns]
```

[ ]:
```
# Step 1: Create a combined column for ITEMID + STARTTIME-INTIME
df_LOS['ITEMID_STARTTIME-INTIME'] = df_LOS['ITEMID'].astype(str) + '_' +␣
↪df_LOS['STARTTIME-INTIME'].astype(str)

# Step 2: Pivot the table to get one row per ICUSTAY_ID and columns for each␣
↪ITEMID_CHARTTIME
df_pivoted = df_LOS.pivot_table(
    index='ICUSTAY_ID',  # Keep ICUSTAY_ID as the index (one row per ID)
    columns='ITEMID_STARTTIME-INTIME',  # New columns will be ITEMID_DAY␣
↪combinations
    values='AMOUNT',  # Fill cells with the VALUE
    aggfunc='first'  # Take the first value
).reset_index()  # Reset index to make ICUSTAY_ID a column again

# Step 3: (Optional) Rename columns if needed (e.g., remove any multi-indexing)
df_pivoted.columns.name = None  # Remove the 'ITEMID_CHARTTIME' label from␣
↪columns
```

[ ]:
```
df_pivoted.fillna(0,inplace=True)
```

[ ]:
```
# Create new column names by removing the last two characters
# except for "ICUSTAY_ID"
new_columns = {}

for col in df_pivoted.columns:
    if col != "ICUSTAY_ID":
        # Remove the last two characters from the column name
        new_columns[col] = col[:-2]
    else:
```

```
        # Keep "ICUSTAY_ID" unchanged
        new_columns[col] = col

# Rename the columns
df_pivoted = df_pivoted.rename(columns=new_columns)

# Display the first few rows to confirm changes
print(df_pivoted.head())
```

```
   ICUSTAY_ID   220949.0_0   220949.0_1  220949.0_2  220949.0_3  221668.0_0  \
0      200001     0.000000  1020.206577       100.0         0.0    0.000000
1      200011     0.000000     0.000000         0.0         0.0    0.000000
2      200024  1228.723358     0.000000         0.0         0.0    0.000000
3      200028     0.000000     0.000000         0.0         0.0   31.672712
4      200038     0.000000     0.000000         0.0         0.0    0.000000

   221668.0_1  221668.0_2  221668.0_3  221749.0_0  …  225158.0_2  \
0      0.0000         0.0         0.0         0.0  …       250.0
1      0.0000         0.0         0.0         0.0  …         0.0
2      0.0000         0.0         0.0         0.0  …         0.0
3      0.8003         0.0         0.0         0.0  …         0.0
4      0.0000         0.0         0.0         0.0  …         0.0

   225158.0_3  225166.0_0  225166.0_1  225166.0_2  225166.0_3  225798.0_0  \
0         0.0         0.0    0.000000    0.000000    0.000000         0.0
1         0.0         0.0    0.000000    0.000000    0.000000         0.0
2         0.0         0.0    0.000000    0.000000    0.000000         0.0
3         0.0        20.0   60.000001   40.000001   40.000001         0.0
4         0.0         0.0    0.000000    0.000000    0.000000         0.0

   225798.0_1  225798.0_2  225798.0_3
0         0.0         0.0         0.0
1         0.0         0.0         0.0
2         0.0         0.0         0.0
3         0.0         0.0         0.0
4         0.0         0.0         0.0

[5 rows x 41 columns]
```

[ ]:
```
# Create a dictionary mapping ITEMIDs to their abbreviations from d_items
itemid_to_abbrev = {}
for _, row in d_items.iterrows():
    itemid_to_abbrev[str(row['ITEMID'])] = row['ABBREVIATION']

# Create new column names by replacing the first 8 characters with the␣
 ↪corresponding abbreviation
new_columns = {}
```

```
for col in df_pivoted.columns:
    if col != "ICUSTAY_ID":
        # Extract the first 8 characters (which represent the ITEMID)
        itemid_str = col[:8]

        # Remove the decimal point if present and convert to match the integer␣
 ↪format in d_items
        if '.' in itemid_str:
            itemid_str = itemid_str.split('.')[0]

        # Look for the abbreviation in the mapping
        if itemid_str in itemid_to_abbrev:
            # Replace the first 8 characters with the abbreviation
            new_columns[col] = d_items.loc[d_items['ITEMID'] ==␣
 ↪int(itemid_str), 'ABBREVIATION'].values[0] + col[8:]
        else:
            # Keep the original column name if no match is found
            new_columns[col] = col
    else:
        # Keep "ICUSTAY_ID" unchanged
        new_columns[col] = col
```

[ ]:
```
# Rename the columns
df_pivoted = df_pivoted.rename(columns=new_columns)

# Drop unprocessed ITEM columns from df_LOS
df_LOS.
 ↪drop(columns=["ITEMID","AMOUNT","STARTTIME-INTIME","ITEMID_STARTTIME-INTIME"],inplace=True)
df_LOS = df_LOS.drop_duplicates()

# Merge df with the processed ITEM columns
df_LOS = df_LOS.merge(df_pivoted, on='ICUSTAY_ID', how='left')
```

[ ]:
```
df_LOS.head()
```

[ ]:
```
   ICUSTAY_ID  HADM_ID     LOS              INTIME    DBSOURCE INSURANCE  \
0      200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
1      200011   121562  1.6330 2188-08-06 01:39:24  metavision  Medicare
2      200024   179633  0.3812 2127-03-03 16:09:07  metavision  Medicare
3      200028   181955  2.9038 2133-10-29 17:13:50  metavision  Medicare
4      200038   175436  1.0588 2143-10-24 20:35:24  metavision  Medicare

  MARITAL_STATUS                ETHNICITY         ADMISSION_LOCATION  \
0         SINGLE                    ASIAN  CLINIC REFERRAL/PREMATURE
1         SINGLE                    WHITE  CLINIC REFERRAL/PREMATURE
2        MARRIED  BLACK/AFRICAN AMERICAN       EMERGENCY ROOM ADMIT
```

```
3        MARRIED                    WHITE  PHYS REFERRAL/NORMAL DELI
4        MARRIED                    WHITE       EMERGENCY ROOM ADMIT

  ADMISSION_TYPE  …  NaCl 0.9%_2 NaCl 0.9%_3 Potassium Chloride - KCL_0  \
0      EMERGENCY  …        250.0         0.0                       0.0
1      EMERGENCY  …          0.0         0.0                       0.0
2      EMERGENCY  …          0.0         0.0                       0.0
3       ELECTIVE  …          0.0         0.0                      20.0
4      EMERGENCY  …          0.0         0.0                       0.0

   Potassium Chloride - KCL_1  Potassium Chloride - KCL_2  \
0                    0.000000                    0.000000
1                    0.000000                    0.000000
2                    0.000000                    0.000000
3                   60.000001                   40.000001
4                    0.000000                    0.000000

   Potassium Chloride - KCL_3  Vancomycin_0  Vancomycin_1  Vancomycin_2  \
0                    0.000000           0.0           0.0           0.0
1                    0.000000           0.0           0.0           0.0
2                    0.000000           0.0           0.0           0.0
3                   40.000001           0.0           0.0           0.0
4                    0.000000           0.0           0.0           0.0

   Vancomycin_3
0           0.0
1           0.0
2           0.0
3           0.0
4           0.0

[5 rows x 87 columns]
```

[ ]:
```
df_LOS.shape
```

[ ]:
```
(5015, 87)
```

[ ]:
```
df_LOS.to_csv('base_df_LOS_input.csv', index=False)
```

[ ]:
```
procedureevents = pd.read_csv('PROCEDUREEVENTS_MV.csv',usecols=['ICUSTAY_ID',␣
↪'STARTTIME', 'ENDTIME', 'ITEMID', 'VALUE','VALUEUOM'])
```

[ ]:
```
# Filter events to keep only rows with ICUSTAY_IDs present in df_LOS
procedureevents = procedureevents[procedureevents['ICUSTAY_ID'].
↪isin(df_LOS['ICUSTAY_ID'])]
```

[ ]:
```
procedureevents.shape
```

[ ]: (58663, 6)

[ ]:
```
top10_itemids = procedureevents['ITEMID'].value_counts().head(10)
top10_itemids_list = top10_itemids.index.tolist()

# Drop duplicates so each ITEMID-ICUSTAY_ID pair is only counted once
unique_pairs = procedureevents[['ICUSTAY_ID', 'ITEMID']].drop_duplicates()

# Count how many unique ICUSTAY_IDs each ITEMID appears in
itemid_counts = unique_pairs['ITEMID'].value_counts()

# Get the top 10
top10_itemids_unique = itemid_counts.head(10)
top10_itemids_unique_list = top10_itemids_unique.index.tolist()

# Combine both lists and get unique ITEMIDs (in case there's overlap)
all_itemids = list(set(top10_itemids_unique_list + top10_itemids_list))
all_itemids
```

[ ]:
```
[225792,
 224263,
 224274,
 224275,
 225459,
 224277,
 225752,
 225401,
 225402,
 225432,
 227194]
```

[ ]:
```
# Filter the DataFrame to keep only rows with ITEMID in all_itemids
procedureevents = procedureevents[procedureevents['ITEMID'].isin(all_itemids)]
```

[ ]:
```
# Get the VALUEUOM for each ITEMID (assuming they are consistent)
itemid_uom = procedureevents.groupby('ITEMID')['VALUEUOM'].first().reset_index()
print(itemid_uom)
```

```
    ITEMID VALUEUOM
0   224263      min
1   224274      min
2   224275      min
3   224277      min
4   225401     None
5   225402     None
6   225432     None
7   225459     None
```

```
8   225752      min
9   225792      min
10  227194     None
```

[ ]:
```python
# Check if any ITEMID has multiple VALUEUOMs
inconsistent_uom = procedureevents.groupby('ITEMID')['VALUEUOM'].nunique() > 1
inconsistent_itemids = inconsistent_uom[inconsistent_uom].index.tolist()

if inconsistent_itemids:
    print(f"Warning: Some ITEMIDs have inconsistent VALUEUOM:␣
 ↪{inconsistent_itemids}")
else:
    print("All ITEMIDs have consistent VALUEUOM values.")
```

```
Warning: Some ITEMIDs have inconsistent VALUEUOM: [224263, 224274, 224275,
224277, 225752, 225792]
```

[ ]:
```python
for itemid in inconsistent_itemids:
    # Get all VALUEUOM values for this ITEMID and their counts
    uom_counts = procedureevents[procedureevents['ITEMID'] ==␣
 ↪itemid]['VALUEUOM'].value_counts()

    print(f"\nITEMID: {itemid}")
    print(uom_counts.to_string())  # `to_string()` avoids truncation in output
```

```
ITEMID: 225792
VALUEUOM
min     2215
hour      27
day       21

ITEMID: 224263
VALUEUOM
min     1746
day       56
hour       6

ITEMID: 224274
VALUEUOM
min    1707
day      41

ITEMID: 224275
VALUEUOM
min     6266
day      141
hour      18
```

```
ITEMID: 225459
Series([], )

ITEMID: 224277
VALUEUOM
min     3689
day       66
hour      20

ITEMID: 225752
VALUEUOM
min     2590
day      103
hour       5

ITEMID: 225401
Series([], )

ITEMID: 225402
Series([], )

ITEMID: 225432
Series([], )

ITEMID: 227194
Series([], )
```

[ ]:
```
boolean_itemids = [item for item in all_itemids if item not in␣
↪inconsistent_itemids]

for itemid in boolean_itemids:
    # Get all VALUE values for the Boolean ITEMID and their counts
    uom_counts = procedureevents[procedureevents['ITEMID'] == itemid]['VALUE'].
↪value_counts()

    print(f"\nITEMID: {itemid}")
    print(uom_counts.to_string())  # `to_string()` avoids truncation in output
```

```
ITEMID: 225459
VALUE
 1.0     7549
-2.0        2
-1.0        1
-21.0       1

ITEMID: 225401
```

```
VALUE
 1.0    2499
-9.0       1
-2.0       1
-1.0       1

ITEMID: 225402
VALUE
1.0    4122

ITEMID: 225432
VALUE
1.0    1476
0.0       1

ITEMID: 227194
VALUE
 1.0    1714
-3.0       1
```

[ ]:
```
procedureevents = procedureevents[
    ~procedureevents['ITEMID'].isin(boolean_itemids) |
    (procedureevents['VALUE'] == 1)
]
```

[ ]:
```
for itemid in inconsistent_itemids:
    print(f"\nITEMID: {itemid}")
    print(procedureevents[procedureevents['ITEMID'] == itemid][['VALUE',␣
 ↪'VALUEUOM']].head())
```

```
ITEMID: 224263
       VALUE VALUEUOM
47   13211.0      min
144   1634.0      min
155   8417.0      min
299   7709.0      min
450   1317.0      min

ITEMID: 224274
       VALUE VALUEUOM
298   1475.0      min
495    517.0      min
515    600.0      min
1787   300.0      min
1853  2565.0      min

ITEMID: 224275
```

```
      VALUE VALUEUOM
241  3236.0      min
242  3242.0      min
483  5245.0      min
494    70.0      min
517  6240.0      min

ITEMID: 224277
      VALUE VALUEUOM
48   2931.0      min
49   2141.0      min
140   924.0      min
332  1947.0      min
333  1947.0      min

ITEMID: 225752
       VALUE VALUEUOM
52     540.0      min
524   4440.0      min
642   4330.0      min
812   9831.0      min
860  26510.0      min

ITEMID: 225792
      VALUE VALUEUOM
142  4462.0      min
475   944.0     hour
520  1891.0      min
603  7654.0     hour
641  8365.0     hour
```

[ ]:
```python
for itemid in inconsistent_itemids:
    # Filter rows for current ITEMID
    mask = procedureevents['ITEMID'] == itemid

    # Convert hours to minutes (multiply VALUE by 60)
    hour_mask = mask & (procedureevents['VALUEUOM'] == 'hour')
    procedureevents.loc[hour_mask, 'VALUE'] *= 60
    procedureevents.loc[hour_mask, 'VALUEUOM'] = 'min'

    # Convert days to minutes (multiply VALUE by 60*24)
    day_mask = mask & (procedureevents['VALUEUOM'] == 'day')
    procedureevents.loc[day_mask, 'VALUE'] *= 60 * 24
    procedureevents.loc[day_mask, 'VALUEUOM'] = 'min'
```

[ ]:
```python
df_LOS = df_LOS.merge(procedureevents, on='ICUSTAY_ID', how='left')
```

[ ]:
```
df_LOS.head()
```

[ ]:
```
   ICUSTAY_ID  HADM_ID     LOS              INTIME    DBSOURCE INSURANCE  \
0      200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
1      200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
2      200011   121562  1.6330 2188-08-06 01:39:24  metavision  Medicare
3      200011   121562  1.6330 2188-08-06 01:39:24  metavision  Medicare
4      200024   179633  0.3812 2127-03-03 16:09:07  metavision  Medicare

  MARITAL_STATUS               ETHNICITY         ADMISSION_LOCATION  \
0         SINGLE                   ASIAN  CLINIC REFERRAL/PREMATURE
1         SINGLE                   ASIAN  CLINIC REFERRAL/PREMATURE
2         SINGLE                   WHITE  CLINIC REFERRAL/PREMATURE
3         SINGLE                   WHITE  CLINIC REFERRAL/PREMATURE
4        MARRIED  BLACK/AFRICAN AMERICAN       EMERGENCY ROOM ADMIT

  ADMISSION_TYPE  …  Potassium Chloride - KCL_3 Vancomycin_0 Vancomycin_1  \
0      EMERGENCY  …                         0.0          0.0          0.0
1      EMERGENCY  …                         0.0          0.0          0.0
2      EMERGENCY  …                         0.0          0.0          0.0
3      EMERGENCY  …                         0.0          0.0          0.0
4      EMERGENCY  …                         0.0          0.0          0.0

   Vancomycin_2  Vancomycin_3            STARTTIME              ENDTIME  \
0           0.0           0.0  2181-11-26 15:30:00  2181-11-28 12:40:00
1           0.0           0.0  2181-11-28 02:30:00  2181-11-28 02:31:00
2           0.0           0.0  2188-08-06 01:44:00  2188-08-07 16:50:00
3           0.0           0.0  2188-08-06 04:37:00  2188-08-06 04:38:00
4           0.0           0.0  2127-03-03 15:00:00  2127-03-03 21:00:00

     ITEMID   VALUE  VALUEUOM
0  225752.0  2710.0       min
1  225459.0     1.0       NaN
2  224274.0  2346.0       min
3  225459.0     1.0       NaN
4  225792.0   360.0       min

[5 rows x 92 columns]
```

[ ]:
```
df_LOS['STARTTIME'] = pd.to_datetime(df_LOS['STARTTIME'])
df_LOS['INTIME'] = pd.to_datetime(df_LOS['INTIME'])
df_LOS['STARTTIME-INTIME'] = round((df_LOS['STARTTIME'] - df_LOS['INTIME']).dt.
 ↪total_seconds() / (3600*24),0)
df_LOS.drop(columns=['STARTTIME','ENDTIME'],inplace=True)
```

[ ]:
```
df_LOS = df_LOS[df_LOS['STARTTIME-INTIME'] <= 3]
```

[ ]:
```
from scipy.stats import pointbiserialr

# Initialize results storage
correlation_results = []

# Process non-binary items (Pearson correlation)
for itemid in inconsistent_itemids:
    df_filtered = df_LOS[df_LOS['ITEMID'] == itemid]
    correlation = df_filtered['VALUE'].corr(df_filtered['LOS'])
    correlation_results.append((itemid, correlation, abs(correlation),␣
 ↪'pearson'))

# Process binary items (Point-Biserial correlation)
for itemid in boolean_itemids:
    # Create binary series (1 if present, 0 otherwise)
    all_stays = df_LOS['ICUSTAY_ID'].unique()
    present_stays = df_LOS[df_LOS['ITEMID'] == itemid]['ICUSTAY_ID'].unique()
    binary_series = pd.Series(
        [1 if stay in present_stays else 0 for stay in all_stays],
        index=all_stays
    )

    # Get corresponding LOS values
    los_series = df_LOS.groupby('ICUSTAY_ID')['LOS'].first()

    # Calculate point-biserial correlation
    corr, _ = pointbiserialr(binary_series, los_series)
    correlation_results.append((itemid, corr, abs(corr), 'point-biserial'))

# Sort by absolute correlation
correlation_results.sort(key=lambda x: x[2], reverse=True)

# Create result DataFrame
corr_df = pd.DataFrame(
    [(itemid, corr, method) for itemid, corr, _, method in correlation_results],
    columns=['ITEMID', 'Correlation', 'Method']
)

# Merge with d_items
result_df = corr_df.merge(
    d_items[['ITEMID', 'LABEL', 'ABBREVIATION', 'UNITNAME']],
    on='ITEMID',
    how='left'
)

# Print formatted results
print("ITEMID\tCorrelation\tMethod\t\tLABEL\tABBREVIATION")
```

```
print("------------------------------------------------------------")
for _, row in result_df.iterrows():
    print(
        f"{row['ITEMID']}\t"
        f"{row['Correlation']:.4f}\t"
        f"{row['Method']}\t"
        f"{row['LABEL']}\t"
        f"{row['ABBREVIATION']}\t"
    )
```

```
ITEMID  Correlation     Method          LABEL   ABBREVIATION
------------------------------------------------------------
225459  0.2866  point-biserial  Chest X-Ray     Chest X-Ray
225401  0.2288  point-biserial  Blood Cultured  Blood Cultured
225432  0.1563  point-biserial  Transthoracic Echo      Transthoracic Echo
225402  0.1451  point-biserial  EKG     EKG
225752  0.0688  pearson Arterial Line   Arterial Line
225792  0.0682  pearson Invasive Ventilation    Invasive Ventilation
224263  0.0533  pearson Multi Lumen     Multi Lumen
224277  -0.0215 pearson 18 Gauge        18 Gauge
224275  0.0123  pearson 20 Gauge        20 Gauge
224274  0.0116  pearson 22 Gauge        22 Gauge
227194  0.0034  point-biserial  Extubation      Extubation
```

[ ]:
```
# --- Extract top 10 ITEMIDs (10 instead of 5 like in chartevents bc higher␣
↪correlations for input) ---
top10_itemids = [itemid for itemid, _, _, _ in correlation_results[:10]]
print("Top 10 ITEMIDs by absolute correlation:", top10_itemids)

# --- Filter df_LOS to keep only top10 ITEMIDs or NaN ---
df_LOS = df_LOS[df_LOS['ITEMID'].isin(top10_itemids) | df_LOS['ITEMID'].isna()]
```

```
Top 10 ITEMIDs by absolute correlation: [225459, 225401, 225432, 225402, 225752,
225792, 224263, 224277, 224275, 224274]
```

[ ]:
```
df_LOS.drop(columns=['VALUEUOM'],inplace=True)
```

[ ]:
```
df_LOS.head()
```

[ ]:
```
   ICUSTAY_ID  HADM_ID     LOS              INTIME    DBSOURCE INSURANCE  \
0      200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
1      200001   152234  3.0786 2181-11-25 19:06:12  metavision  Medicare
2      200011   121562  1.6330 2188-08-06 01:39:24  metavision  Medicare
3      200011   121562  1.6330 2188-08-06 01:39:24  metavision  Medicare
4      200024   179633  0.3812 2127-03-03 16:09:07  metavision  Medicare

  MARITAL_STATUS                ETHNICITY         ADMISSION_LOCATION  \
```

```
0         SINGLE                   ASIAN  CLINIC REFERRAL/PREMATURE
1         SINGLE                   ASIAN  CLINIC REFERRAL/PREMATURE
2         SINGLE                   WHITE  CLINIC REFERRAL/PREMATURE
3         SINGLE                   WHITE  CLINIC REFERRAL/PREMATURE
4        MARRIED  BLACK/AFRICAN AMERICAN       EMERGENCY ROOM ADMIT

  ADMISSION_TYPE  …  Potassium Chloride - KCL_1 Potassium Chloride - KCL_2  \
0      EMERGENCY  …                         0.0                        0.0
1      EMERGENCY  …                         0.0                        0.0
2      EMERGENCY  …                         0.0                        0.0
3      EMERGENCY  …                         0.0                        0.0
4      EMERGENCY  …                         0.0                        0.0

  Potassium Chloride - KCL_3  Vancomycin_0  Vancomycin_1  Vancomycin_2  \
0                        0.0           0.0           0.0           0.0
1                        0.0           0.0           0.0           0.0
2                        0.0           0.0           0.0           0.0
3                        0.0           0.0           0.0           0.0
4                        0.0           0.0           0.0           0.0

   Vancomycin_3    ITEMID   VALUE  STARTTIME-INTIME
0           0.0  225752.0  2710.0               1.0
1           0.0  225459.0     1.0               2.0
2           0.0  224274.0  2346.0               0.0
3           0.0  225459.0     1.0               0.0
4           0.0  225792.0   360.0              -0.0

[5 rows x 90 columns]
```

[ ]:
```
# Convert VALUE to numeric (handling non-numeric values if needed)
df_LOS['VALUE'] = pd.to_numeric(df_LOS['VALUE'], errors='coerce')

# Create a dictionary for aggregation - sum for VALUE, first for all other␣
↪columns
agg_dict = {'VALUE': 'sum'}
other_columns = [col for col in df_LOS.columns
                 if col not in ['ICUSTAY_ID', 'ITEMID', 'STARTTIME-INTIME',␣
↪'VALUE']]
agg_dict.update({col: 'first' for col in other_columns})

# Perform the grouped aggregation
df_LOS = df_LOS.groupby(
    ['ICUSTAY_ID', 'ITEMID', 'STARTTIME-INTIME'],
    as_index=False
).agg(agg_dict)
```

[ ]:
```
# Create a boolean mask for rows to modify
mask = (
    df_LOS['ITEMID'].isin(boolean_itemids) &  # ITEMID in boolean_itemids
    (df_LOS['VALUE'] > 1)                     # VALUE > 1
)

# Set VALUE = 1 where conditions are met
df_LOS.loc[mask, 'VALUE'] = 1
```

[ ]:
```
# Step 1: Create a combined column for ITEMID + STARTTIME-INTIME
df_LOS['ITEMID_STARTTIME-INTIME'] = df_LOS['ITEMID'].astype(str) + '_' +␣
↪df_LOS['STARTTIME-INTIME'].astype(str)

# Step 2: Pivot the table to get one row per ICUSTAY_ID and columns for each␣
↪ITEMID_CHARTTIME
df_pivoted = df_LOS.pivot_table(
    index='ICUSTAY_ID',  # Keep ICUSTAY_ID as the index (one row per ID)
    columns='ITEMID_STARTTIME-INTIME',  # New columns will be ITEMID_DAY␣
↪combinations
    values='VALUE',  # Fill cells with the VALUE
    aggfunc='first'  # Take the first value
).reset_index()  # Reset index to make ICUSTAY_ID a column again

# Step 3: (Optional) Rename columns if needed (e.g., remove any multi-indexing)
df_pivoted.columns.name = None  # Remove the 'ITEMID_CHARTTIME' label from␣
↪columns
df_pivoted.fillna(0,inplace=True)
```

[ ]:
```
# Create new column names by removing the last two characters
# except for "ICUSTAY_ID"
new_columns = {}

for col in df_pivoted.columns:
    if col != "ICUSTAY_ID":
        # Remove the last two characters from the column name
        new_columns[col] = col[:-2]
    else:
        # Keep "ICUSTAY_ID" unchanged
        new_columns[col] = col

# Rename the columns
df_pivoted = df_pivoted.rename(columns=new_columns)

# Display the first few rows to confirm changes
print(df_pivoted.head())
```

```
   ICUSTAY_ID  224263.0_0  224263.0_1  224263.0_2  224263.0_3  224274.0_0  \
```

```
0      200001         0.0         0.0         0.0         0.0         0.0
1      200011         0.0         0.0         0.0         0.0      2346.0
2      200024      1056.0         0.0         0.0         0.0         0.0
3      200028      3758.0         0.0         0.0         0.0         0.0
4      200038         0.0         0.0         0.0         0.0         0.0

   224274.0_1  224274.0_2  224274.0_3  224275.0_0  …  225459.0_2  \
0         0.0         0.0         0.0         0.0  …         1.0
1         0.0         0.0         0.0         0.0  …         0.0
2         0.0         0.0         0.0         0.0  …         0.0
3         0.0         0.0         0.0         0.0  …         0.0
4         0.0         0.0         0.0      1470.0  …         0.0

   225459.0_3  225752.0_0  225752.0_1  225752.0_2  225752.0_3  225792.0_0  \
0         0.0         0.0      2710.0         0.0         0.0         0.0
1         0.0         0.0         0.0         0.0         0.0         0.0
2         0.0       498.0         0.0         0.0         0.0       360.0
3         0.0      2347.0         0.0         0.0         0.0       931.0
4         0.0         0.0         0.0         0.0         0.0         0.0

   225792.0_1  225792.0_2  225792.0_3
0         0.0         0.0         0.0
1         0.0         0.0         0.0
2         0.0         0.0         0.0
3         0.0         0.0         0.0
4         0.0         0.0         0.0

[5 rows x 41 columns]
```

[ ]:
```
# Create a dictionary mapping ITEMIDs to their abbreviations from d_items
itemid_to_abbrev = {}
for _, row in d_items.iterrows():
    itemid_to_abbrev[str(row['ITEMID'])] = row['ABBREVIATION']

# Create new column names by replacing the first 8 characters with the␣
↪corresponding abbreviation
new_columns = {}

for col in df_pivoted.columns:
    if col != "ICUSTAY_ID":
        # Extract the first 8 characters (which represent the ITEMID)
        itemid_str = col[:8]

        # Remove the decimal point if present and convert to match the integer␣
↪format in d_items
        if '.' in itemid_str:
            itemid_str = itemid_str.split('.')[0]
```

```
        # Look for the abbreviation in the mapping
        if itemid_str in itemid_to_abbrev:
            # Replace the first 8 characters with the abbreviation
            new_columns[col] = d_items.loc[d_items['ITEMID'] ==␣
↪int(itemid_str), 'ABBREVIATION'].values[0] + col[8:]
        else:
            # Keep the original column name if no match is found
            new_columns[col] = col
    else:
        # Keep "ICUSTAY_ID" unchanged
        new_columns[col] = col
```

[ ]:
```
# Rename the columns
df_pivoted = df_pivoted.rename(columns=new_columns)

# Drop unprocessed ITEM columns from df_LOS
df_LOS.
↪drop(columns=["ITEMID","VALUE","STARTTIME-INTIME","ITEMID_STARTTIME-INTIME"],inplace=True)
df_LOS = df_LOS.drop_duplicates()

# Merge df with the processed ITEM columns
df_LOS = df_LOS.merge(df_pivoted, on='ICUSTAY_ID', how='left')
```

[ ]:
```
# Increase all display limits
pd.set_option('display.max_rows', None)  # Show all rows
pd.set_option('display.max_columns', None)  # Show all columns
pd.set_option('display.width', None)  # Auto-detect terminal width
pd.set_option('display.max_colwidth', None)  # No truncation

print(df_LOS.dtypes)
```

```
ICUSTAY_ID                            int64
HADM_ID                               int64
LOS                                 float64
INTIME                       datetime64[ns]
DBSOURCE                             object
INSURANCE                            object
MARITAL_STATUS                       object
ETHNICITY                            object
ADMISSION_LOCATION                   object
ADMISSION_TYPE                       object
SUBJECT_ID                            int64
ADMITTIME                            object
GENDER                               object
ICD9_CODE                             int64
AGE                                   int64
```

TIME_BETW_ADMISSION_ICU	float64
42731_DIAGNOSIS	bool
41401_DIAGNOSIS	bool
4019_DIAGNOSIS	bool
2724_DIAGNOSIS	bool
5849_DIAGNOSIS	bool
25000_DIAGNOSIS	bool
40390_DIAGNOSIS	bool
51881_DIAGNOSIS	bool
V5861_DIAGNOSIS	bool
5859_DIAGNOSIS	bool
NUMBER_ICUSTAY	int64
HR_0	float64
HR_1	float64
HR_2	float64
HR_3	float64
ABPs_0	float64
ABPs_1	float64
ABPs_2	float64
ABPs_3	float64
ABPm_0	float64
ABPm_1	float64
ABPm_2	float64
ABPm_3	float64
CVP_0	float64
CVP_1	float64
CVP_2	float64
CVP_3	float64
NBPm_0	float64
NBPm_1	float64
NBPm_2	float64
NBPm_3	float64
Dextrose 5%_0	float64
Dextrose 5%_1	float64
Dextrose 5%_2	float64
Dextrose 5%_3	float64
Midazolam (Versed)_0	float64
Midazolam (Versed)_1	float64
Midazolam (Versed)_2	float64
Midazolam (Versed)_3	float64
Phenylephrine_0	float64
Phenylephrine_1	float64
Phenylephrine_2	float64
Phenylephrine_3	float64
Furosemide (Lasix)_0	float64
Furosemide (Lasix)_1	float64
Furosemide (Lasix)_2	float64
Furosemide (Lasix)_3	float64

Norepinephrine_0	float64
Norepinephrine_1	float64
Norepinephrine_2	float64
Norepinephrine_3	float64
Propofol_0	float64
Propofol_1	float64
Propofol_2	float64
Propofol_3	float64
Insulin - Regular_0	float64
Insulin - Regular_1	float64
Insulin - Regular_2	float64
Insulin - Regular_3	float64
NaCl 0.9%_0	float64
NaCl 0.9%_1	float64
NaCl 0.9%_2	float64
NaCl 0.9%_3	float64
Potassium Chloride - KCL_0	float64
Potassium Chloride - KCL_1	float64
Potassium Chloride - KCL_2	float64
Potassium Chloride - KCL_3	float64
Vancomycin_0	float64
Vancomycin_1	float64
Vancomycin_2	float64
Vancomycin_3	float64
Multi Lumen_0	float64
Multi Lumen_1	float64
Multi Lumen_2	float64
Multi Lumen_3	float64
22 Gauge_0	float64
22 Gauge_1	float64
22 Gauge_2	float64
22 Gauge_3	float64
20 Gauge_0	float64
20 Gauge_1	float64
20 Gauge_2	float64
20 Gauge_3	float64
18 Gauge_0	float64
18 Gauge_1	float64
18 Gauge_2	float64
18 Gauge_3	float64
Blood Cultured_0	float64
Blood Cultured_1	float64
Blood Cultured_2	float64
Blood Cultured_3	float64
EKG_0	float64
EKG_1	float64
EKG_2	float64
EKG_3	float64

```
Transthoracic Echo_0                float64
Transthoracic Echo_1                float64
Transthoracic Echo_2                float64
Transthoracic Echo_3                float64
Chest X-Ray_0                       float64
Chest X-Ray_1                       float64
Chest X-Ray_2                       float64
Chest X-Ray_3                       float64
Arterial Line_0                     float64
Arterial Line_1                     float64
Arterial Line_2                     float64
Arterial Line_3                     float64
Invasive Ventilation_0              float64
Invasive Ventilation_1              float64
Invasive Ventilation_2              float64
Invasive Ventilation_3              float64
dtype: object
```

[ ]:
```python
df_LOS.drop(columns=['DBSOURCE','ICD9_CODE'],inplace=True)
```

[ ]:
```python
# Step 1: Create a mapping from ITEMID to ABBREVIATION
itemid_to_abbr = d_items.set_index('ITEMID')['ABBREVIATION'].to_dict()

# Step 2: Get the ABBREVIATIONs corresponding to boolean_itemids
boolean_abbreviations = [itemid_to_abbr[itemid] for itemid in boolean_itemids
                         if itemid in itemid_to_abbr]

print("ABBREVIATIONS to convert:", boolean_abbreviations)

# Step 3: Get all prefixes from column names
all_prefixes = {col.split('_')[0] for col in df_LOS.columns if '_' in col}

# Step 4: Find matching prefixes (case-sensitive)
prefixes_to_convert = [prefix for prefix in all_prefixes
                       if prefix in boolean_abbreviations]

print("Final prefixes to convert:", prefixes_to_convert)

# Step 6: Convert matching columns to boolean
for prefix in prefixes_to_convert:
    for col in [c for c in df_LOS.columns if c.startswith(prefix + '_')]:
        df_LOS[col] = (df_LOS[col] == 1).astype('boolean')

# Verification
print("\nSample converted columns:")
sample_cols = [c for prefix in prefixes_to_convert
               for c in df_LOS.columns if c.startswith(prefix + '_')][:5]
```

```
print(df_LOS[sample_cols].dtypes)
print(df_LOS[sample_cols].head())
```

```
ABBREVIATIONS to convert: ['Chest X-Ray', 'Blood Cultured', 'EKG',
'Transthoracic Echo', 'Extubation']
Final prefixes to convert: ['Blood Cultured', 'Chest X-Ray', 'Transthoracic
Echo', 'EKG']

Sample converted columns:
Blood Cultured_0    boolean
Blood Cultured_1    boolean
Blood Cultured_2    boolean
Blood Cultured_3    boolean
Chest X-Ray_0       boolean
dtype: object
   Blood Cultured_0  Blood Cultured_1  Blood Cultured_2  Blood Cultured_3  \
0             False             False             False             False
1             False             False             False             False
2             False             False             False             False
3             False             False             False             False
4             False             False             False             False

   Chest X-Ray_0
0          False
1           True
2           True
3          False
4          False
```

[ ]:
```
df_LOS.head()
```

[ ]:
```
    ICUSTAY_ID  HADM_ID     LOS              INTIME INSURANCE MARITAL_STATUS  \
0       200001   152234  3.0786 2181-11-25 19:06:12  Medicare         SINGLE
1       200011   121562  1.6330 2188-08-06 01:39:24  Medicare         SINGLE
2       200024   179633  0.3812 2127-03-03 16:09:07  Medicare        MARRIED
3       200028   181955  2.9038 2133-10-29 17:13:50  Medicare        MARRIED
4       200038   175436  1.0588 2143-10-24 20:35:24  Medicare        MARRIED

                 ETHNICITY         ADMISSION_LOCATION ADMISSION_TYPE  \
0                    ASIAN  CLINIC REFERRAL/PREMATURE      EMERGENCY
1                    WHITE  CLINIC REFERRAL/PREMATURE      EMERGENCY
2  BLACK/AFRICAN AMERICAN       EMERGENCY ROOM ADMIT      EMERGENCY
3                    WHITE  PHYS REFERRAL/NORMAL DELI       ELECTIVE
4                    WHITE       EMERGENCY ROOM ADMIT      EMERGENCY

   SUBJECT_ID            ADMITTIME GENDER  AGE  TIME_BETW_ADMISSION_ICU  \
0       55973  2181-11-18 16:04:00      F   61                    171.0
```

1	93535	2188-08-05 20:27:00	F	84	5.2
2	76603	2127-02-21 16:34:00	F	81	239.6
3	41710	2133-10-29 10:00:00	M	65	7.2
4	29708	2143-10-24 20:33:00	M	85	0.0

	42731_DIAGNOSIS	41401_DIAGNOSIS	4019_DIAGNOSIS	2724_DIAGNOSIS \
0	True	False	False	False
1	False	False	True	False
2	True	False	False	False
3	False	False	False	False
4	False	False	True	False

	5849_DIAGNOSIS	25000_DIAGNOSIS	40390_DIAGNOSIS	51881_DIAGNOSIS \
0	False	False	False	False
1	False	False	False	False
2	True	False	False	False
3	False	True	False	False
4	False	False	False	False

	V5861_DIAGNOSIS	5859_DIAGNOSIS	NUMBER_ICUSTAY	HR_0	HR_1	HR_2	HR_3 \
0	False	False	1	103.0	94.0	81.0	71.0
1	False	False	1	70.0	73.0	76.5	NaN
2	False	True	2	115.0	NaN	NaN	NaN
3	False	False	1	77.0	95.5	82.0	72.0
4	False	False	1	77.0	77.5	NaN	NaN

	ABPs_0	ABPs_1	ABPs_2	ABPs_3	ABPm_0	ABPm_1	ABPm_2	ABPm_3	CVP_0 \
0	NaN	113.5	114.0	107.5	NaN	70.0	70.0	69.5	NaN
1	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	93.5	NaN	NaN	NaN	79.0	NaN	NaN	NaN	NaN
3	106.5	120.0	138.0	NaN	64.0	69.0	81.0	NaN	13.0
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

	CVP_1	CVP_2	CVP_3	NBPm_0	NBPm_1	NBPm_2	NBPm_3	Dextrose 5%_0 \
0	NaN	NaN	NaN	70.0	65.5	NaN	62.5	0.000000
1	NaN	NaN	NaN	76.0	90.0	92.0	NaN	0.000000
2	NaN	NaN	NaN	81.0	NaN	NaN	NaN	1228.723358
3	5.5	NaN	NaN	59.0	NaN	70.0	76.0	0.000000
4	NaN	NaN	NaN	89.0	91.0	NaN	NaN	0.000000

	Dextrose 5%_1	Dextrose 5%_2	Dextrose 5%_3	Midazolam (Versed)_0 \
0	1020.206577	100.0	0.0	0.000000
1	0.000000	0.0	0.0	0.000000
2	0.000000	0.0	0.0	0.000000
3	0.000000	0.0	0.0	31.672712
4	0.000000	0.0	0.0	0.000000

	Midazolam (Versed)_1	Midazolam (Versed)_2	Midazolam (Versed)_3	\
0	0.0000	0.0	0.0	
1	0.0000	0.0	0.0	
2	0.0000	0.0	0.0	
3	0.8003	0.0	0.0	
4	0.0000	0.0	0.0	

	Phenylephrine_0	Phenylephrine_1	Phenylephrine_2	Phenylephrine_3	\
0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	

	Furosemide (Lasix)_0	Furosemide (Lasix)_1	Furosemide (Lasix)_2	\
0	0.000000	0.0	0.0	
1	20.000001	0.0	0.0	
2	0.000000	0.0	0.0	
3	0.000000	0.0	0.0	
4	0.000000	0.0	0.0	

	Furosemide (Lasix)_3	Norepinephrine_0	Norepinephrine_1	Norepinephrine_2	\
0	0.0	0.000000	0.000000	0.0	
1	0.0	0.000000	0.000000	0.0	
2	0.0	22.969218	0.000000	0.0	
3	0.0	12.986643	1.414368	0.0	
4	0.0	0.000000	0.000000	0.0	

	Norepinephrine_3	Propofol_0	Propofol_1	Propofol_2	Propofol_3	\
0	0.0	0.000000	0.0	0.0	0.0	
1	0.0	0.000000	0.0	0.0	0.0	
2	0.0	0.000000	0.0	0.0	0.0	
3	0.0	1962.927911	0.0	0.0	0.0	
4	0.0	0.000000	0.0	0.0	0.0	

	Insulin - Regular_0	Insulin - Regular_1	Insulin - Regular_2	\
0	0.0	0.0	0.0	
1	0.0	0.0	0.0	
2	10.0	0.0	0.0	
3	0.0	0.0	0.0	
4	0.0	0.0	0.0	

	Insulin - Regular_3	NaCl 0.9%_0	NaCl 0.9%_1	NaCl 0.9%_2	NaCl 0.9%_3	\
0	0.0	200.000006	0.000000	250.0	0.0	
1	0.0	0.000000	0.000000	0.0	0.0	
2	0.0	843.787998	0.000000	0.0	0.0	
3	0.0	2283.805288	484.999295	0.0	0.0	

```
4                  0.0   283.500011    10.800691          0.0          0.0

   Potassium Chloride - KCL_0  Potassium Chloride - KCL_1  \
0                         0.0                    0.000000
1                         0.0                    0.000000
2                         0.0                    0.000000
3                        20.0                   60.000001
4                         0.0                    0.000000

   Potassium Chloride - KCL_2  Potassium Chloride - KCL_3  Vancomycin_0  \
0                    0.000000                    0.000000           0.0
1                    0.000000                    0.000000           0.0
2                    0.000000                    0.000000           0.0
3                   40.000001                   40.000001           0.0
4                    0.000000                    0.000000           0.0

   Vancomycin_1  Vancomycin_2  Vancomycin_3  Multi Lumen_0  Multi Lumen_1  \
0           0.0           0.0           0.0            0.0            0.0
1           0.0           0.0           0.0            0.0            0.0
2           0.0           0.0           0.0         1056.0            0.0
3           0.0           0.0           0.0         3758.0            0.0
4           0.0           0.0           0.0            0.0            0.0

   Multi Lumen_2  Multi Lumen_3  22 Gauge_0  22 Gauge_1  22 Gauge_2  \
0            0.0            0.0         0.0         0.0         0.0
1            0.0            0.0      2346.0         0.0         0.0
2            0.0            0.0         0.0         0.0         0.0
3            0.0            0.0         0.0         0.0         0.0
4            0.0            0.0         0.0         0.0         0.0

   22 Gauge_3  20 Gauge_0  20 Gauge_1  20 Gauge_2  20 Gauge_3  18 Gauge_0  \
0         0.0         0.0         0.0         0.0         0.0         0.0
1         0.0         0.0         0.0         0.0         0.0         0.0
2         0.0         0.0         0.0         0.0         0.0         0.0
3         0.0         0.0         0.0         0.0         0.0         0.0
4         0.0      1470.0         0.0         0.0         0.0      1470.0

   18 Gauge_1  18 Gauge_2  18 Gauge_3  Blood Cultured_0  Blood Cultured_1  \
0         0.0         0.0         0.0             False             False
1         0.0         0.0         0.0             False             False
2         0.0         0.0         0.0             False             False
3         0.0         0.0         0.0             False             False
4         0.0         0.0         0.0             False             False

   Blood Cultured_2  Blood Cultured_3  EKG_0  EKG_1  EKG_2  EKG_3  \
0             False             False  False  False  False  False
1             False             False  False  False  False  False
```

```
2            False            False  False  False  False  False
3            False            False  False   True  False  False
4            False            False  False  False  False  False

   Transthoracic Echo_0  Transthoracic Echo_1  Transthoracic Echo_2  \
0                 False                 False                 False
1                 False                 False                 False
2                 False                 False                 False
3                 False                 False                 False
4                 False                 False                 False

   Transthoracic Echo_3  Chest X-Ray_0  Chest X-Ray_1  Chest X-Ray_2  \
0                 False          False          False           True
1                 False           True          False          False
2                 False           True          False          False
3                 False          False          False          False
4                 False          False          False          False

   Chest X-Ray_3  Arterial Line_0  Arterial Line_1  Arterial Line_2  \
0          False              0.0           2710.0              0.0
1          False              0.0              0.0              0.0
2          False            498.0              0.0              0.0
3          False           2347.0              0.0              0.0
4          False              0.0              0.0              0.0

   Arterial Line_3  Invasive Ventilation_0  Invasive Ventilation_1  \
0              0.0                     0.0                     0.0
1              0.0                     0.0                     0.0
2              0.0                   360.0                     0.0
3              0.0                   931.0                     0.0
4              0.0                     0.0                     0.0

   Invasive Ventilation_2  Invasive Ventilation_3
0                     0.0                     0.0
1                     0.0                     0.0
2                     0.0                     0.0
3                     0.0                     0.0
4                     0.0                     0.0
```

[ ]:
```python
# Get the current list of columns
cols = df_LOS.columns.tolist()

# Remove the columns we want to move
cols.remove('SUBJECT_ID')
cols.remove('ADMITTIME')
cols.remove('INTIME')
```

```python
# Insert them at the desired positions (index 2 and 3 for 3rd and 4th columns)
cols.insert(2, 'SUBJECT_ID')
cols.insert(3, 'ADMITTIME')
cols.insert(4, 'INTIME')

# Reassign the columns in the new order
df_LOS = df_LOS[cols]
```

[ ]:
```python
df_LOS.to_csv('df_LOS_final.csv', index=False)
```

[ ]:
```python
df_LOS.head()
```

[ ]:
```
   ICUSTAY_ID  HADM_ID  SUBJECT_ID            ADMITTIME              INTIME  \
0      200001   152234       55973  2181-11-18 16:04:00 2181-11-25 19:06:12
1      200011   121562       93535  2188-08-05 20:27:00 2188-08-06 01:39:24
2      200024   179633       76603  2127-02-21 16:34:00 2127-03-03 16:09:07
3      200028   181955       41710  2133-10-29 10:00:00 2133-10-29 17:13:50
4      200038   175436       29708  2143-10-24 20:33:00 2143-10-24 20:35:24

      LOS INSURANCE MARITAL_STATUS               ETHNICITY  \
0  3.0786  Medicare         SINGLE                   ASIAN
1  1.6330  Medicare         SINGLE                   WHITE
2  0.3812  Medicare        MARRIED  BLACK/AFRICAN AMERICAN
3  2.9038  Medicare        MARRIED                   WHITE
4  1.0588  Medicare        MARRIED                   WHITE

           ADMISSION_LOCATION ADMISSION_TYPE GENDER  AGE  \
0  CLINIC REFERRAL/PREMATURE      EMERGENCY      F   61
1  CLINIC REFERRAL/PREMATURE      EMERGENCY      F   84
2       EMERGENCY ROOM ADMIT      EMERGENCY      F   81
3  PHYS REFERRAL/NORMAL DELI       ELECTIVE      M   65
4       EMERGENCY ROOM ADMIT      EMERGENCY      M   85

   TIME_BETW_ADMISSION_ICU  42731_DIAGNOSIS  41401_DIAGNOSIS  4019_DIAGNOSIS  \
0                    171.0             True            False           False
1                      5.2            False            False            True
2                    239.6             True            False           False
3                      7.2            False            False           False
4                      0.0            False            False            True

   2724_DIAGNOSIS  5849_DIAGNOSIS  25000_DIAGNOSIS  40390_DIAGNOSIS  \
0           False           False            False            False
1           False           False            False            False
2           False            True            False            False
3           False           False             True            False
4           False           False            False            False
```

	51881_DIAGNOSIS	V5861_DIAGNOSIS	5859_DIAGNOSIS	NUMBER_ICUSTAY	HR_0	\
0	False	False	False	1	103.0	
1	False	False	False	1	70.0	
2	False	False	True	2	115.0	
3	False	False	False	1	77.0	
4	False	False	False	1	77.0	

	HR_1	HR_2	HR_3	ABPs_0	ABPs_1	ABPs_2	ABPs_3	ABPm_0	ABPm_1	ABPm_2	\
0	94.0	81.0	71.0	NaN	113.5	114.0	107.5	NaN	70.0	70.0	
1	73.0	76.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	93.5	NaN	NaN	NaN	79.0	NaN	NaN	
3	95.5	82.0	72.0	106.5	120.0	138.0	NaN	64.0	69.0	81.0	
4	77.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	

	ABPm_3	CVP_0	CVP_1	CVP_2	CVP_3	NBPm_0	NBPm_1	NBPm_2	NBPm_3	\
0	69.5	NaN	NaN	NaN	NaN	70.0	65.5	NaN	62.5	
1	NaN	NaN	NaN	NaN	NaN	76.0	90.0	92.0	NaN	
2	NaN	NaN	NaN	NaN	NaN	81.0	NaN	NaN	NaN	
3	NaN	13.0	5.5	NaN	NaN	59.0	NaN	70.0	76.0	
4	NaN	NaN	NaN	NaN	NaN	89.0	91.0	NaN	NaN	

	Dextrose 5%_0	Dextrose 5%_1	Dextrose 5%_2	Dextrose 5%_3	\
0	0.000000	1020.206577	100.0	0.0	
1	0.000000	0.000000	0.0	0.0	
2	1228.723358	0.000000	0.0	0.0	
3	0.000000	0.000000	0.0	0.0	
4	0.000000	0.000000	0.0	0.0	

	Midazolam (Versed)_0	Midazolam (Versed)_1	Midazolam (Versed)_2	\
0	0.000000	0.0000	0.0	
1	0.000000	0.0000	0.0	
2	0.000000	0.0000	0.0	
3	31.672712	0.8003	0.0	
4	0.000000	0.0000	0.0	

	Midazolam (Versed)_3	Phenylephrine_0	Phenylephrine_1	Phenylephrine_2	\
0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	

	Phenylephrine_3	Furosemide (Lasix)_0	Furosemide (Lasix)_1	\
0	0.0	0.000000	0.0	
1	0.0	20.000001	0.0	
2	0.0	0.000000	0.0	
3	0.0	0.000000	0.0	

```
4              0.0              0.000000                   0.0

   Furosemide (Lasix)_2  Furosemide (Lasix)_3  Norepinephrine_0  \
0                   0.0                   0.0          0.000000
1                   0.0                   0.0          0.000000
2                   0.0                   0.0         22.969218
3                   0.0                   0.0         12.986643
4                   0.0                   0.0          0.000000

   Norepinephrine_1  Norepinephrine_2  Norepinephrine_3   Propofol_0  \
0          0.000000               0.0               0.0     0.000000
1          0.000000               0.0               0.0     0.000000
2          0.000000               0.0               0.0     0.000000
3          1.414368               0.0               0.0  1962.927911
4          0.000000               0.0               0.0     0.000000

   Propofol_1  Propofol_2  Propofol_3  Insulin - Regular_0  \
0         0.0         0.0         0.0                  0.0
1         0.0         0.0         0.0                  0.0
2         0.0         0.0         0.0                 10.0
3         0.0         0.0         0.0                  0.0
4         0.0         0.0         0.0                  0.0

   Insulin - Regular_1  Insulin - Regular_2  Insulin - Regular_3  NaCl 0.9%_0  \
0                  0.0                  0.0                  0.0   200.000006
1                  0.0                  0.0                  0.0     0.000000
2                  0.0                  0.0                  0.0   843.787998
3                  0.0                  0.0                  0.0  2283.805288
4                  0.0                  0.0                  0.0   283.500011

   NaCl 0.9%_1  NaCl 0.9%_2  NaCl 0.9%_3  Potassium Chloride - KCL_0  \
0     0.000000        250.0          0.0                         0.0
1     0.000000          0.0          0.0                         0.0
2     0.000000          0.0          0.0                         0.0
3   484.999295          0.0          0.0                        20.0
4    10.800691          0.0          0.0                         0.0

   Potassium Chloride - KCL_1  Potassium Chloride - KCL_2  \
0                    0.000000                    0.000000
1                    0.000000                    0.000000
2                    0.000000                    0.000000
3                   60.000001                   40.000001
4                    0.000000                    0.000000

   Potassium Chloride - KCL_3  Vancomycin_0  Vancomycin_1  Vancomycin_2  \
0                    0.000000           0.0           0.0           0.0
1                    0.000000           0.0           0.0           0.0
```

2	0.000000	0.0	0.0	0.0
3	40.000001	0.0	0.0	0.0
4	0.000000	0.0	0.0	0.0

	Vancomycin_3	Multi Lumen_0	Multi Lumen_1	Multi Lumen_2	Multi Lumen_3 \
0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0
2	0.0	1056.0	0.0	0.0	0.0
3	0.0	3758.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0

	22 Gauge_0	22 Gauge_1	22 Gauge_2	22 Gauge_3	20 Gauge_0	20 Gauge_1 \
0	0.0	0.0	0.0	0.0	0.0	0.0
1	2346.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	1470.0	0.0

	20 Gauge_2	20 Gauge_3	18 Gauge_0	18 Gauge_1	18 Gauge_2	18 Gauge_3 \
0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	1470.0	0.0	0.0	0.0

	Blood Cultured_0	Blood Cultured_1	Blood Cultured_2	Blood Cultured_3 \
0	False	False	False	False
1	False	False	False	False
2	False	False	False	False
3	False	False	False	False
4	False	False	False	False

	EKG_0	EKG_1	EKG_2	EKG_3	Transthoracic Echo_0	Transthoracic Echo_1 \
0	False	False	False	False	False	False
1	False	False	False	False	False	False
2	False	False	False	False	False	False
3	False	True	False	False	False	False
4	False	False	False	False	False	False

	Transthoracic Echo_2	Transthoracic Echo_3	Chest X-Ray_0	Chest X-Ray_1 \
0	False	False	False	False
1	False	False	True	False
2	False	False	True	False
3	False	False	False	False
4	False	False	False	False

	Chest X-Ray_2	Chest X-Ray_3	Arterial Line_0	Arterial Line_1 \

```
0           True          False              0.0           2710.0
1          False          False              0.0              0.0
2          False          False            498.0              0.0
3          False          False           2347.0              0.0
4          False          False              0.0              0.0

   Arterial Line_2  Arterial Line_3  Invasive Ventilation_0  \
0              0.0              0.0                     0.0
1              0.0              0.0                     0.0
2              0.0              0.0                   360.0
3              0.0              0.0                   931.0
4              0.0              0.0                     0.0

   Invasive Ventilation_1  Invasive Ventilation_2  Invasive Ventilation_3
0                     0.0                     0.0                     0.0
1                     0.0                     0.0                     0.0
2                     0.0                     0.0                     0.0
3                     0.0                     0.0                     0.0
4                     0.0                     0.0                     0.0
```

3 Individual Patient Visualization

3.1 Textual Summary

In this section, we create a subset of the full dataset focused on one specific patient to explore and visualize their ICU observations in more detail. We begin by reading the base dataframe, and select a single patient (in this case, SUBJECT_ID = 199) with a moderate, representative Length of Stay (LOS) of 5.25 days. This duration provides enough data for meaningful temporal analysis without being overwhelming and matches the average LOS stay. Therefore, this patient may be representative of the population.

To convert INTIME and DISCHTIME to visualizable, meaningful quantities from which we can extract patterns, we transform them from dates with timestamps to relative time (in days) starting with the hospital ADMITTIME. This allows us to align event measurements with the patient's overall hospitalization timeline.

For the selected patient, ICU admission occurred 5.63 days after hospital admission, with a stay length of 5.25 days, leading to discharge on day 10. This establishes the observation window for which we will extract relevant vitals and interventions in the following steps.

In the following analysis, we included the chart events, as well as taken medication and procedures. Due to memory constraints in Google Colab, we pre-filtered the CHARTEVENTS table locally to include only data from patients with our selected ICD9 code and loaded this preprocessed table.

Even just in the chartevents table, there were 124 different Item IDs for our patient. To keep the analysis and visualization clinically meaningful as well as manageable, we selected only the most frequently recorded items (those appearing more than 50 times). These are likely to include vital signs and essential indicators commonly monitored in ICU.

Furthermore, to produce a visualization with interpretable axes, we kept only chart items measured

in bpm (beats per minute - as heart rate is a highly indicative measurement) and mmHg (millimeters of mercury - as many events were recorded in this unit) so that we can have one y-axis labelled in bpm and one axis labelled with mmHg. This ensures high interpretability as well as no misleading combination of units. Lastly, we produced a second plot which only included the mean value instead of mean, systolic, and diastolic value for the measurements which were recorded in this way to make the visualization more compact.

For medication (inputevents), we again faced a high number of items. Therefore, we decided to only include inputs administered at least 20 times for our visualization. The most frequently administered items included NaCl 0.9% (standard IV fluid), Dextrose 5%(another common fluid for energy support), Insulin (Regular) (for glucose control), as well as Norepinephrine and Propofol (indicate hemodynamic support and sedation). These inputs reflect a typical ICU treatment profile, with both supportive fluids and critical medications. For medication that was administered over a longer time horizon, we computed the duration as this should be treated differently than one-time treatments. The median duration of a medication or fluid administration was 0.05 days (~1.2 hours), with most interventions lasting under 0.1 days. A few longer infusions — including one lasting up to 1.69 days — suggest continuous drips like insulin or fluid maintenance.

Lastly, we analyzed all documented procedures for the hospital admission. The patient underwent several procedures during the ICU stay, including: 1. **Arterial Line** and **Cordis/Introducer** for invasive monitoring 2. **Intubation**, **Invasive Ventilation**, and **Extubation** for respiratory management 3. Multiple **Chest X-Rays**, **EKGs**, and **Urine Cultures** for diagnostics

These procedures, especially those involving ventilation and monitoring, highlight the severity and complexity of the case. To focus on clinically meaningful procedures, we kept only a subset of events related to respiratory support and ICU-specific interventions. Events such as "Nasal Swab" do not any hold valuable information and would add complexity without providing benefits.

3.2 Code

[ ]:
```
import pandas as pd
# get basic information
df_basic = pd.read_csv("base_df.
↪csv",usecols=["HADM_ID","ICUSTAY_ID","LOS","INTIME","SUBJECT_ID"])
```

[ ]:
```
df_basic.head() # SUBJECT 199 has a LOS of 5.25 which could be a good amount to␣
↪visualize
```

[ ]:

	HADM_ID	LOS	ICUSTAY_ID	INTIME	SUBJECT_ID
0	194340	2.4370	217232	2129-09-03 12:31:31	305
1	155897	0.5712	249202	2144-07-01 04:13:59	111
2	138376	22.9075	256064	2166-01-09 13:05:18	124
3	170252	0.9628	247247	2170-10-03 17:17:53	165
4	125590	5.2517	235675	2174-04-12 11:00:25	199

[ ]:
```
subject_id=199
df_basic=df_basic[df_basic["SUBJECT_ID"]==subject_id]
hadm_id = df_basic['HADM_ID'].iloc[0] # we will visualize the first admission
df_basic=df_basic[df_basic["HADM_ID"]==hadm_id]
```

```
icustay_id = df_basic['ICUSTAY_ID'].iloc[0]
df_basic=df_basic[df_basic["ICUSTAY_ID"]==icustay_id]
```

[ ]:
```
admissions_sub = pd.read_csv("ADMISSIONS.csv",␣
↪usecols=["HADM_ID","DISCHTIME","DEATHTIME","ADMITTIME"])
df_basic=df_basic.merge(admissions_sub, on='HADM_ID', how='left')
```

[ ]:
```
df_basic # patient did not die in the hospital
```

[ ]:
```
   HADM_ID     LOS  ICUSTAY_ID               INTIME  SUBJECT_ID  \
0   125590  5.2517      235675  2174-04-12 11:00:25         199

              ADMITTIME            DISCHTIME DEATHTIME
0  2174-04-06 19:50:00  2174-04-19 14:45:00       NaN
```

[ ]:
```
df_basic.drop(columns=["DEATHTIME"],inplace=True)
```

[ ]:
```
# Get the current list of columns
cols = df_basic.columns.tolist()

# Remove the columns we want to move
cols.remove('LOS')
cols.remove('SUBJECT_ID')
cols.remove('ADMITTIME')

# Insert them at the desired positions (index 2 and 3 for 3rd and 4th columns)
cols.insert(0, 'SUBJECT_ID')
cols.insert(3, 'ADMITTIME')
cols.insert(5, 'LOS')

# Reassign the columns in the new order
df_basic = df_basic[cols]
```

[ ]:
```
df_basic
```

[ ]:
```
   SUBJECT_ID  HADM_ID  ICUSTAY_ID            ADMITTIME               INTIME  \
0         199   125590      235675  2174-04-06 19:50:00  2174-04-12 11:00:25

      LOS            DISCHTIME
0  5.2517  2174-04-19 14:45:00
```

[ ]:
```
 # Convert columns to datetime
df_basic['INTIME'] = pd.to_datetime(df_basic['INTIME'])
df_basic['DISCHTIME'] = pd.to_datetime(df_basic['DISCHTIME'])
admittime = pd.to_datetime(df_basic['ADMITTIME'].iloc[0])  # Get the single␣
↪ADMITTIME value
```

```
# Calculate time difference in days, rounded to 2 decimals
df_basic['INTIME'] = round(
    (df_basic['INTIME'] - admittime).dt.total_seconds() / (3600 * 24),
    2
)
df_basic['DISCHTIME'] = round(
    (df_basic['DISCHTIME'] - admittime).dt.total_seconds() / (3600 * 24),
    2
)
```

[ ]:
```
df_basic["OUTTIME"]=df_basic["INTIME"]+df_basic["LOS"]
```

[ ]:
```
df_basic
```

[ ]:
```
   SUBJECT_ID  HADM_ID  ICUSTAY_ID            ADMITTIME  INTIME     LOS  \
0         199   125590      235675  2174-04-06 19:50:00    5.63  5.2517

   DISCHTIME  OUTTIME
0      12.79  10.8817
```

[ ]:
```
# We filtered the event files locally to only include SUBJECT_IDs with our ICD9␣
↪code due to memory limits in Google Colab
chartevents_sub = pd.read_csv('CHARTEVENTS_SUBJECT.csv',␣
↪usecols=["SUBJECT_ID","ITEMID","VALUE", "ICUSTAY_ID", "CHARTTIME","HADM_ID"])
chartevents_sub=chartevents_sub[chartevents_sub["HADM_ID"]==hadm_id]
```

```
<ipython-input-38-bc6664c4883a>:2: DtypeWarning: Columns (5) have mixed types.
Specify dtype option on import or set low_memory=False.
  chartevents_sub = pd.read_csv('CHARTEVENTS_SUBJECT.csv',
usecols=["SUBJECT_ID","ITEMID","VALUE", "ICUSTAY_ID", "CHARTTIME","HADM_ID"])
```

[ ]:
```
chartevents_sub.shape
```

[ ]:
```
(2313, 6)
```

[ ]:
```
chartevents_sub.head()
```

[ ]:

	SUBJECT_ID	HADM_ID	ICUSTAY_ID	ITEMID	CHARTTIME	VALUE
11607	199	125590	235675.0	220180	2174-04-17 06:00:00	47.0
11608	199	125590	235675.0	220181	2174-04-17 06:00:00	76.0
11609	199	125590	235675.0	220210	2174-04-17 06:00:00	16.0
11610	199	125590	235675.0	220277	2174-04-17 06:00:00	95.0
11611	199	125590	235675.0	225664	2174-04-17 06:00:00	160.0

[ ]:
```
chartevents_sub["ITEMID"].unique()
```

[ ]:
```
array([220180, 220181, 220210, 220277, 225664, 220045, 220179, 220050,
       220051, 220052, 220059, 220060, 220061, 220074, 226329, 220228,
       220545, 220546, 220602, 220615, 225624, 227443, 227457, 227465,
       227466, 227467, 220224, 220235, 223830, 224828, 225667, 225698,
       226534, 226537, 227464, 223772, 223773, 224842, 220227, 220292,
       220293, 220339, 223835, 220621, 223873, 223875, 223876, 223962,
       224417, 224418, 224684, 224685, 224687, 224688, 224689, 224690,
       224695, 224696, 224697, 224700, 224738, 224839, 226871, 226873,
       227442, 223874, 224169, 224686, 224421, 224422, 224751, 224753,
       224755, 220046, 220047, 223751, 223752, 223761, 220056, 220058,
       220063, 220066, 220072, 220073, 223769, 223770, 224161, 224162,
       226253, 223958, 223960, 223963, 224752, 224754, 224837, 223834,
       224172, 227565, 227566, 225674, 220734, 227471, 224701, 227455,
       225668, 226531, 226512, 226707, 226730, 220635, 226540, 220645,
       225625, 225677, 227073, 227516, 224838, 220274, 226536, 227468,
       220587, 220644, 225612, 225690])
```

[ ]:
```
chartevents_sub["ITEMID"].value_counts()
```

[ ]:
```
ITEMID
220277    134
220210    132
220045    132
220181     85
220180     85
          …
227468      1
220587      1
220644      1
225612      1
225690      1
Name: count, Length: 124, dtype: int64
```

[ ]:
```
item_counts = chartevents_sub["ITEMID"].value_counts()
frequent_items = item_counts[item_counts > 50]
frequent_items
```

[ ]:
```
ITEMID
220277    134
220210    132
220045    132
220181     85
220180     85
220179     85
220050     78
220051     78
220052     78
```

```
225664     64
220074     57
220061     57
220060     56
220059     56
226329     55
224842     53
223772     51
Name: count, dtype: int64
```

[ ]:
```python
frequent_items_list = frequent_items.index.tolist()

# Filter d_items where ITEMID is in the top list
d_items=pd.read_csv('D_ITEMS.csv')
matching_rows = d_items[d_items['ITEMID'].isin(frequent_items_list)]

matching_rows
```

[ ]:
```
       ROW_ID  ITEMID                                  LABEL  \
10535   14111  225664                   Glucose finger stick
10951   13444  224842                   Cardiac Output (CCO)
11498   12712  220045                              Heart Rate
11502   12716  220050       Arterial Blood Pressure systolic
11503   12717  220051      Arterial Blood Pressure diastolic
11504   12718  220052           Arterial Blood Pressure mean
11507   12721  220059     Pulmonary Artery Pressure systolic
11508   12722  220060    Pulmonary Artery Pressure diastolic
11509   12723  220061         Pulmonary Artery Pressure mean
11515   12729  220074                Central Venous Pressure
11520   12734  220179   Non Invasive Blood Pressure systolic
11521   12735  220180  Non Invasive Blood Pressure diastolic
11522   12736  220181       Non Invasive Blood Pressure mean
11524   12738  220210                       Respiratory Rate
12254   14446  226329              Blood Temperature CCO (C)
12355   12746  220277            O2 saturation pulseoxymetry
12377   12768  223772                                   SvO2

               ABBREVIATION    DBSOURCE      LINKSTO             CATEGORY  \
10535  Glucose finger stick  metavision  chartevents                 Labs
10951              CO (CCO)  metavision  chartevents         Hemodynamics
11498                    HR  metavision  chartevents  Routine Vital Signs
11502                  ABPs  metavision  chartevents  Routine Vital Signs
11503                  ABPd  metavision  chartevents  Routine Vital Signs
11504                  ABPm  metavision  chartevents  Routine Vital Signs
11507                  PAPs  metavision  chartevents         Hemodynamics
11508                  PAPd  metavision  chartevents         Hemodynamics
11509                  PAPm  metavision  chartevents         Hemodynamics
```

```
11515                   CVP  metavision  chartevents         Hemodynamics
11520                  NBPs  metavision  chartevents  Routine Vital Signs
11521                  NBPd  metavision  chartevents  Routine Vital Signs
11522                  NBPm  metavision  chartevents  Routine Vital Signs
11524                    RR  metavision  chartevents          Respiratory
12254    Blood Temp CCO (C)  metavision  chartevents  Routine Vital Signs
12355                  SpO2  metavision  chartevents          Respiratory
12377                  SvO2  metavision  chartevents         Hemodynamics

       UNITNAME PARAM_TYPE  CONCEPTID
10535       NaN    Numeric        NaN
10951     L/min    Numeric        NaN
11498       bpm    Numeric        NaN
11502      mmHg    Numeric        NaN
11503      mmHg    Numeric        NaN
11504      mmHg    Numeric        NaN
11507      mmHg    Numeric        NaN
11508      mmHg    Numeric        NaN
11509      mmHg    Numeric        NaN
11515      mmHg    Numeric        NaN
11520      mmHg    Numeric        NaN
11521      mmHg    Numeric        NaN
11522      mmHg    Numeric        NaN
11524  insp/min    Numeric        NaN
12254        ?C    Numeric        NaN
12355         %    Numeric        NaN
12377         %    Numeric        NaN
```

[ ]:
```
# Step 1: Filter frequent_items_list to keep only items with "bpm" or "mmHg"␣
↪units
# We will keep heart rate as well as blood pressure because it is highly␣
↪indicative
# additional consideration: many items with unit mmHg will make them more␣
↪comparable for visualization
filtered_items = d_items[
    (d_items['ITEMID'].isin(frequent_items_list)) &
    (d_items['UNITNAME'].isin(['bpm', 'mmHg']))
]['ITEMID'].unique().tolist()

# Step 2: Filter chartevents_sub for these ITEMIDs
chartevents_sub = chartevents_sub[chartevents_sub['ITEMID'].
↪isin(filtered_items)]
```

[ ]:
```
chartevents_sub = chartevents_sub.merge(d_items[['ITEMID', 'LABEL',␣
↪'ABBREVIATION', 'UNITNAME']], on='ITEMID', how='left')
```

[ ]:
```
chartevents_sub.head()
```

[ ]:
```
   SUBJECT_ID  HADM_ID  ICUSTAY_ID  ITEMID            CHARTTIME  VALUE  \
0         199   125590    235675.0  220180  2174-04-17 06:00:00   47.0
1         199   125590    235675.0  220181  2174-04-17 06:00:00   76.0
2         199   125590    235675.0  220045  2174-04-17 07:00:00   83.0
3         199   125590    235675.0  220179  2174-04-17 07:00:00  133.0
4         199   125590    235675.0  220180  2174-04-17 07:00:00   79.0

                                   LABEL ABBREVIATION UNITNAME
0  Non Invasive Blood Pressure diastolic         NBPd     mmHg
1       Non Invasive Blood Pressure mean         NBPm     mmHg
2                             Heart Rate           HR      bpm
3   Non Invasive Blood Pressure systolic         NBPs     mmHg
4  Non Invasive Blood Pressure diastolic         NBPd     mmHg
```

[ ]:
```
# Convert columns to datetime
chartevents_sub['CHARTTIME'] = pd.to_datetime(chartevents_sub['CHARTTIME'])
admittime = pd.to_datetime(df_basic['ADMITTIME'].iloc[0])  # Get the single␣
↪ADMITTIME value

# Calculate CHARTTIME - ADMITTIME (time difference in days, rounded to 2␣
↪decimals)
chartevents_sub['CHARTTIME'] = round(
    (chartevents_sub['CHARTTIME'] - admittime).dt.total_seconds() / (3600 * 24),
    2
)
```

[ ]:
```
chartevents_sub['ITEMID'].unique()
```

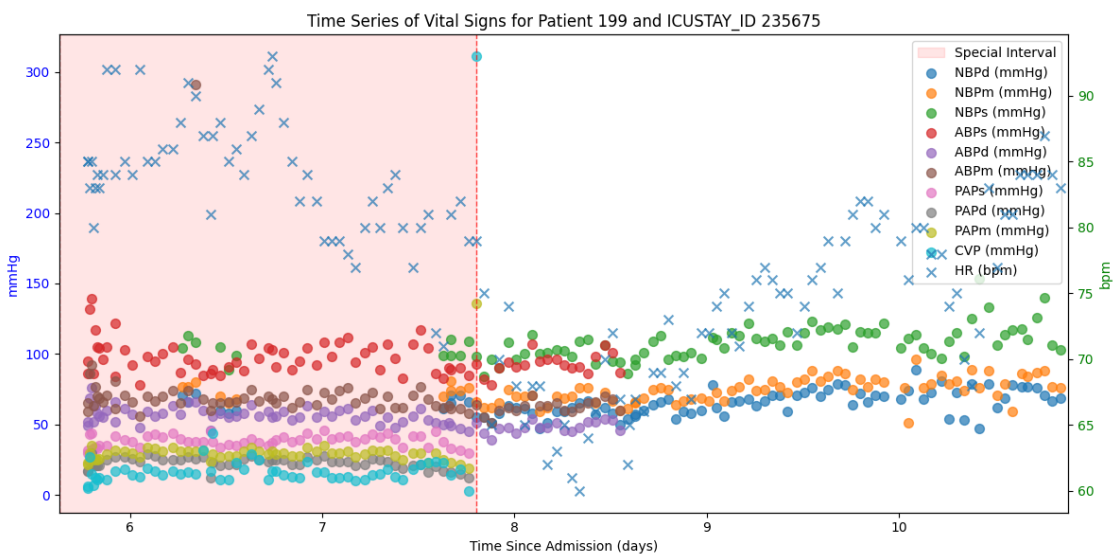
[ ]:
```
array([220180, 220181, 220045, 220179, 220050, 220051, 220052, 220059,
       220060, 220061, 220074])
```

[ ]:
```
chartevents_sub.head()
```

[ ]:
```
   SUBJECT_ID  HADM_ID  ICUSTAY_ID  ITEMID  CHARTTIME  VALUE  \
0         199   125590    235675.0  220180      10.42   47.0
1         199   125590    235675.0  220181      10.42   76.0
2         199   125590    235675.0  220045      10.47   83.0
3         199   125590    235675.0  220179      10.47  133.0
4         199   125590    235675.0  220180      10.47   79.0

                                   LABEL ABBREVIATION UNITNAME
0  Non Invasive Blood Pressure diastolic         NBPd     mmHg
1       Non Invasive Blood Pressure mean         NBPm     mmHg
2                             Heart Rate           HR      bpm
3   Non Invasive Blood Pressure systolic         NBPs     mmHg
4  Non Invasive Blood Pressure diastolic         NBPd     mmHg
```

[ ]:
```
import matplotlib.pyplot as plt
# we visualize only the ICU Stay - we have no chartered events outside of the␣
↪ICU Stay)

# Extract key timestamps (already in float/days format)
intime = df_basic["INTIME"].iloc[0]
outtime = df_basic["OUTTIME"].iloc[0]
# Find last CVP measurement time (new right boundary)
last_cvp_time = chartevents_sub[chartevents_sub["ABBREVIATION"] ==␣
↪"PAPm"]["CHARTTIME"].max()

# Initialize the plot
fig, ax1 = plt.subplots(figsize=(12, 6))
ax1.set_xlim(intime, outtime)
ax1.set_xlabel("Time Since Admission (days)")

# Add vertical lines and shaded region
ax1.axvline(intime, color='red', linestyle='--', linewidth=1, alpha=0.7)
ax1.axvline(last_cvp_time, color='red', linestyle='--', linewidth=1, alpha=0.7)
ax1.axvspan(intime, last_cvp_time, color='red', alpha=0.1, label='Special␣
↪Interval')

# --- Prepare data ---
# Separate data by unit (mmHg vs. bpm)
df_mmHg = chartevents_sub[chartevents_sub["UNITNAME"] == "mmHg"]
df_bpm = chartevents_sub[chartevents_sub["UNITNAME"] == "bpm"]

# Get unique abbreviations for legend
abbreviations = chartevents_sub["ABBREVIATION"].unique()

# --- Plot mmHg data (left y-axis) ---
for abbr in abbreviations:
    subset = df_mmHg[df_mmHg["ABBREVIATION"] == abbr]
    if not subset.empty:
        ax1.scatter(
            subset["CHARTTIME"],
            subset["VALUE"],
            label=f"{abbr} (mmHg)",
            alpha=0.7,
            s=50  # Marker size
        )
ax1.set_ylabel("mmHg", color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# --- Plot bpm data (right y-axis) ---
ax2 = ax1.twinx()
for abbr in abbreviations:
```

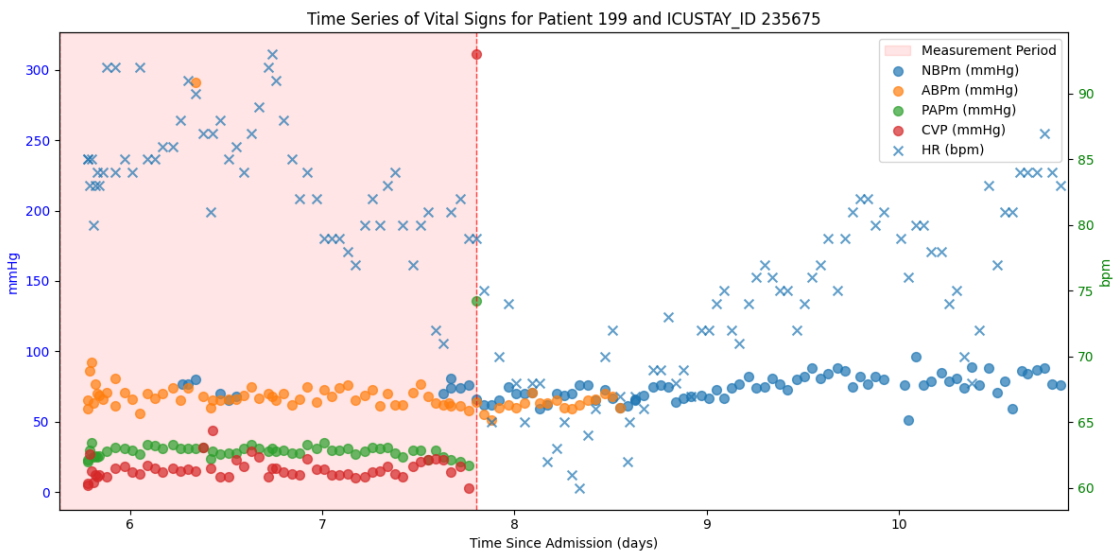
```
    subset = df_bpm[df_bpm["ABBREVIATION"] == abbr]
    if not subset.empty:
        ax2.scatter(
            subset["CHARTTIME"],
            subset["VALUE"],
            marker='x',  # Different marker for bpm
            label=f"{abbr} (bpm)",
            alpha=0.7,
            s=50
        )
ax2.set_ylabel("bpm", color='green')
ax2.tick_params(axis='y', labelcolor='green')

# Add legend and title
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

plt.title(f"Time Series of Vital Signs for Patient {subject_id} and ICUSTAY_ID␣
↪{icustay_id}")
plt.tight_layout()
plt.show()

plt.savefig("vitals_plot1.png", dpi=300)
```



[ ]:
```
# Modify plot for better visualization (less variables)
import matplotlib.pyplot as plt
```

```python
# Get key timestamps (float format in days)
intime = df_basic["INTIME"].iloc[0]
outtime = df_basic["OUTTIME"].iloc[0]

# Find last CVP measurement time (new right boundary)
last_cvp_time = chartevents_sub[chartevents_sub["ABBREVIATION"] ==␣
↪"PAPm"]["CHARTTIME"].max()

# Filter out abbreviations ending with 's' or 'd'
valid_abbrs = [abbr for abbr in chartevents_sub["ABBREVIATION"].unique()
               if not (abbr.endswith('s') or abbr.endswith('d'))]

# Initialize plot with adjusted x-axis limits
fig, ax1 = plt.subplots(figsize=(12, 6))
ax1.set_xlim(intime, outtime)  # Updated x-axis range
ax1.set_xlabel("Time Since Admission (days)")

# Add vertical lines and shaded region
ax1.axvline(intime, color='red', linestyle='--', linewidth=1, alpha=0.7)
ax1.axvline(last_cvp_time, color='red', linestyle='--', linewidth=1, alpha=0.7)
ax1.axvspan(intime, last_cvp_time, color='red', alpha=0.1, label='Measurement␣
↪Period')

# Prepare data subsets
df_mmHg = chartevents_sub[(chartevents_sub["UNITNAME"] == "mmHg") &
                          (chartevents_sub["ABBREVIATION"].isin(valid_abbrs))]
df_bpm = chartevents_sub[(chartevents_sub["UNITNAME"] == "bpm") &
                         (chartevents_sub["ABBREVIATION"].isin(valid_abbrs))]

# Plot mmHg data (left axis)
for abbr in valid_abbrs:
    subset = df_mmHg[df_mmHg["ABBREVIATION"] == abbr]
    if not subset.empty:
        ax1.scatter(subset["CHARTTIME"], subset["VALUE"], label=f"{abbr}␣
↪(mmHg)", alpha=0.7, s=50)
ax1.set_ylabel("mmHg", color='blue')
ax1.tick_params(axis='y', labelcolor='blue')

# Plot bpm data (right axis)
ax2 = ax1.twinx()
for abbr in valid_abbrs:
    subset = df_bpm[df_bpm["ABBREVIATION"] == abbr]
    if not subset.empty:
        ax2.scatter(subset["CHARTTIME"], subset["VALUE"], marker='x',
                    label=f"{abbr} (bpm)", alpha=0.7, s=50)
ax2.set_ylabel("bpm", color='green')
ax2.tick_params(axis='y', labelcolor='green')
```

```
# Combine legends
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

plt.title(f"Time Series of Vital Signs for Patient {subject_id} and ICUSTAY_ID␣
↪{icustay_id}")
plt.tight_layout()
plt.show()

plt.savefig("vitals_plot2.png", dpi=300)
```



[ ]:
```
inputevents_sub = pd.read_csv('INPUTEVENTS_MV_SUBJECT.csv',␣
↪usecols=['ICUSTAY_ID','HADM_ID','STARTTIME', 'ENDTIME', 'ITEMID',␣
↪'AMOUNT','RATE'])
inputevents_sub=inputevents_sub[inputevents_sub["HADM_ID"]==hadm_id]
inputevents_sub.shape
```

[ ]: (593, 7)

[ ]:
```
inputevents_sub["ITEMID"].value_counts()
```

[ ]:
```
ITEMID
225158    125
220949     76
223258     60
221906     48
```

```
225943     44
222168     38
221794     26
221986     17
225150     15
225893     14
226452     14
225166     14
225828      9
227522      9
221668      7
221744      7
226089      7
221456      7
225798      6
225942      6
225859      6
225154      5
225975      5
221289      4
225168      3
226453      3
223262      3
225799      3
222011      3
223260      2
227523      2
225911      2
225944      1
226372      1
226364      1
Name: count, dtype: int64
```

[ ]:
```
item_counts = inputevents_sub["ITEMID"].value_counts()
frequent_items = item_counts[item_counts > 20]
frequent_items
frequent_items_list = frequent_items.index.tolist()

# Filter d_items where ITEMID is in the top list
d_items=pd.read_csv('D_ITEMS.csv')
matching_rows = d_items[d_items['ITEMID'].isin(frequent_items_list)]
matching_rows
```

[ ]:
```
       ROW_ID  ITEMID               LABEL        ABBREVIATION    DBSOURCE  \
9548    12979  221794  Furosemide (Lasix)  Furosemide (Lasix)  metavision
9553    12984  221906      Norepinephrine      Norepinephrine  metavision
9564    12995  222168            Propofol            Propofol  metavision
```

```
9572    13003  223258   Insulin - Regular   Insulin - Regular  metavision
11675   14375  225943            Solution            Solution  metavision
12075   12865  220949         Dextrose 5%         Dextrose 5%  metavision
12126   13758  225158           NaCl 0.9%           NaCl 0.9%  metavision

              LINKSTO       CATEGORY UNITNAME PARAM_TYPE  CONCEPTID
9548   inputevents_mv    Medications       mg   Solution        NaN
9553   inputevents_mv    Medications       mg   Solution        NaN
9564   inputevents_mv    Medications       mg   Solution        NaN
9572   inputevents_mv    Medications    units   Solution        NaN
11675  inputevents_mv  Fluids/Intake       mL   Solution        NaN
12075  inputevents_mv  Fluids/Intake       mL   Solution        NaN
12126  inputevents_mv  Fluids/Intake       mL   Solution        NaN
```

[ ]:
```
# Step 2: Filter inputevents_sub for these ITEMIDs
inputevents_sub = inputevents_sub[inputevents_sub['ITEMID'].
 ↪isin(frequent_items_list)]
inputevents_sub = inputevents_sub.merge(d_items[['ITEMID', 'ABBREVIATION',␣
 ↪'UNITNAME']], on='ITEMID', how='left')
```

[ ]:
```
inputevents_sub.head()
```

[ ]:
```
   HADM_ID  ICUSTAY_ID            STARTTIME              ENDTIME  ITEMID  \
0   125590    235675.0  2174-04-15 09:32:00  2174-04-15 09:33:00  220949
1   125590    235675.0  2174-04-13 10:30:00  2174-04-13 10:51:00  222168
2   125590    235675.0  2174-04-13 10:30:00  2174-04-13 10:51:00  225943
3   125590    235675.0  2174-04-13 18:02:00  2174-04-13 20:41:00  221906
4   125590    235675.0  2174-04-13 18:02:00  2174-04-13 20:41:00  225158

       AMOUNT       RATE    ABBREVIATION UNITNAME
0  200.000000        NaN     Dextrose 5%       mL
1   45.652181  15.594788        Propofol       mg
2    4.565219  13.043483        Solution       mL
3    1.329201   0.059970  Norepinephrine       mg
4   41.537507  15.674531       NaCl 0.9%       mL
```

[ ]:
```
# Transform data to be visualized
# Convert columns to datetime
inputevents_sub['STARTTIME'] = pd.to_datetime(inputevents_sub['STARTTIME'])
inputevents_sub['ENDTIME'] = pd.to_datetime(inputevents_sub['ENDTIME'])
admittime = pd.to_datetime(df_basic['ADMITTIME'].iloc[0])  # Get the single␣
 ↪ADMITTIME value

# Calculate time difference in days, rounded to 2 decimals
inputevents_sub['STARTTIME'] = round(
    (inputevents_sub['STARTTIME'] - admittime).dt.total_seconds() / (3600 * 24),
    2
```

```
)
inputevents_sub['ENDTIME'] = round(
    (inputevents_sub['ENDTIME'] - admittime).dt.total_seconds() / (3600 * 24),
    2
)
inputevents_sub['INPUTTIME'] = round(
    (inputevents_sub['ENDTIME'] - inputevents_sub['STARTTIME']),
    2
)
```

[ ]:
```
inputevents_sub['INPUTTIME'] = round((inputevents_sub['ENDTIME'] -␣
↪inputevents_sub['STARTTIME']),2)
inputevents_sub['AMOUNT'] = round(inputevents_sub['AMOUNT'],0)
```

[ ]:
```
inputevents_sub.head()
```

[ ]:
```
   HADM_ID  ICUSTAY_ID  STARTTIME  ENDTIME  ITEMID  AMOUNT       RATE  \
0   125590    235675.0       8.57     8.57  220949   200.0        NaN
1   125590    235675.0       6.61     6.63  222168    46.0  15.594788
2   125590    235675.0       6.61     6.63  225943     5.0  13.043483
3   125590    235675.0       6.92     7.04  221906     1.0   0.059970
4   125590    235675.0       6.92     7.04  225158    42.0  15.674531

     ABBREVIATION UNITNAME  INPUTTIME
0     Dextrose 5%       mL       0.00
1        Propofol       mg       0.02
2        Solution       mL       0.02
3  Norepinephrine       mg       0.12
4       NaCl 0.9%       mL       0.12
```

[ ]:
```
inputevents_sub["INPUTTIME"].describe()
```

[ ]:
```
count    417.000000
mean       0.099520
std        0.176889
min        0.000000
25%        0.010000
50%        0.050000
75%        0.100000
max        1.690000
Name: INPUTTIME, dtype: float64
```

[ ]:
```
inputevents_sub.nlargest(10, 'INPUTTIME')
```

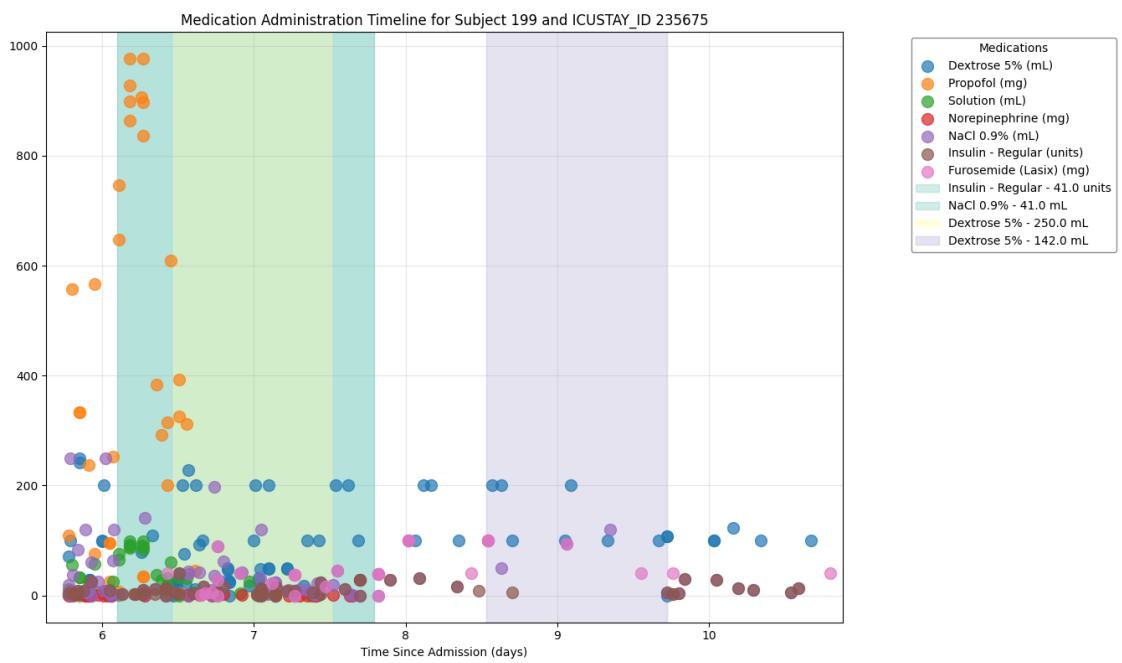
[ ]:
```
     HADM_ID  ICUSTAY_ID  STARTTIME  ENDTIME  ITEMID  AMOUNT       RATE  \
77    125590    235675.0       6.10     7.79  223258    41.0   1.000000
336   125590    235675.0       6.10     7.79  225158    41.0   1.000000
```

```
316   125590    235675.0       8.53     9.72  220949   142.0   5.000000
49    125590    235675.0       6.47     7.51  220949   250.0  10.000000
213   125590    235675.0       6.57     7.20  220949   228.0  14.999999
45    125590    235675.0       8.02     8.54  221794   100.0   8.042896
46    125590    235675.0       8.02     8.54  225158   100.0   8.042895
75    125590    235675.0       8.54     9.06  221794   100.0   7.992001
76    125590    235675.0       8.54     9.06  225158   100.0   7.992000
276   125590    235675.0       8.02     8.54  221794   100.0   7.994861

           ABBREVIATION UNITNAME  INPUTTIME
77    Insulin - Regular    units       1.69
336           NaCl 0.9%       mL       1.69
316         Dextrose 5%       mL       1.19
49          Dextrose 5%       mL       1.04
213         Dextrose 5%       mL       0.63
45   Furosemide (Lasix)       mg       0.52
46            NaCl 0.9%       mL       0.52
75   Furosemide (Lasix)       mg       0.52
76            NaCl 0.9%       mL       0.52
276  Furosemide (Lasix)       mg       0.52
```

[ ]:
```python
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.colors import ListedColormap
import matplotlib.patches as mpatches
from itertools import cycle

# Split data into two groups
short_events = inputevents_sub[inputevents_sub["INPUTTIME"] < 1]
long_events = inputevents_sub[inputevents_sub["INPUTTIME"] >= 1]

# Setup figure and axis
fig, ax = plt.subplots(figsize=(14, 8))
intime = df_basic["INTIME"].iloc[0]
outtime = df_basic["OUTTIME"].iloc[0]
ax.set_xlim(intime, outtime)
ax.set_xlabel("Time Since Admission (days)")

# Create a color cycle for unique time ranges
cmap = ListedColormap(plt.cm.Set3.colors)
color_cycle = cycle(cmap.colors)

# Create a dictionary to track colors for each unique time range
time_range_colors = {}
legend_handles = []

# --- Plot long events FIRST (so they stay in background) ---
```

```python
# First group by time ranges to assign consistent colors
grouped = long_events.groupby(['STARTTIME', 'ENDTIME', 'ABBREVIATION',␣
↪'AMOUNT', 'UNITNAME'])

# Assign colors to each unique time range
for (start, end, abbr, amount, unit), group in grouped:
    time_key = (start, end)

    # If this time range hasn't been seen before, assign a new color
    if time_key not in time_range_colors:
        time_range_colors[time_key] = next(color_cycle)

    # Plot all events with this time range
    color = time_range_colors[time_key]
    ax.axvspan(
        start,
        end,
        alpha=0.4,
        color=color,
        zorder=1  # Force to background
    )

    # Create a legend entry (only once per time range and abbreviation)
    label = f"{abbr} - {amount} {unit}"
    if label not in [h.get_label() for h in legend_handles]:
        patch = mpatches.Patch(
            color=color,
            alpha=0.4,
            label=label
        )
        legend_handles.append(patch)

# --- Then plot short events (scatter on top) ---
for abbr in short_events["ABBREVIATION"].unique():
    subset = short_events[short_events["ABBREVIATION"] == abbr]
    if not subset.empty:
        unit = subset["UNITNAME"].iloc[0]
        scatter = ax.scatter(
            subset["STARTTIME"],
            subset["AMOUNT"],
            label=f"{abbr} ({unit})",
            alpha=0.7,
            s=100,
            zorder=2  # Middle layer
        )
        legend_handles.append(scatter)
```

```python
# Create the legend with all handles
# First put scatter points at the top, then shaded regions
scatter_handles = [h for h in legend_handles if not isinstance(h, mpatches.
 ↪Patch)]
patch_handles = [h for h in legend_handles if isinstance(h, mpatches.Patch)]
all_handles = scatter_handles + patch_handles

# Create the legend with specified ordering
ax.legend(
    handles=all_handles,
    loc='upper right',
    bbox_to_anchor=(1.35, 1),  # Slightly wider to accommodate longer labels
    framealpha=0.9,
    edgecolor='gray',
    title="Medications"
)

plt.title(f"Medication Administration Timeline for Subject {subject_id} and␣
 ↪ICUSTAY_ID {icustay_id}")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plt.savefig("medication_plot.png", dpi=300)
```



[ ]:
```
procedureevents_sub = pd.read_csv('PROCEDUREEVENTS_MV.csv',␣
↪usecols=['SUBJECT_ID','HADM_ID','STARTTIME', 'ENDTIME',␣
↪'ITEMID','VALUE','VALUEUOM'])
procedureevents_sub=procedureevents_sub[procedureevents_sub["HADM_ID"]==hadm_id]
procedureevents_sub.shape
```

[ ]: (21, 7)

[ ]:
```
procedureevents_labelled = procedureevents_sub.merge(d_items[['ITEMID',␣
↪'LABEL', 'ABBREVIATION', 'UNITNAME']], on='ITEMID', how='left')
```

[ ]:
```
procedureevents_labelled["LABEL"].unique()
```

[ ]:
```
array(['OR Received', '20 Gauge', 'CCO PAC', '16 Gauge', 'Arterial Line',
       'Cordis/Introducer', 'Intubation', 'Invasive Ventilation',
       'Nasal Swab', 'Chest X-Ray', 'EKG', 'Urine Culture', 'Extubation',
       'Chest Tube Removed', '18 Gauge'], dtype=object)
```

[ ]:
```
procedureevents_labelled
```

[ ]:

	SUBJECT_ID	HADM_ID	STARTTIME	ENDTIME	ITEMID	\
0	199	125590	2174-04-12 14:23:00	2174-04-12 14:24:00	225469	
1	199	125590	2174-04-12 14:23:00	2174-04-12 14:24:00	225469	
2	199	125590	2174-04-12 14:33:00	2174-04-12 20:03:00	224275	
3	199	125590	2174-04-12 14:33:00	2174-04-14 21:04:00	224269	
4	199	125590	2174-04-12 14:33:00	2174-04-15 13:13:00	224276	
5	199	125590	2174-04-12 14:33:00	2174-04-15 10:27:00	225752	
6	199	125590	2174-04-12 14:33:00	2174-04-16 11:00:00	224267	
7	199	125590	2174-04-12 14:33:00	2174-04-16 11:00:00	224276	
8	199	125590	2174-04-12 14:50:00	2174-04-12 14:51:00	224385	
9	199	125590	2174-04-12 14:53:00	2174-04-14 07:22:00	225792	
10	199	125590	2174-04-12 14:58:00	2174-04-12 14:59:00	225966	
11	199	125590	2174-04-12 15:02:00	2174-04-12 15:03:00	225459	
12	199	125590	2174-04-12 15:17:00	2174-04-12 15:18:00	225402	
13	199	125590	2174-04-12 16:00:00	2174-04-12 16:01:00	225454	
14	199	125590	2174-04-13 08:30:00	2174-04-13 08:31:00	225459	
15	199	125590	2174-04-14 07:26:00	2174-04-14 07:27:00	227194	
16	199	125590	2174-04-14 11:04:00	2174-04-14 11:05:00	227712	
17	199	125590	2174-04-14 11:05:00	2174-04-14 11:06:00	225459	
18	199	125590	2174-04-15 08:00:00	2174-04-15 08:01:00	225459	
19	199	125590	2174-04-16 10:00:00	2174-04-17 17:02:00	224277	
20	199	125590	2174-04-16 20:43:00	2174-04-17 17:02:00	224275	

	VALUE	VALUEUOM	LABEL	ABBREVIATION	UNITNAME
0	1.0	NaN	OR Received	OR Received	NaN
1	1.0	NaN	OR Received	OR Received	NaN
2	330.0	min	20 Gauge	20 Gauge	NaN

```
3   3271.0      min              CCO PAC              CCO PAC      NaN
4   4240.0      min             16 Gauge             16 Gauge      NaN
5   4074.0      min        Arterial Line        Arterial Line      NaN
6   5547.0      min    Cordis/Introducer    Cordis/Introducer      NaN
7   5547.0      min             16 Gauge             16 Gauge      NaN
8      1.0      NaN           Intubation           Intubation      NaN
9   2429.0      min  Invasive Ventilation  Invasive Ventilation      NaN
10     1.0      NaN           Nasal Swab           Nasal Swab      NaN
11     1.0      NaN          Chest X-Ray          Chest X-Ray      NaN
12     1.0      NaN                  EKG                  EKG      NaN
13     1.0      NaN        Urine Culture        Urine Culture      NaN
14     1.0      NaN          Chest X-Ray          Chest X-Ray      NaN
15     1.0      NaN           Extubation           Extubation      NaN
16     1.0      NaN    Chest Tube Removed    Chest Tube Removed      NaN
17     1.0      NaN          Chest X-Ray          Chest X-Ray      NaN
18     1.0      NaN          Chest X-Ray          Chest X-Ray      NaN
19  1862.0      min             18 Gauge             18 Gauge      NaN
20  1219.0      min             20 Gauge             20 Gauge      NaN
```

[ ]:
```
# Transform data to be visualized
# Convert columns to datetime
procedureevents_sub['STARTTIME'] = pd.
 ↪to_datetime(procedureevents_sub['STARTTIME'])
procedureevents_sub['ENDTIME'] = pd.to_datetime(procedureevents_sub['ENDTIME'])
admittime = pd.to_datetime(df_basic['ADMITTIME'].iloc[0])  # Get the single␣
 ↪ADMITTIME value

# Calculate time difference in days, rounded to 2 decimals
procedureevents_sub['STARTTIME'] = round(
    (procedureevents_sub['STARTTIME'] - admittime).dt.total_seconds() / (3600 *␣
 ↪24),
    2
)
procedureevents_sub['ENDTIME'] = round(
    (procedureevents_sub['ENDTIME'] - admittime).dt.total_seconds() / (3600 *␣
 ↪24),
    2
)
procedureevents_sub['INPUTTIME'] = round(
    (procedureevents_sub['ENDTIME'] - procedureevents_sub['STARTTIME']),
    2
)
```

[ ]:
```
procedureevents_sub=procedureevents_sub.merge(d_items[['ITEMID',␣
 ↪'ABBREVIATION']], on='ITEMID', how='left')
```

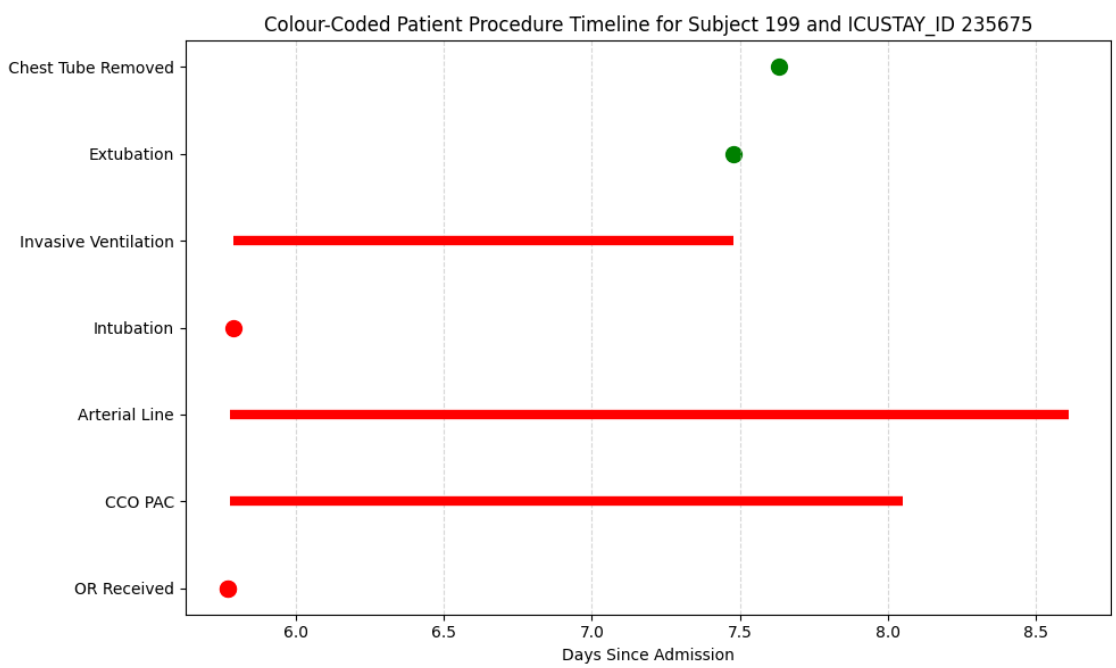
[ ]:
```
important_abbreviations = [
    'Intubation', 'Invasive Ventilation', 'Extubation', 'Chest Tube Removed',
    'Arterial Line', 'CCO PAC', 'OR Received'
]

procedureevents_sub = procedureevents_sub[procedureevents_sub['ABBREVIATION'].
 ↪isin(important_abbreviations)]
```

[ ]:
```
procedureevents_sub
```

[ ]:
```
     SUBJECT_ID  HADM_ID  STARTTIME  ENDTIME  ITEMID   VALUE VALUEUOM  \
0           199   125590       5.77     5.77  225469     1.0      NaN
1           199   125590       5.77     5.77  225469     1.0      NaN
3           199   125590       5.78     8.05  224269  3271.0      min
5           199   125590       5.78     8.61  225752  4074.0      min
8           199   125590       5.79     5.79  224385     1.0      NaN
9           199   125590       5.79     7.48  225792  2429.0      min
15          199   125590       7.48     7.48  227194     1.0      NaN
16          199   125590       7.63     7.64  227712     1.0      NaN

     INPUTTIME          ABBREVIATION
0         0.00           OR Received
1         0.00           OR Received
3         2.27               CCO PAC
5         2.83         Arterial Line
8         0.00            Intubation
9         1.69  Invasive Ventilation
15        0.00            Extubation
16        0.01    Chest Tube Removed
```

[ ]:
```
import matplotlib.pyplot as plt
import pandas as pd

pd.options.mode.chained_assignment = None  # Turn off warning
# Step 1: Define good and bad events
good_events = ['Extubation', 'Chest Tube Removed']
bad_events = ['Intubation', 'Invasive Ventilation', 'Arterial Line', 'CCO PAC',␣
 ↪'OR Received']

# Step 2: Assign colors
def get_color(abbrev):
    if abbrev in good_events:
        return 'green'
    elif abbrev in bad_events:
        return 'red'
    else:
        return 'gray'
```

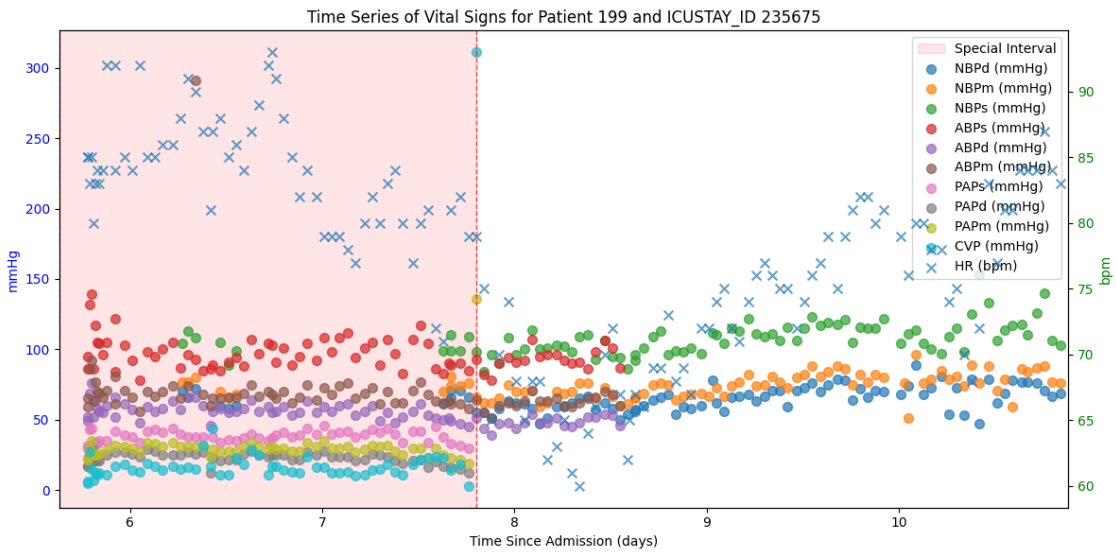
```
procedureevents_sub['color'] = procedureevents_sub['ABBREVIATION'].
 ↪apply(get_color)

# Step 3: Create the plot
fig, ax = plt.subplots(figsize=(10, 6))

for idx, row in procedureevents_sub.iterrows():
    duration = row['ENDTIME'] - row['STARTTIME']
    if duration < 0.1:  # If duration is very short, plot a dot
        ax.scatter(x=row['STARTTIME'], y=row['ABBREVIATION'],
                   color=row['color'], s=100)  # s=100 makes the dot big
    else:  # Otherwise plot a line as before
        ax.hlines(y=row['ABBREVIATION'], xmin=row['STARTTIME'],␣
 ↪xmax=row['ENDTIME'],
                  color=row['color'], linewidth=6)

# Step 4: Format plot
ax.set_xlabel('Days Since Admission')
ax.set_title(f'Colour-Coded Patient Procedure Timeline for Subject {subject_id}␣
 ↪and ICUSTAY_ID {icustay_id}')
plt.grid(True, axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

plt.savefig("procedures_plot.png", dpi=300)
```



3.3 Result: Visualization of Individual Patient Data

[ ]:
```
from IPython.display import Image
Image(filename='vitals_plot1.png')
```

[ ]:



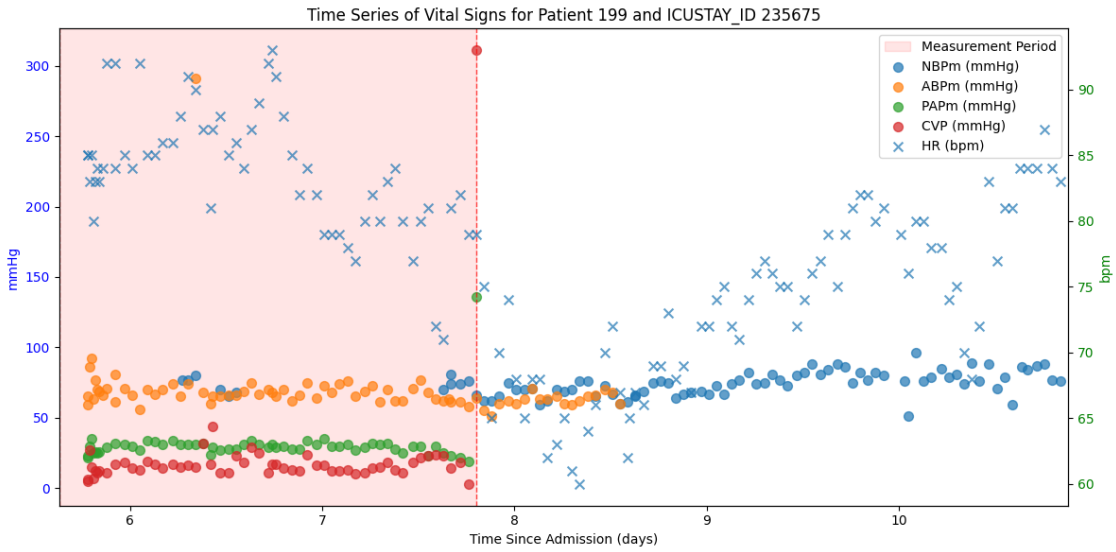
This plot visualizes the most crucial vital signs charted during the ICU stay, aligned by time since hospital admission. It should be noted that the plot only covers the span of the ICU stay as there were no measurements before the admission to the ICU as well as after the discharge of the ICU. The x-axis shows the number of days since admission, while the y-axes represent the values of vitals in mmHg (left, blue) and bpm (right, green). Vital signs include non-invasive blood pressure (NBP), arterial blood pressure (ABP), pulmonary arterial pressure (PAP), central venous pressure (CVP), and heart rate, each categorized by diastolic, mean, or systolic values.

This graph allows us to see when key measurements were taken and how vital signs fluctuated over time. Frequent and tightly clustered measurements reflect periods of intensive monitoring.

The shaded red region marks an interval with special characteristics. Within this interval, there are generally elevated HR values, possibly suggesting hemodynamic instability or an acute clinical event. Moreover, we can see that there are several measurements that were only taken during this interval, indicating a need for closer observation. After this period, HR appears to decrease and PAP as well as CVP are not monitored anymore. This suggests that the patient was at high risk during the marked interval with stabilization around day 8.

[ ]:
```
Image(filename='vitals_plot2.png')
```

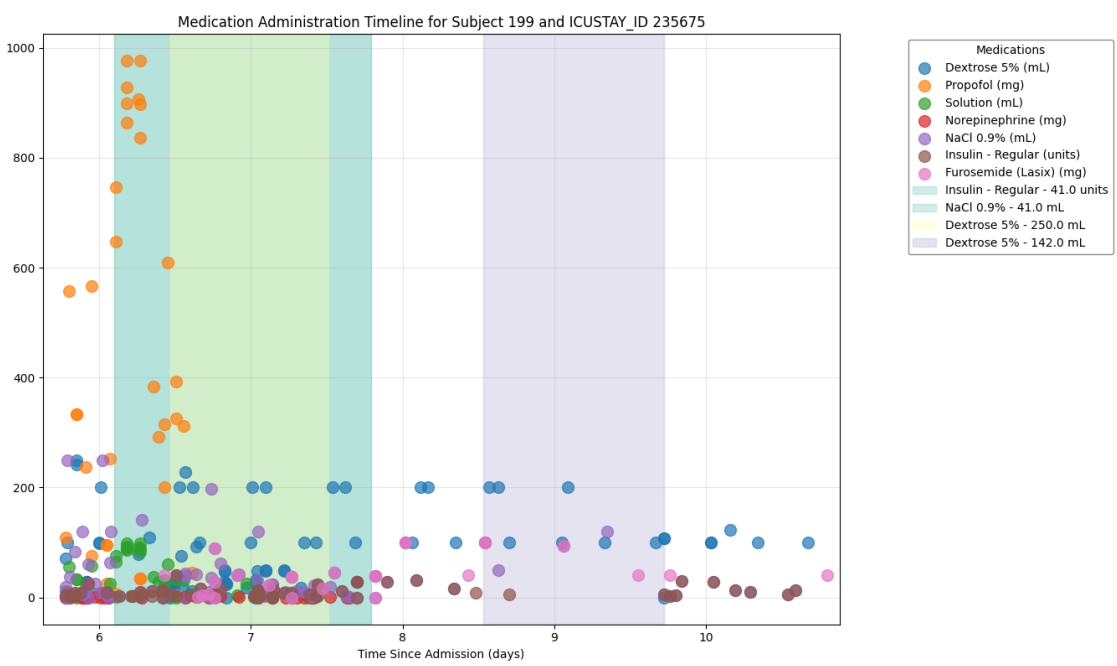
[ ]:



To improve readability, this plot visualizes only the mean values for measurements that were taken as systolic as well as diastolic. and retained only more stable vital sign indicators. As mentioned above, we can also here see a period of special measurement and care.

[ ]:
```
Image(filename='medication_plot.png')
```

[ ]:



This plot overlays the most important short and long administration events for the ICU stay. **Short events** (under 1 day) are shown as **scatter points**, with size and color indicating dosage

and type. **Longer infusions** are visualized as **shaded time spans**, showing the full duration of administration.

This combined timeline allows us to see when treatments started, how long they lasted, and which medications were administered in parallel.

This visualization provides a clear picture of treatment dynamics during the ICU stay. For example: - **Dextrose 5%** was administered twice over a longer time period, consistent with fluid support. - **Propofol** and **Norepinephrine** were used for sedation and blood pressure regulation, both key indicators of critical illness. - **Insulin** and **NaCi 0.9%** were administered together over the longest span (up to 1.69 days), suggesting glucose control efforts.

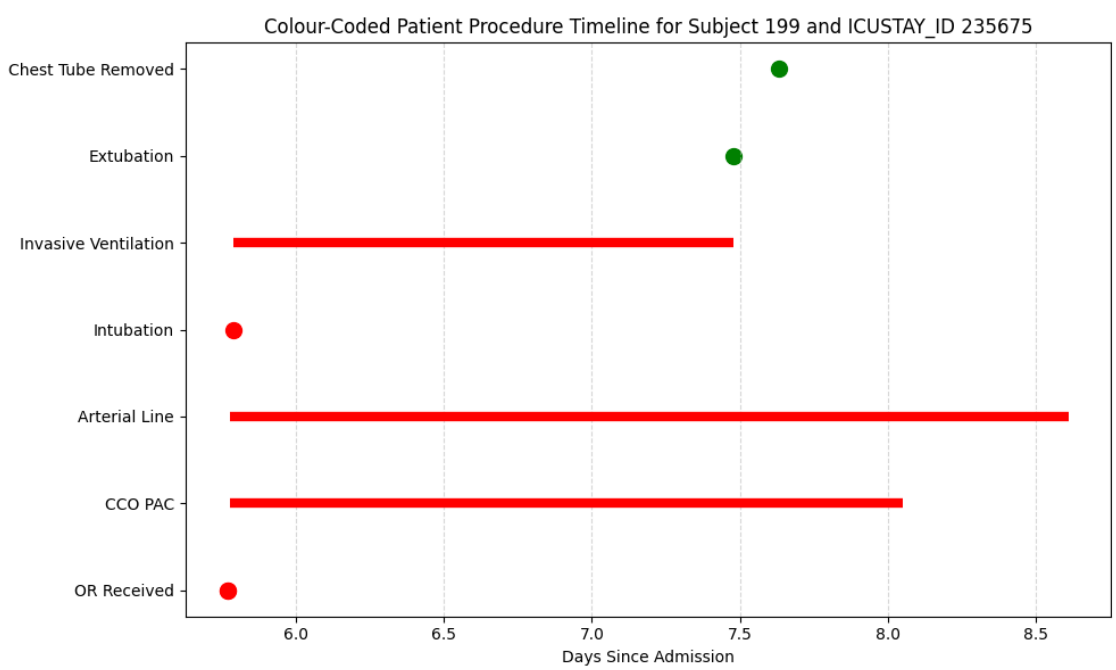
Notably, a substantial amount of Propofol, a sedative commonly used during mechanical ventilation or invasive procedures, was administered between days 5.8 and 7.5, suggesting a critical care phase requiring sedation. During this interval, also Norepinephrine, a vasopressor, was given, indicating efforts to manage hypotension — a common complication in congestive heart failure (CHF). The presence of insulin may reflect stress hyperglycemia or underlying diabetes.

Generally, we can see a much higher density of points during from ICU admission until approximately day 8. This confirms our findings from before that the patient was at high risk during this period.

Please note that the label "Solution" is not further specified in the database, although it shows up frequently. It may refer to sterile water or saline used to dilute medications.

[ ]:
```
Image(filename='procedures_plot.png')
```

[ ]:



In this plot, we visualize the most imporant procedures by their timing and duration. **Green**

markers represent "positive" or recovery events (e.g., extubation). **Red** indicates invasive or high-intensity interventions (e.g., arterial line or CCO PAC).

Short-duration procedures are shown as **dots**, while longer procedures like ventilation are shown as **horizontal bars** across their duration.

We can clearly observe the patient's trajectory. The ICU stay begins with **intubation and invasive ventilation**, suggesting acute respiratory failure. Also **Arterial lines** and **CCO PAC** are inserted also in the beggining — indicating close hemodynamic monitoring. Around **day 7.5**, the patient is **extubated** and has the **chest tube removed**, marking clinical improvement. This perfectly aligns with the findings from earlier visualizations which indicate a recovery phase starting around day 7.5.

4 Statistical Analysis

In this section, we will explore various statistical aspects of the data including value distributions, group statistics, and relationships between features. This understanding is essential before building predictive models. As our goal is to just explore before the predicting, we also include mortality data at this stage (though this is not part of the prediction dataframe as this data can obviously not be used for prediction purposes).

4.1 Data Exploration

4.1.1 Textual Summary

First, we created the function comprehensive_summary() to generate a full statistical and visual summary of the dataset. The goal is to quickly understand the structure, completeness, and distribution of each type of variable. This helps ensure data quality and supports informed modeling decisions.

With this function, we can observe the data shape and types as well as the percentage of missing values per column (to be visualized in a horizontal bar chart) to identify features with a lot of missing data. It also shows summary statistics (mean, std, min, etc.) for numeric variables and plots the distribution and boxplot of the top six numeric variables, giving insight into skewness, outliers, and general variability. For categorical variables, it shows the number of unique categories and a count plot of the most frequent ones; it also detects and plots any time-series columns with a naming pattern like feature_0, feature_1, etc, showing the mean trend over time for up to 3 time-series groups.

By running this function on the cleaned df_LOS dataframe, we get a detailed picture of what the data looks like before proceeding to modeling.

From the visualizations, we can observe that there are a lot of columns with a high percentage of missing data. They are CVP_3, CVP_2, ABPs_3, ABPm_3, CVP_0, ABPs_2, ABPm_2, CVP_1, ABPs_0, ABPm_0, ABPS_1, ABPM_1, NBPm_3 and HR_3 (from higher percentage to lower), all with a percentage above 50% of missing data. This was already expected as these are the features generated from the chart events. Not all measurements are taken for every parient and especially not every day. In contrast to the features generated from input and procedure events, it would also not be meaningful to simply impute 0.

From the numeric summary, we observe that the average ICU Length of Stay (LOS) is approximately 4 days, with a large standard deviation (around 5 days). It suggests considerable variability across patients, with some staying just a few hours and others over 50 days. The AGE variable has a surprisingly high mean (95 years). Most patients are older adults, with the interquartile range (IQR) from 65 to 84 years, reinforcing that this is a primarily elderly ICU population. The high mean is, however, due to a few individuals which are recorded to have an age of 300. Further research found that this is because the Mimic III database assigns all patients older than 89 the age 300 for privacy reasons. According to the Mimic III Data Dictionary, the median age of this group is 91. Therefore, we will replace the value 300 with 91 in our further analysis.

For TIME_BETW_ADMISSION_ICU, the median is 0, meaning many patients are transferred directly into the ICU at admission. However, the high mean and maximum values show that for some patients, there is a significant delay (possibly days or even weeks) before ICU entry. The variable NUMBER_ICUSTAY has a mean slightly above 1, confirming that most patients had a single ICU stay during their hospital admission, but a few had up to 5. Some of the time-series features (e.g., Invasive Ventilation_, Arterial Line_) show extremely large maximum values and high standard deviations. This may indicate strong right-skewed distributions or the presence of outliers.

From the distributions and boxplots, we can observe that LOS is right-skewed and there are a lot of outliers. Variable AGE indicates a more balanced distribution, with most values between 60 and 80, but also a few outliers with younger patients. For the variables HR_0 and HR_1, visualizations show a normal distribution, with values around the mean, but still some outliers.

Among the categorical features, INSURANCE is the most relevant for grouping patients. We can see in the distribution that the majority are covered by Medicare (around 79%), followed by Private insurance (around 16%), and a small portion with Medicaid (around 5%). As Medicare is a U.S. federal health insurance program primarily for people aged 65 and older, this distribution reflects a predominantly elderly population, consistent with the age statistics. From the MARRITAL STATUS distribution, we conclude that most patients are married or widowed, again reflecting an older population. In terms of ETHNICITY, the majority are white (around 81%), with smaller groups of black, hispanic, and asian patients.

For time-series variables, we can conclude that Average Insulin regular has its peak on day 1 on ICU; Average Blood cultured decrease over the days spent in ICU; and Average Transthoracic Echo also peaks on the first day in ICU.

4.1.2 Code

[ ]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Load your preprocessed LOS file and admissions data
df_LOS = pd.read_csv('df_LOS_final.csv')  # Your preprocessed file
admissions = pd.read_csv('ADMISSIONS (1).csv')  # From MIMIC-III

# Merge with admissions to get mortality data
```

```python
df_merged = pd.merge(df_LOS, admissions[['SUBJECT_ID', 'HADM_ID', 'DEATHTIME']],
                     on=['SUBJECT_ID', 'HADM_ID'], how='left')

# Create mortality flag
df_merged['MORTALITY'] = df_merged['DEATHTIME'].notna().astype(int)

# Display merged columns
print("Available columns after merge:\n", df_merged.columns.tolist())
df_merged.head(2)
```

```
Available columns after merge:
 ['ICUSTAY_ID', 'HADM_ID', 'SUBJECT_ID', 'ADMITTIME', 'INTIME', 'LOS',
'INSURANCE', 'MARITAL_STATUS', 'ETHNICITY', 'ADMISSION_LOCATION',
'ADMISSION_TYPE', 'GENDER', 'AGE', 'TIME_BETW_ADMISSION_ICU', '42731_DIAGNOSIS',
'41401_DIAGNOSIS', '4019_DIAGNOSIS', '2724_DIAGNOSIS', '5849_DIAGNOSIS',
'25000_DIAGNOSIS', '40390_DIAGNOSIS', '51881_DIAGNOSIS', 'V5861_DIAGNOSIS',
'5859_DIAGNOSIS', 'NUMBER_ICUSTAY', 'HR_0', 'HR_1', 'HR_2', 'HR_3', 'ABPs_0',
'ABPs_1', 'ABPs_2', 'ABPs_3', 'ABPm_0', 'ABPm_1', 'ABPm_2', 'ABPm_3', 'CVP_0',
'CVP_1', 'CVP_2', 'CVP_3', 'NBPm_0', 'NBPm_1', 'NBPm_2', 'NBPm_3', 'Dextrose
5%_0', 'Dextrose 5%_1', 'Dextrose 5%_2', 'Dextrose 5%_3', 'Midazolam
(Versed)_0', 'Midazolam (Versed)_1', 'Midazolam (Versed)_2', 'Midazolam
(Versed)_3', 'Phenylephrine_0', 'Phenylephrine_1', 'Phenylephrine_2',
'Phenylephrine_3', 'Furosemide (Lasix)_0', 'Furosemide (Lasix)_1', 'Furosemide
(Lasix)_2', 'Furosemide (Lasix)_3', 'Norepinephrine_0', 'Norepinephrine_1',
'Norepinephrine_2', 'Norepinephrine_3', 'Propofol_0', 'Propofol_1',
'Propofol_2', 'Propofol_3', 'Insulin - Regular_0', 'Insulin - Regular_1',
'Insulin - Regular_2', 'Insulin - Regular_3', 'NaCl 0.9%_0', 'NaCl 0.9%_1',
'NaCl 0.9%_2', 'NaCl 0.9%_3', 'Potassium Chloride - KCL_0', 'Potassium Chloride
- KCL_1', 'Potassium Chloride - KCL_2', 'Potassium Chloride - KCL_3',
'Vancomycin_0', 'Vancomycin_1', 'Vancomycin_2', 'Vancomycin_3', 'Multi Lumen_0',
'Multi Lumen_1', 'Multi Lumen_2', 'Multi Lumen_3', '22 Gauge_0', '22 Gauge_1',
'22 Gauge_2', '22 Gauge_3', '20 Gauge_0', '20 Gauge_1', '20 Gauge_2', '20
Gauge_3', '18 Gauge_0', '18 Gauge_1', '18 Gauge_2', '18 Gauge_3', 'Blood
Cultured_0', 'Blood Cultured_1', 'Blood Cultured_2', 'Blood Cultured_3',
'EKG_0', 'EKG_1', 'EKG_2', 'EKG_3', 'Transthoracic Echo_0', 'Transthoracic
Echo_1', 'Transthoracic Echo_2', 'Transthoracic Echo_3', 'Chest X-Ray_0', 'Chest
X-Ray_1', 'Chest X-Ray_2', 'Chest X-Ray_3', 'Arterial Line_0', 'Arterial
Line_1', 'Arterial Line_2', 'Arterial Line_3', 'Invasive Ventilation_0',
'Invasive Ventilation_1', 'Invasive Ventilation_2', 'Invasive Ventilation_3',
'DEATHTIME', 'MORTALITY']
```

[ ]:

```
   ICUSTAY_ID  HADM_ID  SUBJECT_ID            ADMITTIME               INTIME  \
0      200001   152234       55973  2181-11-18 16:04:00  2181-11-25 19:06:12
1      200011   121562       93535  2188-08-05 20:27:00  2188-08-06 01:39:24

      LOS INSURANCE MARITAL_STATUS ETHNICITY         ADMISSION_LOCATION  …  \
0  3.0786  Medicare         SINGLE     ASIAN  CLINIC REFERRAL/PREMATURE  …
```

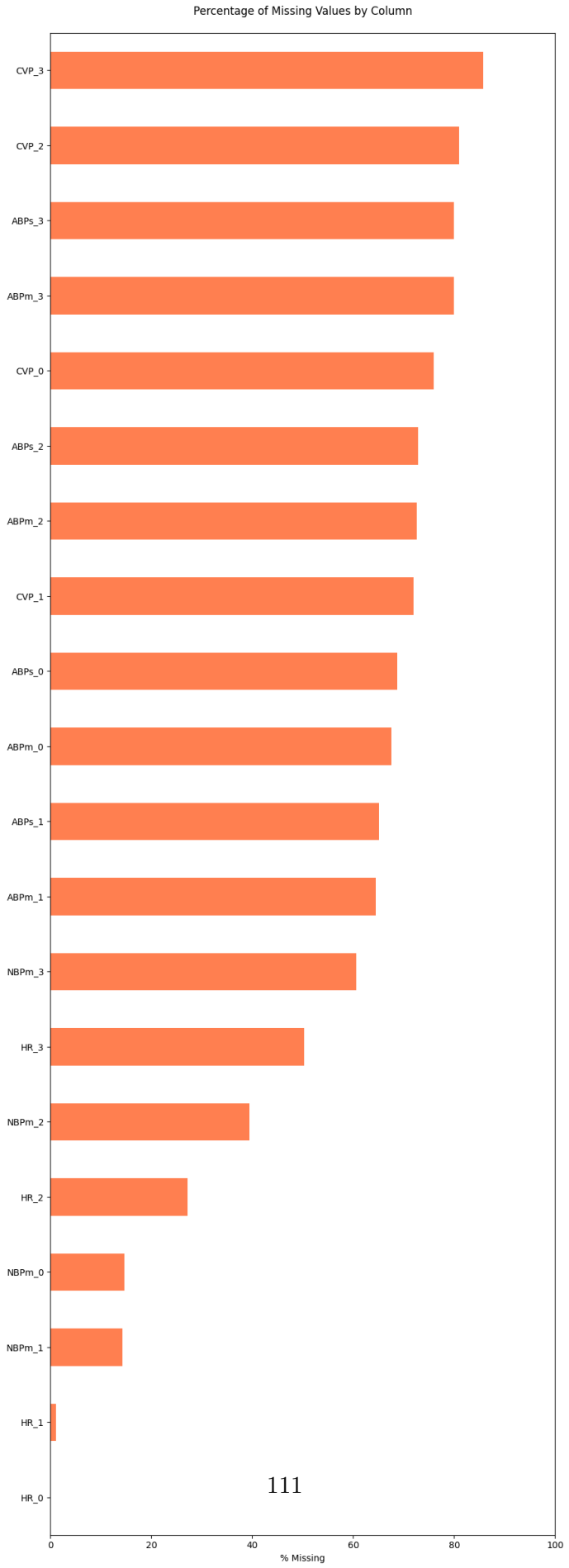
```
1  1.6330  Medicare         SINGLE     WHITE  CLINIC REFERRAL/PREMATURE  …

  Arterial Line_0 Arterial Line_1  Arterial Line_2  Arterial Line_3  \
0             0.0          2710.0              0.0              0.0
1             0.0             0.0              0.0              0.0

   Invasive Ventilation_0  Invasive Ventilation_1  Invasive Ventilation_2  \
0                     0.0                     0.0                     0.0
1                     0.0                     0.0                     0.0

   Invasive Ventilation_3  DEATHTIME  MORTALITY
0                     0.0        NaN          0
1                     0.0        NaN          0

[2 rows x 127 columns]
```

[ ]:
```
# 1. SETUP CELL (run this first)
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

def comprehensive_summary(df, max_cat=20, max_ts=5):
    """Generate complete summary statistics for all variables"""
    # First remove ID columns from analysis
    df = df.drop(columns=['ICUSTAY_ID', 'SUBJECT_ID', 'HADM_ID'],␣
 ↪errors='ignore')

    # Helper function to detect time-series columns
    def is_ts_col(col):
        return any(col.endswith(f'_{i}') for i in range(10))  # Checks for _0␣
 ↪to _9 suffixes

    print(" "*80)
    print(f"  DATA SHAPE: {df.shape[0]} rows × {df.shape[1]} columns")
    print(" "*80)

    # 1. Data Type Summary
    print("\n  DATA TYPES:")
    print(df.dtypes.value_counts().to_string())

    # 2. Missing Values Analysis
    missing = df.isnull().sum()
    missing_pct = (missing/len(df)*100).round(1)
    missing_df = pd.concat([missing, missing_pct], axis=1,
                           keys=['Missing Count', '% Missing'])
```

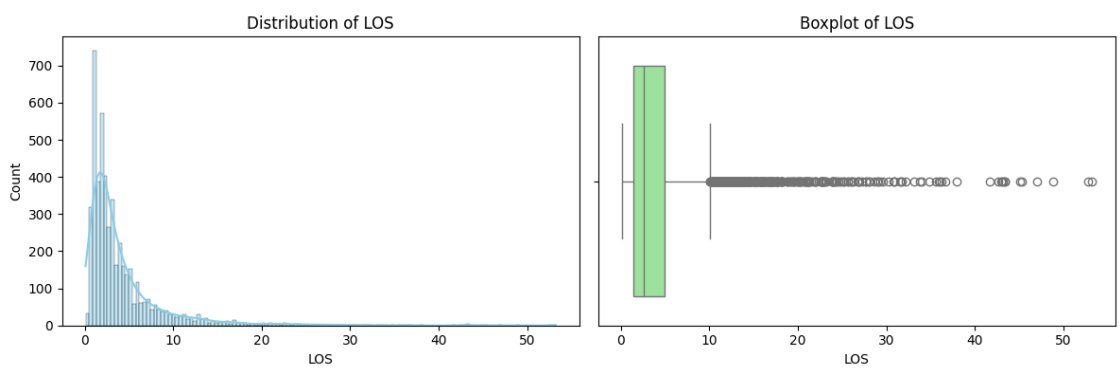
```
    plt.figure(figsize=(10, max(5, len(df.columns)//4)))
    missing_df[missing_df['Missing Count']>0].sort_values('% Missing')['%␣
↪Missing']\
        .plot.barh(color='coral')
    plt.title('Percentage of Missing Values by Column', pad=20)
    plt.xlabel('% Missing')
    plt.xlim(0, 100)
    plt.show()

    # 3. Numeric Variables Analysis (excluding IDs)
    numeric_cols = [col for col in df.select_dtypes(include=np.number).columns
                    if col not in ['ICUSTAY_ID', 'SUBJECT_ID', 'HADM_ID']]

    if numeric_cols:
        print(f"\n  NUMERIC VARIABLES ({len(numeric_cols)}):")
        display(df[numeric_cols].describe().transpose())

        # Plot top 6 non-ID numeric variables
        for col in numeric_cols[:6]:
            plt.figure(figsize=(12, 4))
            plt.subplot(1, 2, 1)
            sns.histplot(df[col].dropna(), kde=True, color='skyblue')
            plt.title(f'Distribution of {col}')

            plt.subplot(1, 2, 2)
            sns.boxplot(x=df[col].dropna(), color='lightgreen')
            plt.title(f'Boxplot of {col}')
            plt.tight_layout()
            plt.show()

    # 4. Categorical Variables Analysis (excluding IDs)
    cat_cols = [col for col in df.select_dtypes(include=['object', 'category']).
↪columns
                if col not in ['ICUSTAY_ID', 'SUBJECT_ID', 'HADM_ID']]

    if cat_cols:
        print(f"\n  CATEGORICAL VARIABLES ({len(cat_cols)}):")
        for col in cat_cols[:5]:  # Limit to 5 categories
            print(f"\n  {col} (nunique={df[col].nunique()})")
            if df[col].nunique() <= max_cat:
                display(df[col].value_counts(dropna=False).to_frame('Count'))

                plt.figure(figsize=(10, 4))
                df[col].value_counts(dropna=False).plot.bar(color='orchid')
                plt.title(f'Distribution of {col}')
                plt.xticks(rotation=45)
```

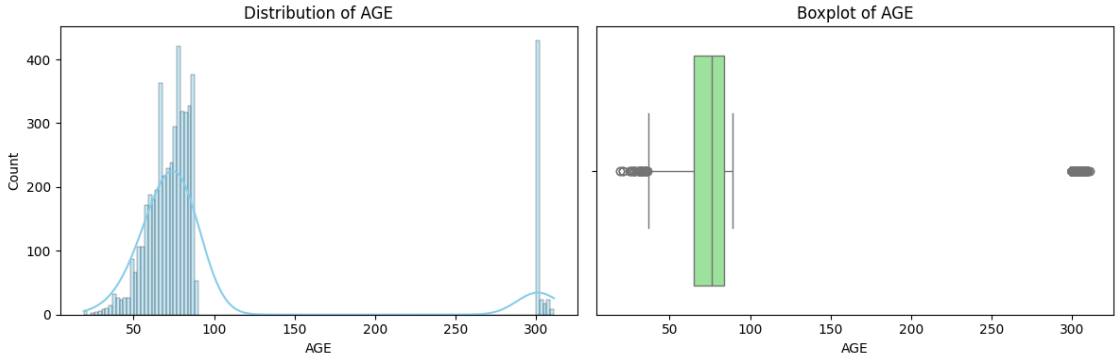
```
                plt.show()

    # 5. Time-Series Variables (excluding IDs)
    ts_cols = [col for col in df.columns if is_ts_col(col)]
    if ts_cols:
        ts_prefixes = list(set(col.rsplit('_',1)[0] for col in ts_cols))
        print(f"\n  TIME-SERIES VARIABLES ({len(ts_prefixes)} prefixes):")

        for prefix in ts_prefixes[:max_ts]:
            prefix_cols = sorted([col for col in ts_cols if col.
 ↪startswith(prefix)],
                                 key=lambda x: int(x.rsplit('_',1)[1]))

            plt.figure(figsize=(10, 4))
            df[prefix_cols].mean().plot(marker='o', color='teal')
            plt.title(f'Average {prefix} Over Time')
            plt.ylabel(prefix)
            plt.xlabel('Day in ICU')
            plt.grid(True)
            plt.show()

# 2. RUN ANALYSIS CELL (run this after the setup)
comprehensive_summary(df_LOS, max_cat=15, max_ts=3)
```

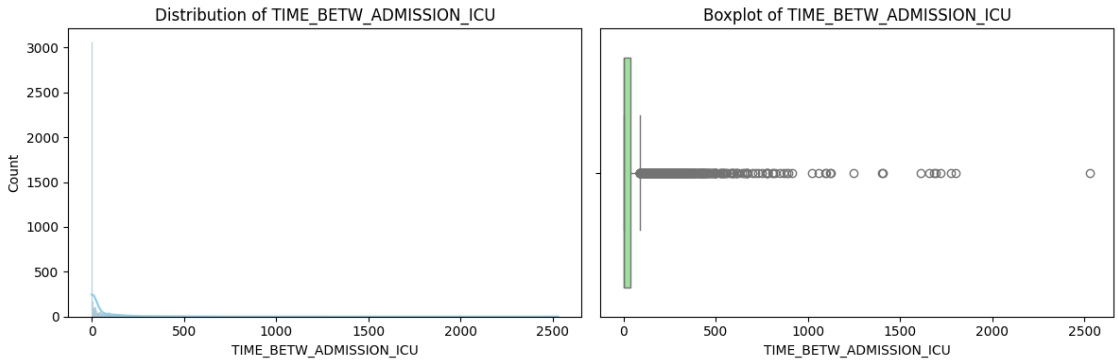
```
 DATA SHAPE: 4950 rows × 122 columns


 DATA TYPES:
float64    86
bool       26
object      8
int64       2
```

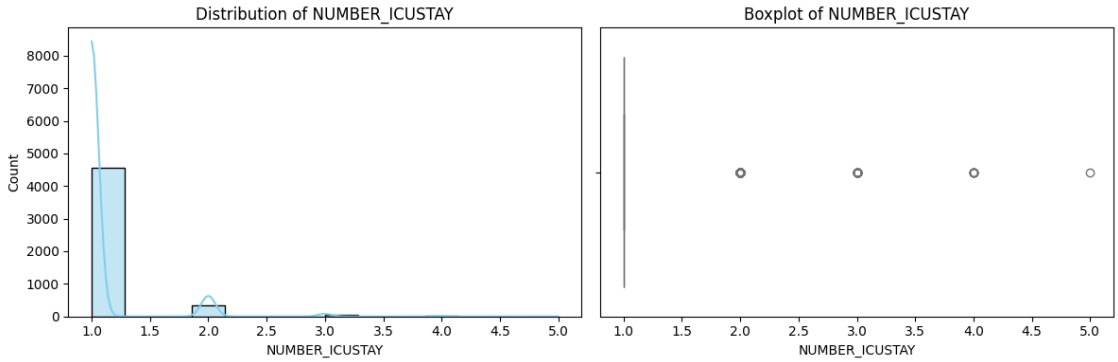

Percentage of Missing Values by Column
CVP_3
CVP_2
ABPs_3
ABPm_3
CVP_0
ABPs_2
ABPm_2
CVP_1
ABPs_0
ABPm_0
ABPs_1
ABPm_1
NBPm_3
HR_3
NBPm_2
HR_2
NBPm_0
NBPm_1
HR_1
HR_0
0
20
40
60
80
100
% Missing

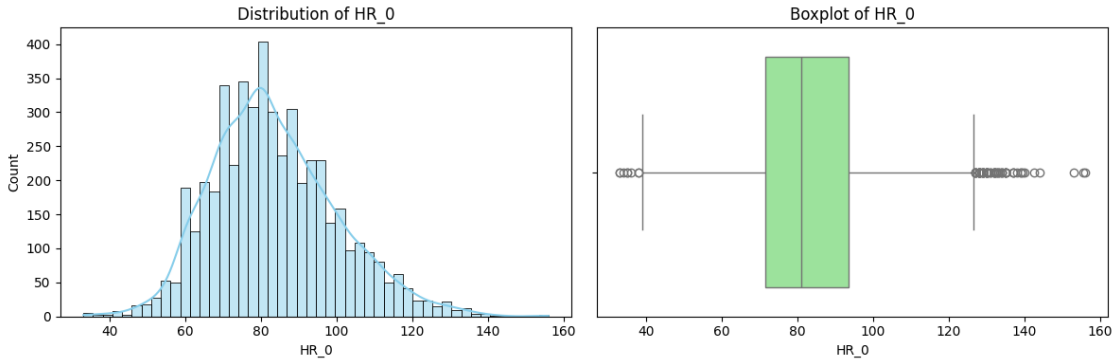
```
 NUMERIC VARIABLES (88):

                          count          mean            std      min  \
LOS                      4950.0      4.213046       4.974814   0.0892
AGE                      4950.0     94.803030      70.506758  19.0000
TIME_BETW_ADMISSION_ICU  4950.0     48.537657     132.671424   0.0000
NUMBER_ICUSTAY           4950.0      1.092121       0.337581   1.0000
HR_0                     4945.0     83.472396      16.600194  33.0000
…                         …           …             …       …
Arterial Line_3          4950.0   2339.399798  117568.432434   0.0000
Invasive Ventilation_0   4950.0  11033.824447  269571.672771   0.0000
Invasive Ventilation_1   4950.0   7092.060202  271180.898191   0.0000
Invasive Ventilation_2   4950.0   1645.073333   96442.851246   0.0000
Invasive Ventilation_3   4950.0    100.841818    2002.265759   0.0000

                                25%      50%       75%           max
LOS                        1.434725   2.5625    4.9055  5.325210e+01
AGE                       65.000000  76.0000   84.0000  3.110000e+02
TIME_BETW_ADMISSION_ICU    0.000000   0.0000   35.3750  2.530500e+03
NUMBER_ICUSTAY             1.000000   1.0000    1.0000  5.000000e+00
HR_0                      71.500000  81.0000   93.5000  1.560000e+02
…                              …       …         …            …
Arterial Line_3            0.000000   0.0000    0.0000  7.464960e+06
Invasive Ventilation_0     0.000000   0.0000  580.0000  1.037552e+07
Invasive Ventilation_1     0.000000   0.0000    0.0000  1.607328e+07
Invasive Ventilation_2     0.000000   0.0000    0.0000  6.739200e+06
Invasive Ventilation_3     0.000000   0.0000    0.0000  1.229090e+05

[88 rows x 8 columns]
```

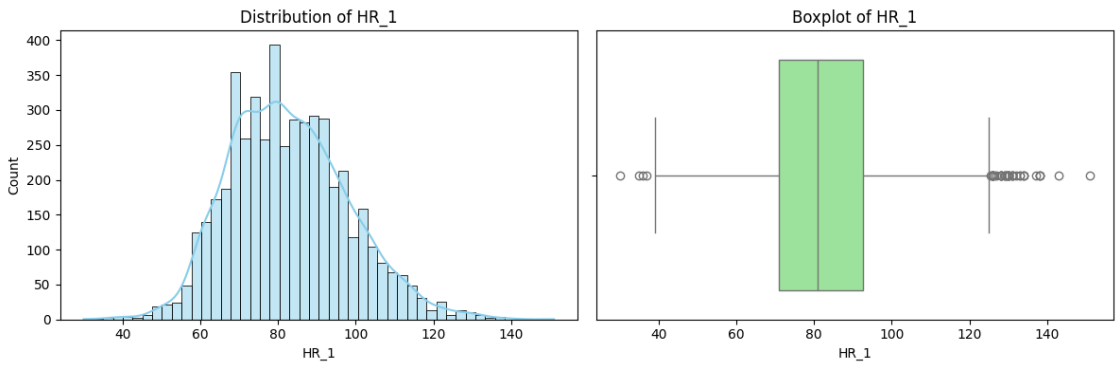



Distribution of AGE
Count
AGE
Boxplot of AGE
AGE

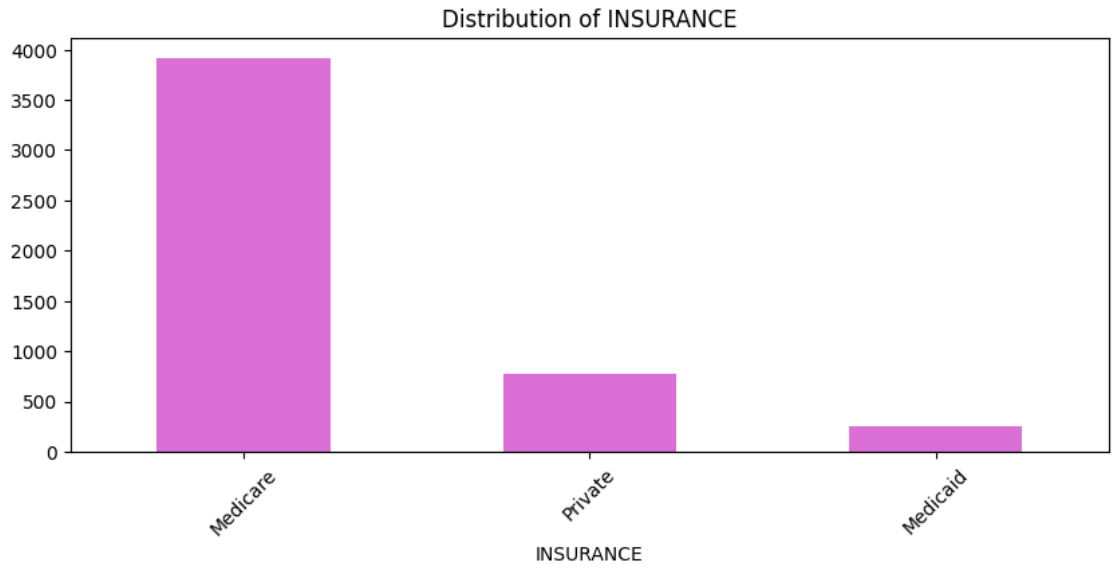

Distribution of TIME_BETW_ADMISSION_ICU
Count
TIME_BETW_ADMISSION_ICU
Boxplot of TIME_BETW_ADMISSION_ICU
TIME_BETW_ADMISSION_ICU


Distribution of NUMBER_ICUSTAY
Count
NUMBER_ICUSTAY
Boxplot of NUMBER_ICUSTAY
NUMBER_ICUSTAY

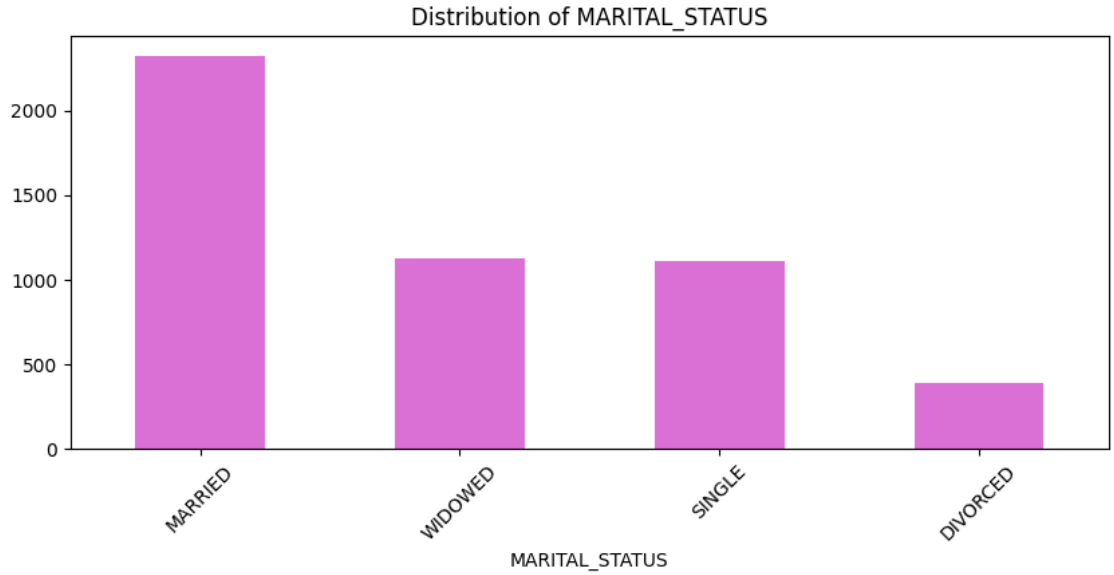




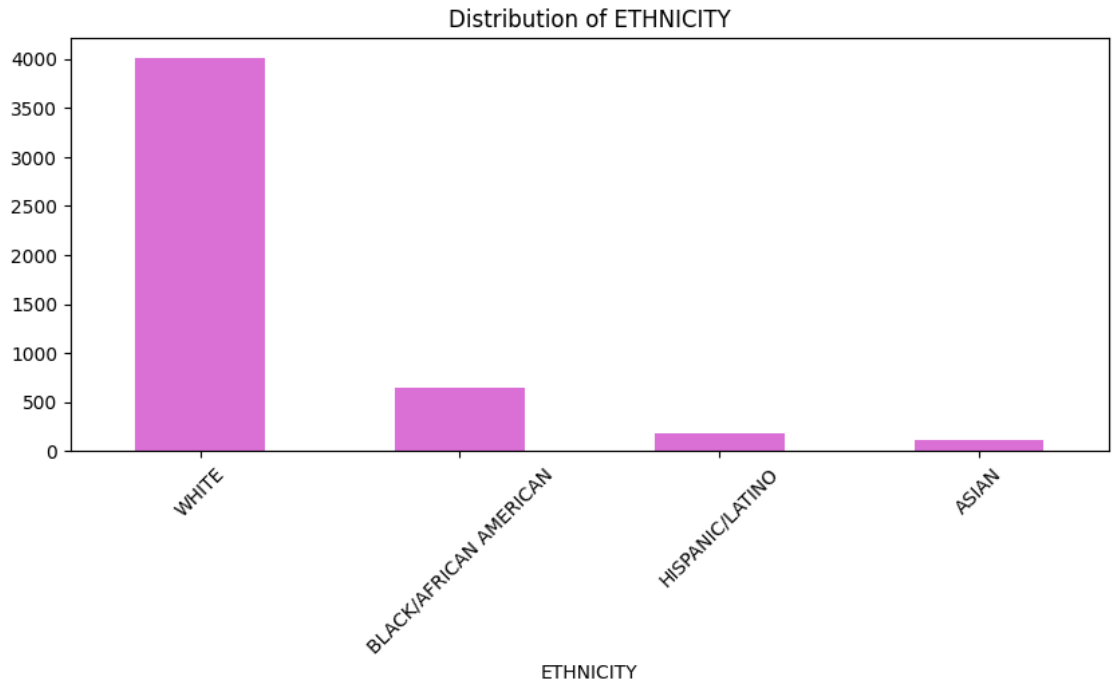

```
CATEGORICAL VARIABLES (8):

ADMITTIME (nunique=4582)

INTIME (nunique=4950)

INSURANCE (nunique=3)

           Count
INSURANCE
Medicare    3920
Private      777
Medicaid     253
```



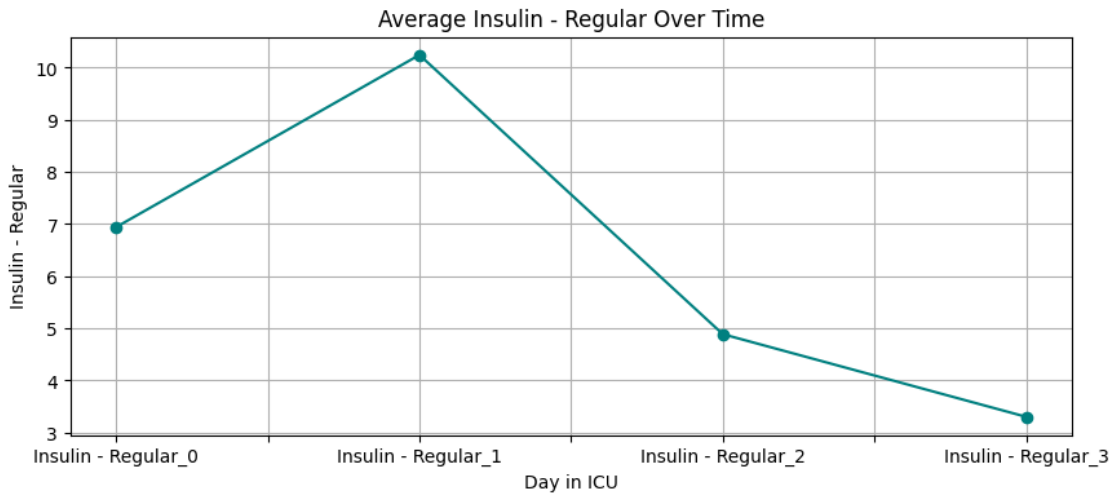

```
 MARITAL_STATUS (nunique=4)

                Count
MARITAL_STATUS
MARRIED          2322
WIDOWED          1128
SINGLE           1107
DIVORCED          393
```

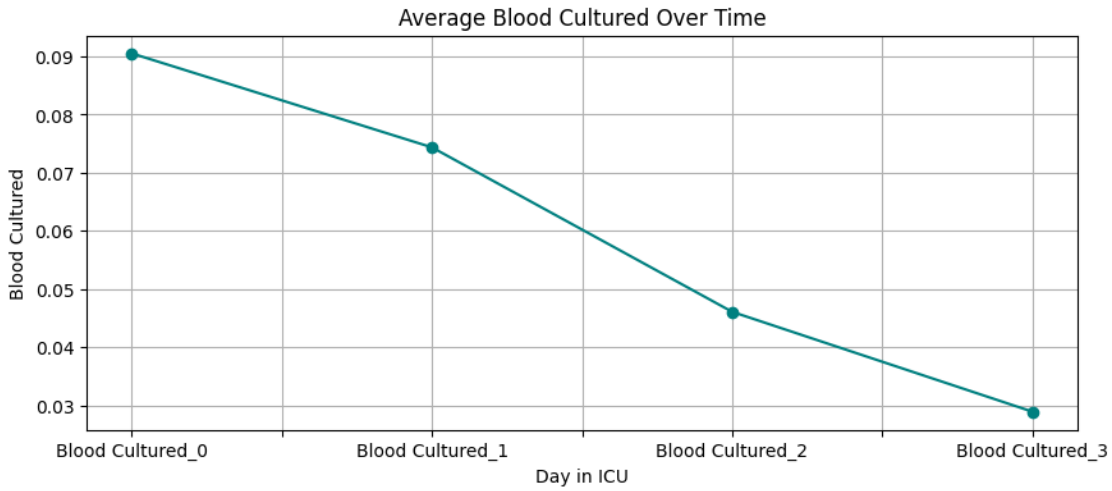


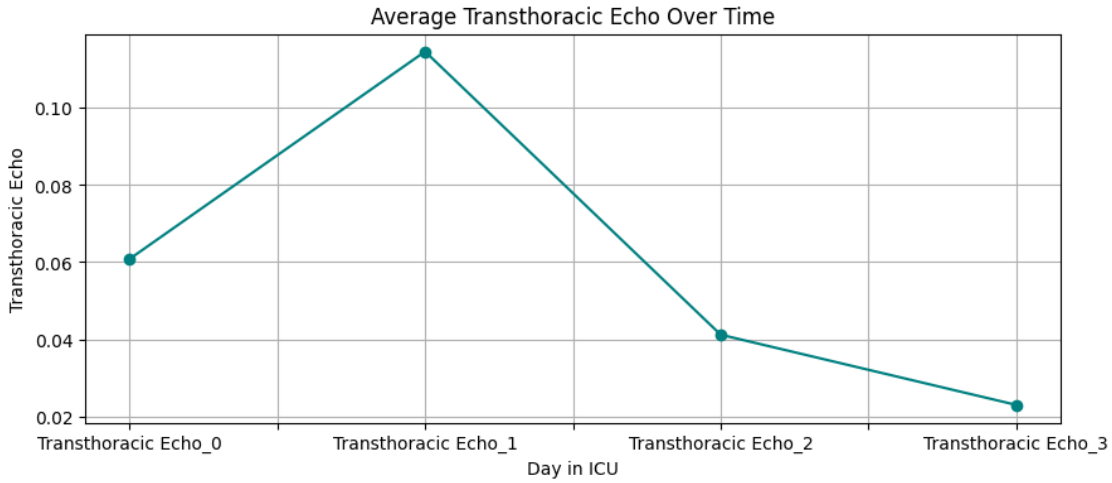
```
ETHNICITY (nunique=4)

                        Count
ETHNICITY
WHITE                    4013
BLACK/AFRICAN AMERICAN    641
HISPANIC/LATINO           181
ASIAN                     115
```




```
TIME-SERIES VARIABLES (25 prefixes):
```


Average Insulin - Regular Over Time
Insulin - Regular
10
9
8
7
6
5
4
3
Insulin - Regular_0
Insulin - Regular_1
Insulin - Regular_2
Insulin - Regular_3
Day in ICU


Average Blood Cultured Over Time
Blood Cultured
0.09
0.08
0.07
0.06
0.05
0.04
0.03
Blood Cultured_0
Blood Cultured_1
Blood Cultured_2
Blood Cultured_3
Day in ICU


Average Transthoracic Echo Over Time
Transthoracic Echo
0.10
0.08
0.06
0.04
0.02
Transthoracic Echo_0
Transthoracic Echo_1
Transthoracic Echo_2
Transthoracic Echo_3
Day in ICU

4.2 Key Variable Deep Dives

4.2.1 Textual Summary

To gain deeper insights into the most important features, we now did a deep dive into the areas demographics, LOS, Clinical Variables, and Mortality.

For demographics, we first had to correct unrealistic age values (above 200). As mentioned before, the Mimic III database assigns all patients older than 89 the age 300 for privacy reasons. According to the Mimic III Data Dictionary, the median age of this group is 91, wherefore we replaced all values above 200 with 91. We then visualized the age distribution and age by gender.

The corrected age histogram shows that most ICU patients are between 70 and 90 years old, with a mean of 73.4 and a median of 76. This supports the expectation that ICU admissions due to Congestive Heart Failure are more common among seniors.

The boxplot shows similar age distributions for males and females, with no major difference in median or spread between the two groups.

After that, we analyzed the distribution of LOS and how it differs between survivors and non-survivors. A histogram shows the overall distribution of LOS values, while a boxplot compares LOS across mortality status. We can observe that the LOS distribution is right-skewed, with most patients staying a few days and a few staying significantly longer. The average LOS is 4.2 days. The boxplot suggests that patients who died tend to have slightly longer stays, although there is a considerable overlap. The overall mortality rate in our cohort is 12.6%.

Moving on, we analyzed the clinical variables. We decided to focus on the patients' heart rate at the day of ICU admission (HR_0) and Vancomycin Dose.

The initial heart rate (HR_0) and Vancomycin dose are both clinically relevant variables when predicting outcomes such as length of stay and mortality in patients with congestive heart failure (CHF). A patient's heart rate on the day of ICU admission provides an immediate indication of cardiovascular stability. In the context of CHF, an elevated or irregular heart rate often reflects acute decompensation or underlying arrhythmias, such as atrial fibrillation, which are associated with worse clinical trajectories. These complications typically require intensive management and prolonged monitoring, contributing to longer hospital stays. Moreover, persistent tachycardia or arrhythmias are also linked to higher in-hospital mortality, particularly in patients whose cardiac function is already compromised.

The administration and dosage of Vancomycin, a powerful antibiotic, further signal the presence of a serious infection, such as pneumonia or sepsis—both of which are common and dangerous in CHF patients. Infections in this population tend to be harder to manage due to impaired perfusion and often reduced kidney function, which can complicate dosing and drug clearance. As a result, Vancomycin use may be associated with both increased treatment complexity and longer recovery times, thereby extending the length of stay. Additionally, infections like sepsis significantly elevate the risk of mortality in CHF patients, especially when compounded by renal dysfunction or multi-organ failure.

In our analysis, we could observe that the heart rate has a distribution close to normal with a mean

of 83.5 bpm and a median of 81 bpm. The distribution of Vancomycin dosage shows that most patients receive a rather low dosage, with a few outliers on the high dosage side.

Next, we explored their relationship to mortality. The boxplot showed slightly higher heart rates among patients who died, suggesting a possible link between elevated heart rate and poor outcomes.

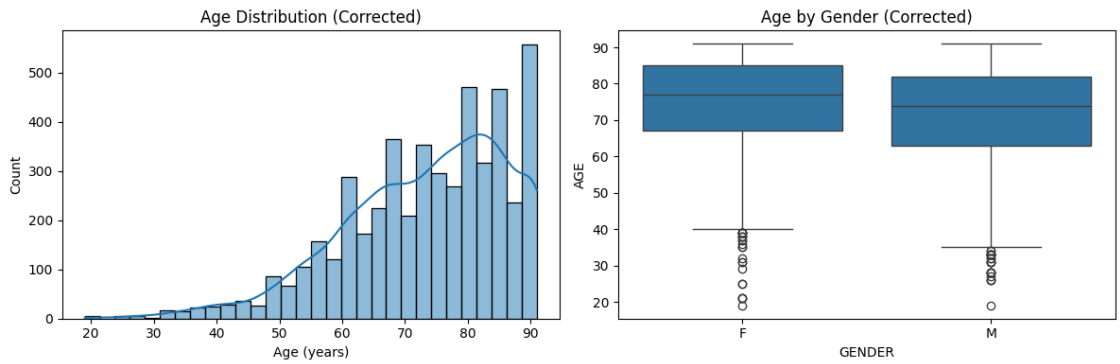
Lastly, we plotted the development of the avergae heart rate over the first three days of the ICU stay. The goal was to visualize how the heart rate changes over the stay, which can help identify trends or stability in patient condition over time. The plot shows that the heart rate is relatively stable with only minor differences in the average across the days.

The next step was to analyze mortality: To better understand mortality patterns, we grouped patients into age ranges (< 40, 40–60, 60–80, and 80+) and calculated the mortality rate for each group. The barplot shows how mortality increases with age, especially in the 80+ group, which reflects the higher vulnerability of elderly ICU patients. We also compared mortality by gender. The mortality rate was slightly higher for females (13.0%) compared to males (12.3%), but the difference is small and may not be clinically significant without further statistical testing.

4.2.2 Code

Demographic Analysis

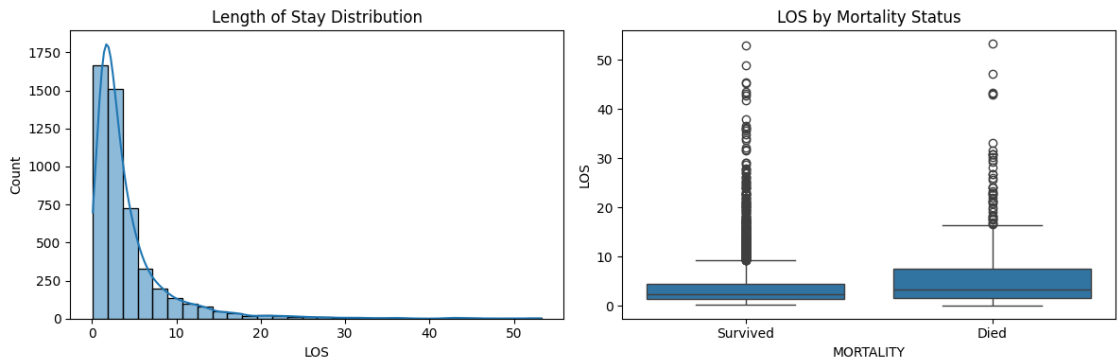
[ ]:
```
# Age analysis - with correction for ages >200
df_merged['AGE'] = np.where(df_merged['AGE'] > 200, 91, df_merged['AGE'])

plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
sns.histplot(df_merged['AGE'], bins=30, kde=True)
plt.title('Age Distribution (Corrected)')
plt.xlabel('Age (years)')

plt.subplot(1, 2, 2)
sns.boxplot(x='GENDER', y='AGE', data=df_merged)
plt.title('Age by Gender (Corrected)')
plt.tight_layout()
plt.show()

# Age statistics
print("\nAge Statistics (Corrected):")
print(f"Mean: {df_merged['AGE'].mean():.1f} years")
print(f"Median: {df_merged['AGE'].median():.1f} years")
print(f"Std Dev: {df_merged['AGE'].std():.1f} years")
```




```
Age Statistics (Corrected):
Mean: 73.4 years
Median: 76.0 years
Std Dev: 13.1 years
```

LOS Distribution

[ ]:
```python
# LOS distribution
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
sns.histplot(df_merged['LOS'], bins=30, kde=True)
plt.title('Length of Stay Distribution')

plt.subplot(1, 2, 2)
sns.boxplot(x='MORTALITY', y='LOS', data=df_merged)
plt.title('LOS by Mortality Status')
plt.xticks([0, 1], ['Survived', 'Died'])
plt.tight_layout()
plt.show()

# LOS statistics
print("\nLOS Statistics:")
print(f"Mean: {df_merged['LOS'].mean():.1f} days")
print(f"Median: {df_merged['LOS'].median():.1f} days")
print(f"Std Dev: {df_merged['LOS'].std():.1f} days")
print(f"Mortality Rate: {df_merged['MORTALITY'].mean()*100:.1f}%")
```

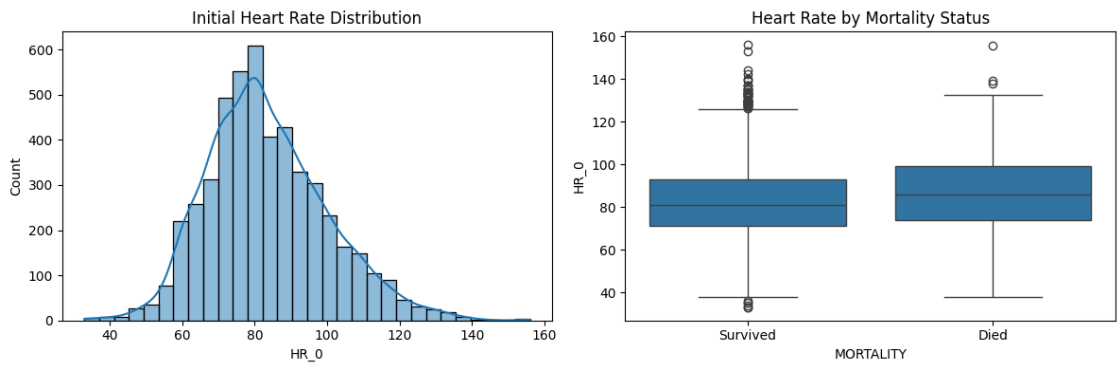



```
LOS Statistics:
Mean: 4.2 days
Median: 2.6 days
Std Dev: 5.0 days
Mortality Rate: 12.6%
```

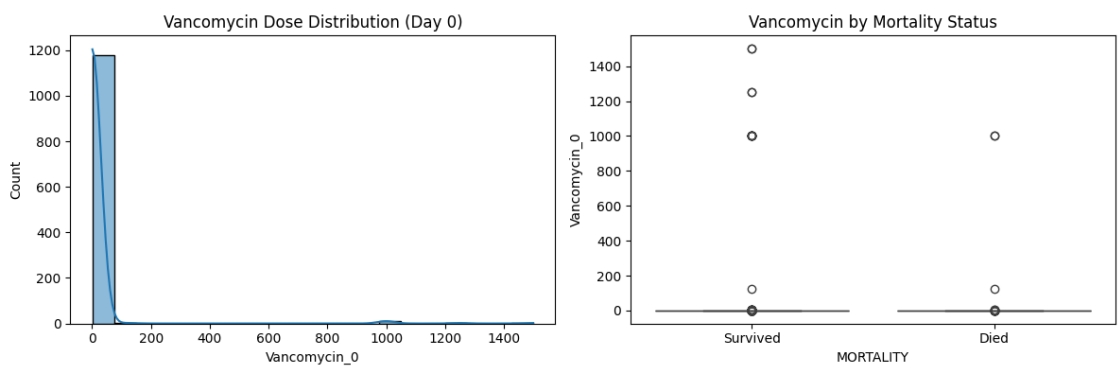
Clinical Variables Analysis

[ ]:
```
# Heart Rate analysis (assuming HR_0 is initial heart rate)
if 'HR_0' in df_merged.columns:
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    sns.histplot(df_merged['HR_0'].dropna(), bins=30, kde=True)
    plt.title('Initial Heart Rate Distribution')

    plt.subplot(1, 2, 2)
    sns.boxplot(x='MORTALITY', y='HR_0', data=df_merged)
    plt.title('Heart Rate by Mortality Status')
    plt.xticks([0, 1], ['Survived', 'Died'])
    plt.tight_layout()
    plt.show()

    print("\nHeart Rate Statistics:")
    print(f"Mean: {df_merged['HR_0'].mean():.1f} bpm")
    print(f"Median: {df_merged['HR_0'].median():.1f} bpm")

# Common medications analysis (example for Vancomycin)
if 'Vancomycin_0' in df_merged.columns:
    plt.figure(figsize=(12, 4))
    plt.subplot(1, 2, 1)
    sns.histplot(df_merged[df_merged['Vancomycin_0']>0]['Vancomycin_0'],␣
 ↪bins=20, kde=True)
    plt.title('Vancomycin Dose Distribution (Day 0)')
```

```
    plt.subplot(1, 2, 2)
    sns.boxplot(x='MORTALITY', y='Vancomycin_0',␣
↪data=df_merged[df_merged['Vancomycin_0']>0])
    plt.title('Vancomycin by Mortality Status')
    plt.xticks([0, 1], ['Survived', 'Died'])
    plt.tight_layout()
    plt.show()
```

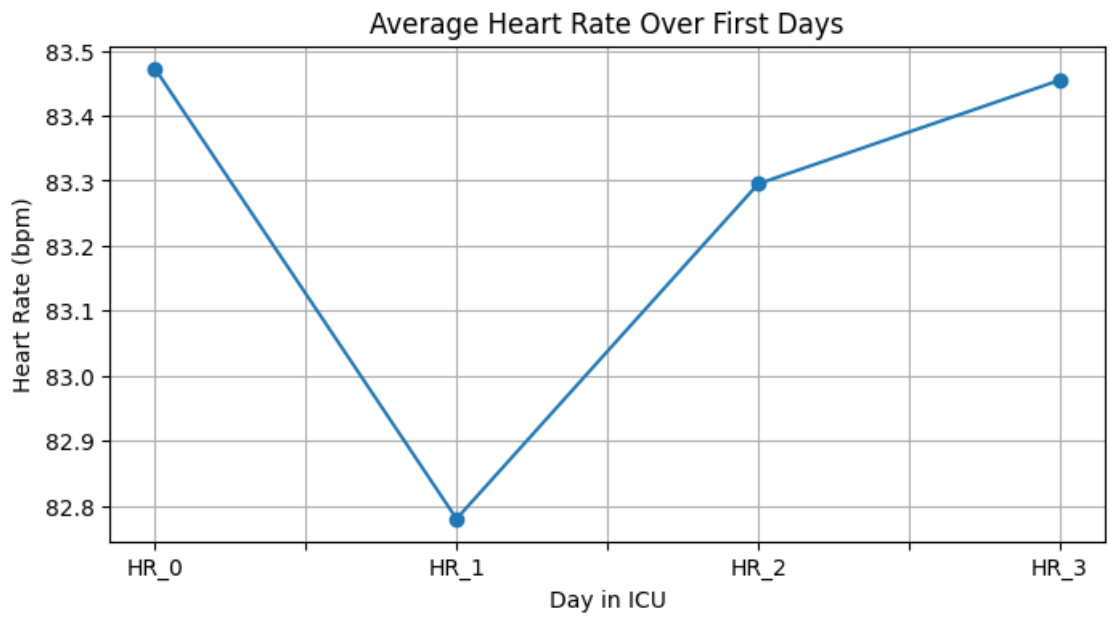



```
Heart Rate Statistics:
Mean: 83.5 bpm
Median: 81.0 bpm
```



[ ]:
```
# Plot mean heart rate over first 3 days (if available)
hr_cols = [col for col in df_merged.columns if col.startswith('HR_')]
if len(hr_cols) > 1:
    hr_means = df_merged[hr_cols].mean()
    plt.figure(figsize=(8, 4))
    hr_means.plot(marker='o')
```

```
    plt.title('Average Heart Rate Over First Days')
    plt.ylabel('Heart Rate (bpm)')
    plt.xlabel('Day in ICU')
    plt.grid(True)
    plt.show()
```

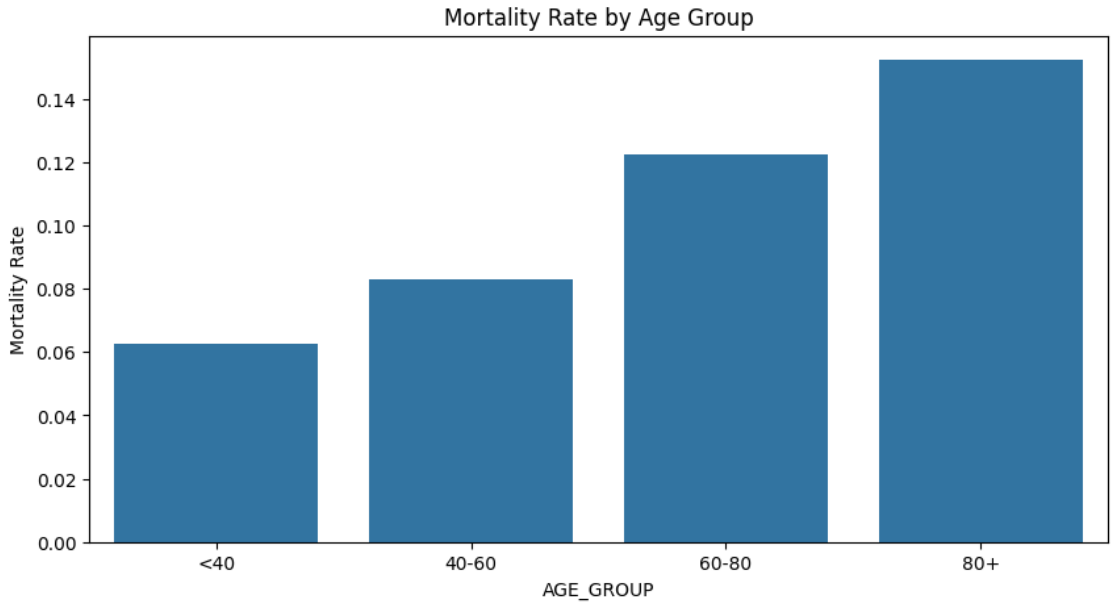


Mortality Analysis

[ ]:
```
# Mortality by age groups
df_merged['AGE_GROUP'] = pd.cut(df_merged['AGE'],
                                bins=[0, 40, 60, 80, 120],
                                labels=['<40', '40-60', '60-80', '80+'])

plt.figure(figsize=(10, 5))
mortality_rate = df_merged.groupby('AGE_GROUP')['MORTALITY'].mean().
 ↪reset_index()
sns.barplot(x='AGE_GROUP', y='MORTALITY', data=mortality_rate)
plt.title('Mortality Rate by Age Group')
plt.ylabel('Mortality Rate')
plt.show()

# Mortality by gender
gender_mortality = df_merged.groupby('GENDER')['MORTALITY'].mean().reset_index()
print("\nMortality by Gender:")
print(gender_mortality)
```

```
<ipython-input-32-91cc4c2e509d>:7: FutureWarning: The default of observed=False
```

```
is deprecated and will be changed to True in a future version of pandas. Pass
observed=False to retain current behavior or observed=True to adopt the future
default and silence this warning.
  mortality_rate =
df_merged.groupby('AGE_GROUP')['MORTALITY'].mean().reset_index()
```




```
Mortality by Gender:
  GENDER  MORTALITY
0      F   0.130074
1      M   0.123355
```

5 Model Building

We start the model building by splitting the data into training and test set. To prevent data leakage, we used GroupShuffleSplit to split the data by SUBJECT_ID, ensuring that the same patient does not appear in both the training and test sets.

After the split, the training set contains 3941 rows and the test set contains 1009 rows, which maintains an 80/20 split and avoids overfitting on repeated patient data.

Next we will identify categorical features using LabelEncoder to convert them into numeric values so that they can be used by the machine learning models. These encoded versions are appended to the dataset, and the original non-numeric columns are removed.

[ ]:
```
# Import all required libraries
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer, KNNImputer
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor,␣
↪HistGradientBoostingRegressor
from sklearn.linear_model import Ridge
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import GroupShuffleSplit, GridSearchCV, GroupKFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_validate, GroupKFold
import shap
%matplotlib inline
```

[ ]:
```
df=pd.read_csv("df_LOS_final.csv")
print(f"Dataset shape: {df.shape}")
print("\nFirst 3 rows:")
display(df.head(3))

# Check for missing values (if any remain after preprocessing)
print("\nMissing values per column:")
print(df.isnull().sum())
```

```
Dataset shape: (4950, 125)

First 3 rows:

   ICUSTAY_ID  HADM_ID  SUBJECT_ID            ADMITTIME               INTIME  \
0      200001   152234       55973  2181-11-18 16:04:00  2181-11-25 19:06:12
1      200011   121562       93535  2188-08-05 20:27:00  2188-08-06 01:39:24
2      200024   179633       76603  2127-02-21 16:34:00  2127-03-03 16:09:07

      LOS INSURANCE MARITAL_STATUS                ETHNICITY  \
0  3.0786  Medicare         SINGLE                    ASIAN
1  1.6330  Medicare         SINGLE                    WHITE
2  0.3812  Medicare        MARRIED  BLACK/AFRICAN AMERICAN

          ADMISSION_LOCATION  … Chest X-Ray_2 Chest X-Ray_3  \
0  CLINIC REFERRAL/PREMATURE  …          True         False
1  CLINIC REFERRAL/PREMATURE  …         False         False
2       EMERGENCY ROOM ADMIT  …         False         False

   Arterial Line_0  Arterial Line_1  Arterial Line_2  Arterial Line_3  \
0              0.0           2710.0              0.0              0.0
1              0.0              0.0              0.0              0.0
2            498.0              0.0              0.0              0.0
```

```
   Invasive Ventilation_0  Invasive Ventilation_1  Invasive Ventilation_2  \
0                     0.0                     0.0                     0.0
1                     0.0                     0.0                     0.0
2                   360.0                     0.0                     0.0

   Invasive Ventilation_3
0                     0.0
1                     0.0
2                     0.0

[3 rows x 125 columns]


Missing values per column:
ICUSTAY_ID                0
HADM_ID                   0
SUBJECT_ID                0
ADMITTIME                 0
INTIME                    0
                         ..
Arterial Line_3           0
Invasive Ventilation_0    0
Invasive Ventilation_1    0
Invasive Ventilation_2    0
Invasive Ventilation_3    0
Length: 125, dtype: int64
```

[ ]:
```
# Define X and y
X = df.drop(columns=["ICUSTAY_ID", "HADM_ID", "SUBJECT_ID", "ADMITTIME",␣
 ↪"INTIME", "LOS"])
y = df["LOS"]
```

[ ]:
```
# Split data by patient ID to avoid leakage
groups = df['SUBJECT_ID']  # Group by patient ID
gss = GroupShuffleSplit(test_size=0.2, random_state=42)
train_idx, test_idx = next(gss.split(X, y, groups))

X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

print(f"Train size: {len(X_train)}, Test size: {len(X_test)}")
```

```
Train size: 3941, Test size: 1009
```

[ ]:
```
non_numeric = X_train.select_dtypes(include=['object']).columns
print("Non-numeric columns to handle:", non_numeric.tolist())
```

```
from sklearn.preprocessing import LabelEncoder
for col in non_numeric:
    le = LabelEncoder()
    X_train[col + '_encoded'] = le.fit_transform(X_train[col])
    X_test[col + '_encoded'] = le.transform(X_test[col])
X_train_numeric = X_train.drop(non_numeric, axis=1)
X_test_numeric = X_test.drop(non_numeric, axis=1)
```

5.1 Initial Models

5.1.1 Textual Summary

Baseline Model:

To establish a baseline for model performance, we predicted the median Length of Stay from the training set for all test cases. This gives us a reference point to compare more complex models and tells us how well a simple, naive approach would perform.

The baseline model gives an MAE of 2.68, meaning that on average, predictions were about 2.7 days off. The RMSE of 5.05 reflects that some predictions had large errors. The R^2 score is -0.11, indicating that this naive model performs worse than simply predicting the mean — it explains none of the variance in the data. This confirms that there's room for significant improvement using actual predictive models.

Random Forest:

For the Random Forest, we will start by training a Random Forest Regressor with a pipeline that includes MICE (Iterative) Imputation to handle any missing values. This approach allows the model to learn from patterns in the data while intelligently filling in gaps during training. A sophisticated approach for imputing NAs is important considering that we have a lot of missing vital signs for the features crafted based on the chart events. We used GroupKFold cross-validation (5 folds), again grouped by SUBJECT_ID, to evaluate performance in a patient-independent way. After cross-validation, we also evaluated the model on the held-out test set to assess real-world generalization.

In cross-validation, the Random Forest model achieved a MAE of 1.46, RMSE of 3.37, and an R^2 of 0.55. This is a strong improvement over the baseline, since the baseline MAE was 2.68 and R^2 was -0.11, so 1.46 MAE and 0.55 R^2 is a solid improvement, indicating the model captures meaningful relationships in the data and explains over half of the variance in LOS. On the test set, performance improved further: MAE dropped to 1.27, RMSE to 2.65, and R^2 increased to 0.69. These results suggest that the model generalizes well and performs reliably when exposed to new, unseen patient data.

Gradient Boosting:

In this model, we combined Gradient Boosting Regression, a powerful ensemble learning method, with KNN Imputation. Gradient Boosting works by training a sequence of decision trees, where each new tree tries to improve the loss based on Gradient Descent. It is especially effective for capturing complex, non-linear relationships in data. KNN Imputation fills in missing values by looking at the "k" most similar samples (nearest neighbors) and using their values to estimate the missing ones. This helps retain more information compared to dropping rows or using mean

imputation. The two components are combined in a pipeline, allowing the imputation and model training to be done together during cross-validation and grid search. We used GridSearchCV with group-aware cross-validation to tune the key hyperparameters like number of estimators, learning rate, and tree depth. After selecting the best model, we evaluated it using cross-validation as well as the test set.

After tuning, the best Gradient Boosting model achieved a cross-validated MAE of 1.39, RMSE of 3.39, and an R^2 of 0.54 — very close to Random Forest, indicating consistent learning quality. On the test set, it reached a MAE of 1.26, RMSE of 2.73, and R^2 of 0.67, confirming strong generalization. These results show that Gradient Boosting is a reliable option for LOS prediction, though in this case, Random Forest performed slightly better on the test set.

We also tested HistGradientBoostingRegressor, which already supports missing values. This model eliminates the need for a separate imputation step, simplifying the pipeline. We evaluated it using 5-fold group-aware cross-validation and tested it on the test set.

We could conclude that this model performed slightly better than the previous Gradient Boosting variant. It achieved a cross-validated MAE of 1.44, RMSE of 3.35, and R^2 of 0.55. On the test set, it reached a MAE of 1.24, RMSE of 2.59, and a R^2 of 0.71 — the highest among all models tested so far. This suggests that removing the imputation step and using a model built to handle missing values directly may result in both simpler pipelines and better performance.

Neural Network:

For the Neural Network, we implemented a Multi-Layer Perceptron (MLP) using a pipeline that includes KNN Imputer for missing values, StandardScaler to normalize the features, and MLPRegressor as the prediction model.

Since neural networks are sensitive to scale and data quality, this preprocessing pipeline is essential. We used GridSearchCV with GroupKFold to tune the architecture (number of layers) and learning rate, while preserving patient independence during cross-validation.

We could observe that the cross-validation results were poor (MAE: 6.44, RMSE: ~80, R^2: -639), likely due to instability during training or sensitivity to hyperparameters and data distribution. These numbers suggest overfitting to some folds and underperformance on others, which neural networks are prone to on small or imbalanced tabular datasets. However, the neural network achieved reasonable test performance, with a MAE of 1.98, RMSE of 3.49, and R^2 of 0.47. This is lower than Random Forest and Gradient Boosting, suggesting that the MLP struggled to learn patterns as well on this tabular medical dataset. The fold-level error plot shows large variability in both MAE and RMSE across folds, confirming instability during training. The network may not have converged properly in some folds or was affected by noisy inputs. The learning curve shows a noticeable gap between training and validation RMSE, indicating some degree of overfitting. While error decreases as the training size grows, the validation RMSE flattens early, suggesting that the model may not benefit much from more learning in its current form or setup. These results suggest that neural networks may not be the most reliable choice for this problem without further tuning or regularization.

Regularized Linear Model:

Next we trained a Ridge Regression model, which is a linear regression technique with L2 regularization. While it cannot capture complex patterns like tree-based models, it is useful for benchmarking and for understanding linear feature importance. The model pipeline includes Iterative Imputation

to handle missing values and StandardScaler to normalize features — important for linear models to work correctly.

When analyzing the results, we could observe that this model performed poorly in cross-validation, with a MAE of 4.94, RMSE of 54.21, and an R^2 of -292.59, indicating that it failed to generalize well across folds. These numbers likely reflect the model's inability to capture the non-linear patterns in the data. However, on the test set, the results improved significantly: MAE of 1.97, RMSE of 3.49, and R^2 of 0.47.

When plotting the MAE and RMSE across folds, we can observe the same pattern as in the Neural Network. The error is relatively low for folds 1, 2, and 5 (RMSE between 3.85 and 8.45). In contrast, it is exceptionally high (182.43) for fold 3 and 4 (71.98). Considering that the Neural Network struggled with the exact same folds, this indicates that fold 3 and 4 include many outliers or cases that are harder to predict. This high variance makes the Linear Regression an unreliable model and the positive results of the Random Forest and Gradient Boosting even more remarkable.

Still, the Ridge model's coefficients give insight into which features had the strongest linear relationship with LOS. For example, ABPs_3 and NBPm_0 had the highest positive coefficients, suggesting that higher arterial or non-invasive pressure readings may be associated with longer ICU stays. Negative coefficients (e.g., 20 Gauge_3) suggest features that contribute to shorter LOS.

These values make sense clinically and statistically. High positive coefficients for ABPs_3 and NBPm_0 indicate that elevated arterial and non-invasive blood pressure measurements are associated with longer ICU stays, which aligns with the understanding that abnormal vital signs may reflect more severe or unstable conditions requiring prolonged monitoring. Conversely, the negative coefficient for 20 Gauge_3 suggests that the use of a 20-gauge catheter may be linked to shorter lengths of stay, possibly because it is used in less critical cases or facilitates quicker treatment delivery. Interestingly, demographic information are not in the top 10 features by absolute coefficient.

5.1.2 Code

Baseline Model

[ ]:
```
# Baseline with bootstrapped CI
baseline_pred = np.median(y_train)
baseline_metrics = {
    'MAE': mean_absolute_error(y_test, [baseline_pred]*len(y_test)),
    'RMSE': np.sqrt(mean_squared_error(y_test, [baseline_pred]*len(y_test))),
    'R2': r2_score(y_test, [baseline_pred]*len(y_test))
}
print(f"Baseline MAE: {baseline_metrics['MAE']:.2f}")
print(f"Baseline RMSE: {baseline_metrics['RMSE']:.2f}")
print(f"Baseline R²: {baseline_metrics['R2']:.2f}")
```

```
Baseline MAE: 2.68
Baseline RMSE: 5.05
Baseline R²: -0.11
```

Random Forest with MICE Imputation

[ ]:
```python
from sklearn.metrics import mean_squared_error
import numpy as np

# Preprocessing pipeline
rf_pipe = make_pipeline(
    IterativeImputer(),
    RandomForestRegressor(random_state=42)
)

# Cross-validated training
cv = GroupKFold(n_splits=5)
rf_scores = cross_validate(rf_pipe, X_train_numeric, y_train,
                           cv=cv, groups=groups[train_idx],
                           scoring=['neg_mean_absolute_error',␣
 ↪'neg_mean_squared_error', 'r2'],
                           n_jobs=-1)

# Calculate RMSE from MSE scores
rf_rmse = np.sqrt(-rf_scores['test_neg_mean_squared_error']).mean()

print(f"RF CV MAE: {-rf_scores['test_neg_mean_absolute_error'].mean():.2f}")
print(f"RF CV RMSE: {rf_rmse:.2f}")  # Corrected RMSE calculation
print(f"RF CV R²: {rf_scores['test_r2'].mean():.2f}")

# Final test evaluation
rf_pipe.fit(X_train_numeric, y_train)
y_pred_rf = rf_pipe.predict(X_test_numeric)

# Calculate test set metrics
test_mae = mean_absolute_error(y_test, y_pred_rf)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_rf))
test_r2 = r2_score(y_test, y_pred_rf)

print("\nTest Set Performance:")
print(f"MAE: {test_mae:.2f}")
print(f"RMSE: {test_rmse:.2f}")
print(f"R²: {test_r2:.2f}")
```

```
RF CV MAE: 1.46
RF CV RMSE: 3.37
RF CV R²: 0.55

Test Set Performance:
MAE: 1.27
RMSE: 2.65
R²: 0.69
```

Gradient Boosting

[ ]:
```python
from sklearn.model_selection import GroupKFold, cross_validate, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Cross-validated grid search
print("=== Cross-Validated Performance During Tuning ===")

# 1. Define the pipeline
gb_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),  # Handle missing values
    ('regressor', GradientBoostingRegressor(random_state=42))
])

# 2. Define hyperparameters to search
gb_params = {
    'regressor__n_estimators': [50, 100, 200],
    'regressor__max_depth': [3, 5, 7],
    'regressor__learning_rate': [0.01, 0.1, 0.2]
}

# 3. Proceed with GridSearchCV (ensure X_train_numeric, y_train, groups,␣
 ↪train_idx exist)
group_kfold = GroupKFold(n_splits=5).split(X_train_numeric, y_train,␣
 ↪groups=groups[train_idx])

gb_search = GridSearchCV(
    gb_pipe,
    gb_params,
    cv=group_kfold,
    scoring='neg_mean_absolute_error',
    refit=True,
    n_jobs=-1
)

# Fit without groups parameter
gb_search.fit(X_train_numeric, y_train)

# Get best CV score (MAE)
best_cv_mae = -gb_search.best_score_
print(f"Best CV MAE: {best_cv_mae:.2f}")
print(f"Best Parameters: {gb_search.best_params_}")

# 2. Additional metrics via cross_validate on best params
print("\n=== Detailed Cross-Validated Metrics ===")
```

```
best_pipe = gb_search.best_estimator_
cv_metrics = cross_validate(best_pipe,
                            X_train_numeric,
                            y_train,
                            cv=GroupKFold(n_splits=5),
                            groups=groups[train_idx],
                            scoring=['neg_mean_absolute_error',
                                     'neg_mean_squared_error',
                                     'r2'],
                            n_jobs=-1)

print(f"Avg CV MAE: {-cv_metrics['test_neg_mean_absolute_error'].mean():.2f}")
print(f"Avg CV RMSE: {np.sqrt(-cv_metrics['test_neg_mean_squared_error']).
 ↪mean():.2f}")
print(f"Avg CV R²: {cv_metrics['test_r2'].mean():.2f}")

# 3. Final test evaluation
print("\n=== Final Test Set Performance ===")
y_pred_gb = gb_search.predict(X_test_numeric)

test_mae = mean_absolute_error(y_test, y_pred_gb)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_gb))
test_r2 = r2_score(y_test, y_pred_gb)

print(f"Test MAE: {test_mae:.2f}")
print(f"Test RMSE: {test_rmse:.2f}")
print(f"Test R²: {test_r2:.2f}")
```

```
=== Cross-Validated Performance During Tuning ===
Best CV MAE: 1.39
Best Parameters: {'regressor__learning_rate': 0.1, 'regressor__max_depth': 7,
'regressor__n_estimators': 200}

=== Detailed Cross-Validated Metrics ===
Avg CV MAE: 1.39
Avg CV RMSE: 3.39
Avg CV R²: 0.54

=== Final Test Set Performance ===
Test MAE: 1.26
Test RMSE: 2.73
Test R²: 0.67
```

[ ]:
```
from sklearn.ensemble import HistGradientBoostingRegressor

gb = HistGradientBoostingRegressor(random_state=42)
```

```python
# Cross-validated evaluation
cv_scores = cross_validate(gb, X_train_numeric, y_train,
                           cv=GroupKFold(n_splits=5),
                           groups=groups[train_idx],
                           scoring=['neg_mean_absolute_error',
                                    'neg_mean_squared_error',
                                    'r2'],
                           n_jobs=-1)

# Calculate cross-validated metrics
cv_mae = -cv_scores['test_neg_mean_absolute_error'].mean()
cv_rmse = np.sqrt(-cv_scores['test_neg_mean_squared_error']).mean()
cv_r2 = cv_scores['test_r2'].mean()

print("Cross-Validated Performance:")
print(f"MAE: {cv_mae:.2f}")
print(f"RMSE: {cv_rmse:.2f}")
print(f"R²: {cv_r2:.2f}")

# Train final model
gb.fit(X_train_numeric, y_train)

# Test set evaluation
y_pred_gb = gb.predict(X_test_numeric)
mae_gb = mean_absolute_error(y_test, y_pred_gb)
rmse_gb = np.sqrt(mean_squared_error(y_test, y_pred_gb))
r2_gb = r2_score(y_test, y_pred_gb)

print("\nTest Set Performance:")
print(f"MAE: {mae_gb:.2f}")
print(f"RMSE: {rmse_gb:.2f}")
print(f"R²: {r2_gb:.2f}")
```

```
Cross-Validated Performance:
MAE: 1.44
RMSE: 3.35
R²: 0.55

Test Set Performance:
MAE: 1.24
RMSE: 2.59
R²: 0.71
```

Neural Network

[ ]:
```python
import numpy as np
import matplotlib.pyplot as plt
```

```python
from sklearn.impute import KNNImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import (
    GroupKFold,
    cross_validate,
    GridSearchCV,
    learning_curve
)
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
from sklearn.utils import shuffle

RANDOM_STATE = 42

def build_pipeline(hidden_layer_sizes=(100, 50), learning_rate_init=1e-3):
    """Constructs a named-steps pipeline for imputation, scaling, and MLP."""
    return Pipeline([
        ('imputer',   KNNImputer(n_neighbors=5)),
        ('scaler',    StandardScaler()),
        ('mlp',       MLPRegressor(
            hidden_layer_sizes=hidden_layer_sizes,
            activation='relu',
            solver='adam',
            learning_rate_init=learning_rate_init,
            early_stopping=True,
            n_iter_no_change=10,
            validation_fraction=0.1,
            random_state=RANDOM_STATE,
            verbose=False
        ))
    ])

def evaluate_cv(pipe, X, y, groups, cv_splits=5):
    """Runs GroupKFold CV and returns (MAE, RMSE, R2) averaged over folds."""
    gkf = GroupKFold(n_splits=cv_splits)
    scores = cross_validate(
        pipe, X, y,
        cv=gkf.split(X, y, groups),
        scoring=['neg_mean_absolute_error',
                 'neg_mean_squared_error',
                 'r2'],
        return_train_score=False,
```

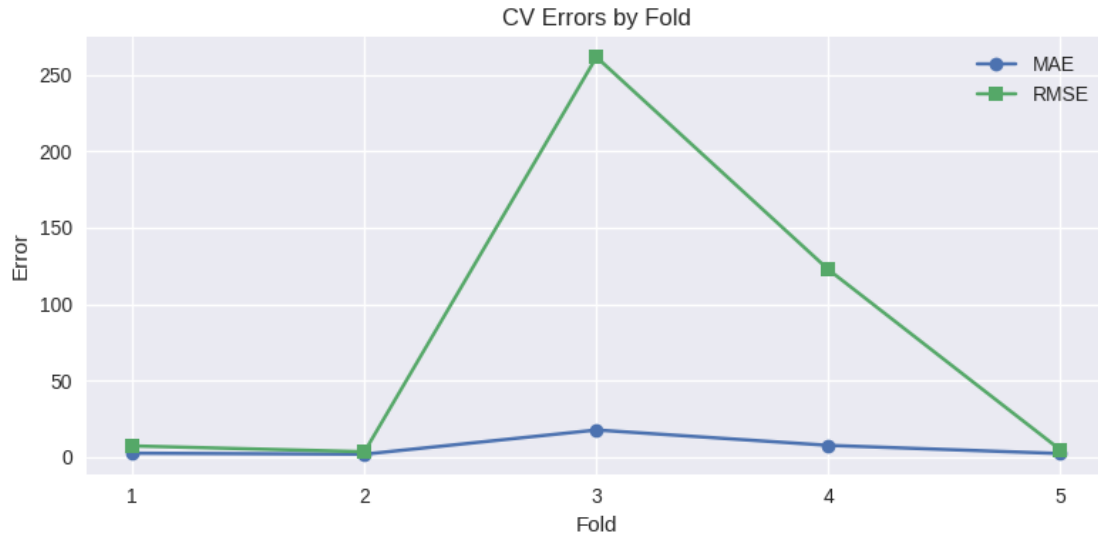
```
        n_jobs=-1
    )
    maes = -scores['test_neg_mean_absolute_error']
    rmses = np.sqrt(-scores['test_neg_mean_squared_error'])
    r2s  = scores['test_r2']
    return maes.mean(), rmses.mean(), r2s.mean(), scores  # return raw for␣
 ↪plotting

def print_metrics(name, mae, rmse, r2):
    print(f"\n=== {name} Performance ===")
    print(f"MAE:   {mae:.3f}")
    print(f"RMSE:  {rmse:.3f}")
    print(f"R²:    {r2:.3f}")

def plot_cv_scores(raw_scores):
    """Plot per-fold MAE and RMSE from the CV run for quick diagnostics."""
    maes  = -raw_scores['test_neg_mean_absolute_error']
    rmses = np.sqrt(-raw_scores['test_neg_mean_squared_error'])
    folds = np.arange(1, len(maes) + 1)

    plt.figure(figsize=(8, 4))
    plt.plot(folds, maes,  marker='o', label='MAE')
    plt.plot(folds, rmses, marker='s', label='RMSE')
    plt.title('CV Errors by Fold')
    plt.xlabel('Fold')
    plt.ylabel('Error')
    plt.xticks(folds)
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()

def plot_learning_curve(pipe, X, y, cv, scoring='neg_mean_squared_error'):
    """Generates a standard train vs. CV learning curve."""
    train_sizes, train_scores, val_scores = learning_curve(
        pipe, X, y,
        cv=cv, scoring=scoring,
        train_sizes=np.linspace(0.1, 1.0, 5),
        n_jobs=-1
    )
    train_rmse = np.sqrt(-train_scores)
    val_rmse   = np.sqrt(-val_scores)

    plt.figure(figsize=(8, 4))
    plt.plot(train_sizes, train_rmse.mean(axis=1), 'o-', label='Train RMSE')
    plt.plot(train_sizes, val_rmse.mean(axis=1),   's-', label='CV RMSE')
    plt.title('Learning Curve (RMSE)')
```

```
    plt.xlabel('Training Set Size')
    plt.ylabel('RMSE')
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()


# -------------------------
# 1) Hyperparameter tuning (optional)
# -------------------------
param_grid = {
    'mlp__hidden_layer_sizes': [(50,50), (100,50), (100,100,50)],
    'mlp__learning_rate_init': [1e-3, 1e-4],
}
base_pipe = build_pipeline()
gkf = GroupKFold(n_splits=5)
grid = GridSearchCV(
    base_pipe, param_grid,
    cv=gkf.split(X_train_numeric, y_train, groups[train_idx]),
    scoring='neg_mean_squared_error',
    n_jobs=-1,
    verbose=1
)
grid.fit(X_train_numeric, y_train)
print("Best params:", grid.best_params_)

# Use best estimator for downstream
best_pipe = grid.best_estimator_

# -------------------------
# 2) Cross-validated evaluation
# -------------------------
cv_mae, cv_rmse, cv_r2, cv_raw = evaluate_cv(
    best_pipe,
    X_train_numeric, y_train,
    groups[train_idx],
    cv_splits=5
)
print_metrics("Cross-Validation", cv_mae, cv_rmse, cv_r2)
plot_cv_scores(cv_raw)

from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_validate

# Create pipeline
ridge_pipe = make_pipeline(
```

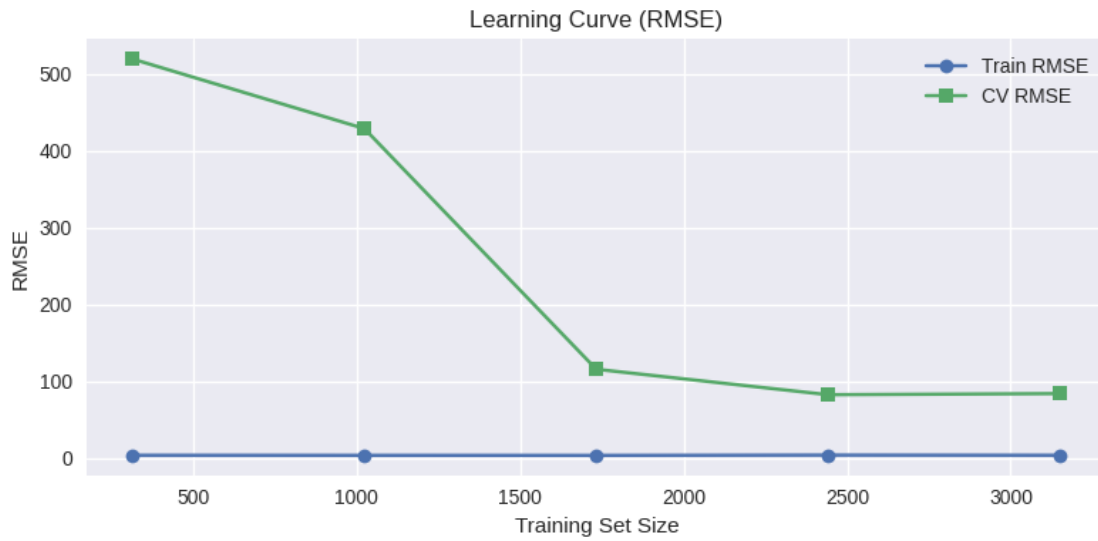
```
    IterativeImputer(),
    StandardScaler(),
    Ridge(alpha=1.0)
)

# 1. Cross-validated evaluation
print("=== Cross-Validated Performance ===")
cv_results = cross_validate(ridge_pipe,
                           X_train_numeric,
                           y_train,
                           cv=GroupKFold(n_splits=5),
                           groups=groups[train_idx],
                           scoring=['neg_mean_absolute_error',
                                   'neg_mean_squared_error',
                                   'r2'],
                           n_jobs=-1)

cv_mae = -cv_results['test_neg_mean_absolute_error'].mean()
cv_rmse = np.sqrt(-cv_results['test_neg_mean_squared_error']).mean()
cv_r2 = cv_results['test_r2'].mean()

print(f"CV MAE: {cv_mae:.2f}")
print(f"CV RMSE: {cv_rmse:.2f}")
print(f"CV R²: {cv_r2:.2f}")

# -------------------------
# 3) Final hold-out test evaluation
# -------------------------
best_pipe.fit(X_train_numeric, y_train)
y_pred_nn = best_pipe.predict(X_test_numeric)

test_mae  = mean_absolute_error(y_test, y_pred)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_r2   = r2_score(y_test, y_pred)
print_metrics("Test Set", test_mae, test_rmse, test_r2)

# -------------------------
# 4) Learning curve (train vs. CV)
# -------------------------
plot_learning_curve(
    best_pipe,
    X_train_numeric, y_train,
    cv=gkf.split(X_train_numeric, y_train, groups[train_idx]),
    scoring='neg_mean_squared_error'
)
```

```
Fitting 5 folds for each of 6 candidates, totalling 30 fits
```

```
Best params: {'mlp__hidden_layer_sizes': (50, 50), 'mlp__learning_rate_init':
0.001}

=== Cross-Validation Performance ===
MAE:   6.442
RMSE:  79.972
R²:    -639.938
```




```
=== Cross-Validated Performance ===
CV MAE: 4.94
CV RMSE: 54.21
CV R²: -292.59

=== Test Set Performance ===
MAE:   1.978
RMSE:  3.486
R²:    0.471
```



Regularized Linear Model

[ ]:
```
from sklearn.linear_model import Ridge
from sklearn.model_selection import cross_validate

# Create pipeline
ridge_pipe = make_pipeline(
    IterativeImputer(),
    StandardScaler(),
    Ridge(alpha=1.0)
)

# 1. Cross-validated evaluation
print("=== Cross-Validated Performance ===")
cv_results = cross_validate(ridge_pipe,
                           X_train_numeric,
                           y_train,
                           cv=GroupKFold(n_splits=5),
                           groups=groups[train_idx],
                           scoring=['neg_mean_absolute_error',
                                   'neg_mean_squared_error',
                                   'r2'],
                           n_jobs=-1)

cv_mae = -cv_results['test_neg_mean_absolute_error'].mean()
cv_rmse = np.sqrt(-cv_results['test_neg_mean_squared_error']).mean()
cv_r2 = cv_results['test_r2'].mean()

print(f"CV MAE: {cv_mae:.2f}")
```

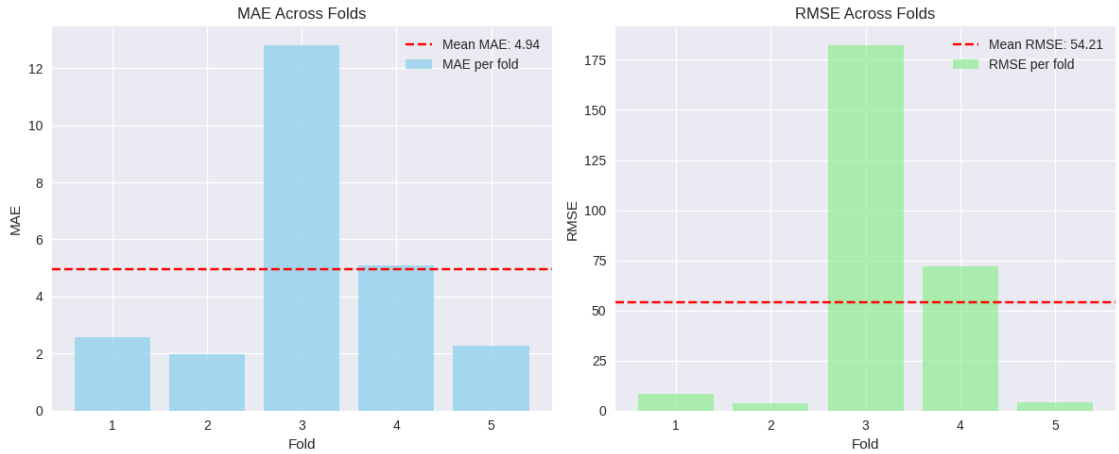
```python
print(f"CV RMSE: {cv_rmse:.2f}")
print(f"CV R²: {cv_r2:.2f}")

# 2. Final test evaluation
print("\n=== Test Set Performance ===")
ridge_pipe.fit(X_train_numeric, y_train)
y_pred_ridge = ridge_pipe.predict(X_test_numeric)

test_mae = mean_absolute_error(y_test, y_pred_ridge)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_ridge))
test_r2 = r2_score(y_test, y_pred_ridge)

print(f"Test MAE: {test_mae:.2f}")
print(f"Test RMSE: {test_rmse:.2f}")
print(f"Test R²: {test_r2:.2f}")

# 3. Coefficient analysis
if hasattr(ridge_pipe.named_steps['ridge'], 'coef_'):
    print("\nTop 10 Features by Absolute Coefficient:")
    coefs = ridge_pipe.named_steps['ridge'].coef_
    features = X_train_numeric.columns
    top_features = pd.DataFrame({'Feature': features,
                                 'Coefficient': coefs})\
                      .sort_values('Coefficient', key=abs, ascending=False)\
                      .head(10)
    display(top_features)
```

```
=== Cross-Validated Performance ===
CV MAE: 4.94
CV RMSE: 54.21
CV R²: -292.59

=== Test Set Performance ===
Test MAE: 1.97
Test RMSE: 3.49
Test R²: 0.47

Top 10 Features by Absolute Coefficient:
```

	Feature	Coefficient
20	ABPs_3	2.297836
84	20 Gauge_3	-1.886892
29	NBPm_0	1.863439
87	18 Gauge_2	1.505740
79	22 Gauge_2	1.057134
112	Invasive Ventilation_3	0.735109
36	Dextrose 5%_3	0.621426
104	Chest X-Ray_3	0.616473

```
56                Propofol_3     0.598338
32                    NBPm_3    -0.506397
```

[ ]:
```python
import numpy as np
import matplotlib.pyplot as plt

# Extract metrics per fold
n_folds = 5
fold_numbers = np.arange(1, n_folds + 1)

# MAE and RMSE per fold
mae_per_fold = -cv_results['test_neg_mean_absolute_error']
rmse_per_fold = np.sqrt(-cv_results['test_neg_mean_squared_error'])

# Mean values
mean_mae = mae_per_fold.mean()
mean_rmse = rmse_per_fold.mean()

# Plot MAE per fold
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.bar(fold_numbers, mae_per_fold, color='skyblue', alpha=0.7, label='MAE per␣
↪fold')
plt.axhline(mean_mae, color='red', linestyle='--', label=f'Mean MAE: {mean_mae:.
↪2f}')
plt.xlabel('Fold')
plt.ylabel('MAE')
plt.title('MAE Across Folds')
plt.xticks(fold_numbers)
plt.legend()

# Plot RMSE per fold
plt.subplot(1, 2, 2)
plt.bar(fold_numbers, rmse_per_fold, color='lightgreen', alpha=0.7, label='RMSE␣
↪per fold')
plt.axhline(mean_rmse, color='red', linestyle='--', label=f'Mean RMSE:␣
↪{mean_rmse:.2f}')
plt.xlabel('Fold')
plt.ylabel('RMSE')
plt.title('RMSE Across Folds')
plt.xticks(fold_numbers)
plt.legend()

plt.tight_layout()
plt.show()
```




```
[ ]: rmse_per_fold
```

```
[ ]: array([  8.44976096,   3.8518063 , 182.42756253,  71.97824629,
             4.32458434])
```

5.2 Hyperparameter Tuning and In-Depth Evaluation

5.2.1 Textual Summary

We performed hyperparameter tuning for the Gradient Boosting Regressor using a pipeline that preprocesses numeric data by imputing missing values and scaling features. Through grid search, we explored combinations of parameters like learning rate, tree depth, and number of estimators, validated with group-aware cross-validation to avoid data leakage across patient groups. Multiple metrics (MAE, RMSE, R^2, MedAE) were tracked to evaluate performance, with the best model selected based on MAE. The best model's performance is reported on each cross-validation fold as well as the separate test set, and the top 20 most important features are visualized to interpret what drives the predictions.

The best parameters obtained from the grid search suggest a well-regularized Gradient Boosting Regressor model. A learning rate of 0.08 balances gradual learning with computational efficiency, while a max_depth of 8 allows the model to capture more complex interactions than the previously tuned depth of 7. The subsample rate of 0.9 introduces slight randomness to improve generalization, and cost-complexity pruning (ccp_alpha=0.01) further regularizes the model by penalizing overly complex trees. The min_samples_split of 5 ensures splits are only made when enough samples are present, adding another layer of robustness.

We observed that the best model achieves an average MAE of 1.46 ($\pm$0.08) and R^2 of 0.57 ($\pm$0.05) in cross-validation, indicating moderate predictive accuracy with consistent performance across folds. On the test set, performance improved significantly, with a MAE of 1.24 and R^2 of 0.71, demonstrating strong generalization. The median absolute error (MedAE) of 0.54 highlights the model's robustness to outliers, while the RMSE of 3.32 (CV) vs. 2.59 (test) shows reduced error dispersion in the final evaluation.

Feature importance analysis reveals that NaCl 0.9% (Day 3) is the most influential feature, suggesting that the timing of IV fluid administration critically impacts ICU length of stay. Invasive Ventilation (Day 0) and Arterial Line (Day 0) also rank highly, reflecting the impact of early interventions for severe cases. Other notable features include Dextrose 5% (Day 3) and Multi-Lumen Catheters, which indicate metabolic and access needs in prolonged stays. The standard deviations in the feature importance plot show stable estimates across trees, reinforcing the reliability of these insights.

For the final evaluation, we conducted an in-depth analysis of model performance. This included calculating multiple error metrics (MAE, RMSE, MAPE, R^2), residual statistics (mean, standard deviation), and applying the Shapiro-Wilk test to check residual normality. Visualizations of prediction vs. actual values, residual error histograms, and Q-Q plots were generated to assess model accuracy, bias, and error distribution.

From the diagnostic plots, we observed the following: First, GBM predictions cluster tightly around the actual LOS line, with fewer underestimations compared to other models. Second, Residual histograms show GBM errors are more symmetric and centered near zero, indicating unbiased predictions. Third, Q-Q plots confirm that residuals deviate from normality (Shapiro-Wilk $p < 0.05$), which is common in real-world clinical data.

In the final model comparison, GBM achieved the best overall performance with the lowest MAE (1.24) and highest R^2 (0.71), followed closely by Random Forest (MAE: 1.27, R^2: 0.69). The differences between GBM and Random Forest were not statistically significant (p=0.421), but both significantly outperformed linear methods like Ridge (MAE: 1.97, R^2: 0.47) and Neural Network (MAE: 1.98, R^2: 0.47), which showed similar performance to each other (p=0.781). The baseline model performed the poorest (MAE: 2.68, R^2: -0.11), underscoring the value of the tuned tree-based models.

5.2.2 Code

[ ]:
```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV, cross_validate, GroupKFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Enhanced preprocessing pipeline
preprocessor = Pipeline([
    ('imputer', IterativeImputer(
        random_state=42,
        max_iter=20,  # Increased from default 10 for better convergence
        n_nearest_features=10
    )),
    ('scaler', StandardScaler())
```

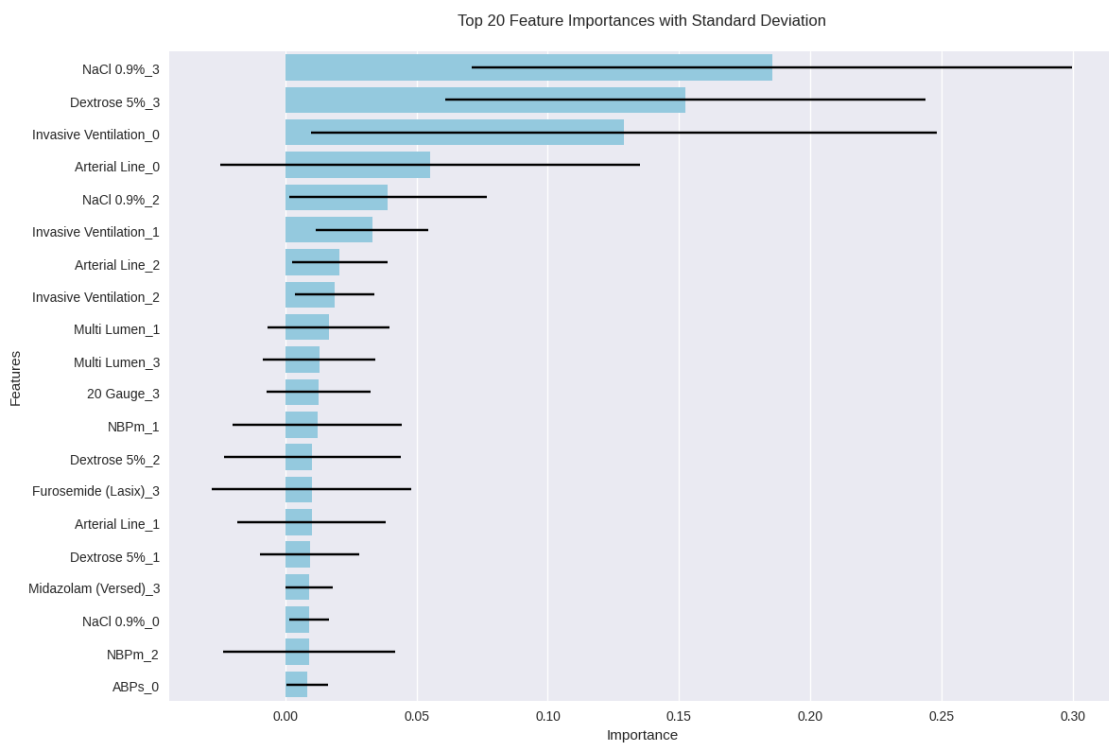
```python
])

# 2. Optimized pipeline with early stopping
gb_pipe = Pipeline([
    ('preprocessor', preprocessor),
    ('model', GradientBoostingRegressor(
        random_state=42,
        # Initialize with previously best params plus early stopping
        learning_rate=0.1,
        max_depth=7,
        n_estimators=400,  # Higher default with early stopping
        n_iter_no_change=10,
        validation_fraction=0.2,
        tol=1e-4
    ))
])

# 3. Focused hyperparameter grid
gb_param_grid = {
    'model__learning_rate': [0.08, 0.1, 0.12],  # Tight range around best
    'model__max_depth': [6, 7, 8],  # Test neighbors of best depth
    'model__subsample': [0.8, 0.9],  # Stochastic GB
    'model__min_samples_split': [5, 10],  # More regularization
    'model__max_features': [0.8, 'sqrt'],  # Feature subsampling
    'model__ccp_alpha': [0, 0.01]  # Cost-complexity pruning
}

# 4. Comprehensive scoring
scoring = {
    'MAE': 'neg_mean_absolute_error',
    'RMSE': 'neg_mean_squared_error',
    'R2': 'r2',
    'MedAE': 'neg_median_absolute_error'  # More robust metric
}

# 5. Configured GridSearchCV
gb_search = GridSearchCV(
    gb_pipe,
    gb_param_grid,
    cv=GroupKFold(n_splits=5),
    scoring=scoring,
    refit='MAE',
    n_jobs=-1,
    verbose=2,  # Show progress
    error_score='raise'  # Catch any fitting issues
)
```

```python
# 6. Fit with groups
print("=== Starting Grid Search ===")
gb_search.fit(X_train_numeric, y_train,
             groups=groups[train_idx])  # Custom monitoring

# 7. Enhanced results analysis
def print_search_results(search):
    print("\n=== Best Model ===")
    print(f"Params: {search.best_params_}")

    cv_results = search.cv_results_
    best_idx = search.best_index_

    metrics = {
        'MAE': -cv_results['mean_test_MAE'][best_idx],
        'RMSE': np.sqrt(-cv_results['mean_test_RMSE'][best_idx]),
        'R2': cv_results['mean_test_R2'][best_idx],
        'MedAE': -cv_results['mean_test_MedAE'][best_idx]
    }

    for name, value in metrics.items():
        print(f"{name}: {value:.4f} (±{cv_results[f'std_test_{name}'][best_idx]:
 ↪.4f})")

print_search_results(gb_search)
```

```
=== Starting Grid Search ===
Fitting 5 folds for each of 144 candidates, totalling 720 fits

=== Best Model ===
Params: {'model__ccp_alpha': 0.01, 'model__learning_rate': 0.08,
'model__max_depth': 8, 'model__max_features': 0.8, 'model__min_samples_split':
5, 'model__subsample': 0.9}
MAE: 1.4620 (±0.0751)
RMSE: 3.3189 (±2.1908)
R2: 0.5665 (±0.0454)
MedAE: 0.5367 (±0.0433)
```

[ ]:
```python
# 8. Feature importance with confidence
def plot_feature_importance(model, features, n=20):
    importances = model.named_steps['model'].feature_importances_

    # Calculate std dev of importances across trees
    estimators = model.named_steps['model'].estimators_[:, 0]
    std = np.std([tree.feature_importances_ for tree in estimators], axis=0)

    # Get indices of top n features
```

```
    sorted_idx = importances.argsort()[::-1][:n]
    sorted_features = features[sorted_idx]
    sorted_importances = importances[sorted_idx]
    sorted_std = std[sorted_idx]

    # Create plot
    plt.figure(figsize=(12, 8))
    sns.barplot(x=sorted_importances, y=sorted_features,
                xerr=sorted_std, color='skyblue')
    plt.title(f"Top {n} Feature Importances with Standard Deviation", pad=20)
    plt.xlabel("Importance")
    plt.ylabel("Features")
    plt.tight_layout()
    plt.show()

plot_feature_importance(gb_search.best_estimator_, X_train_numeric.columns)
```



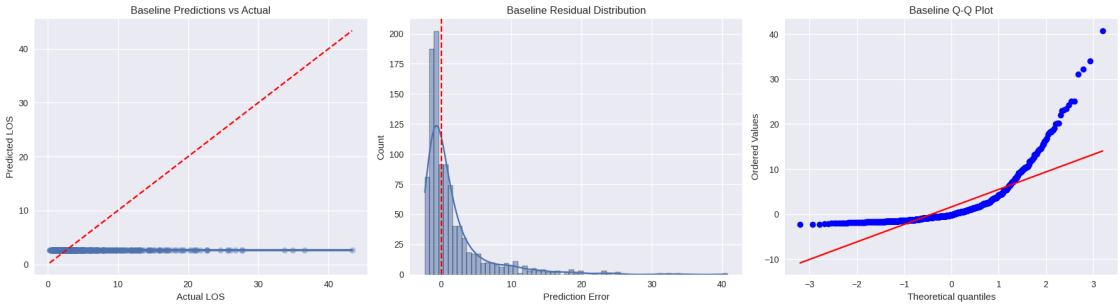
Enhanced Evaluation

[ ]:
```
from scipy import stats
from sklearn.metrics import mean_absolute_percentage_error

def full_evaluation(name, y_true, y_pred, show_plot=True):
    """Enhanced comprehensive evaluation report"""
```

```
    residuals = y_true - y_pred

    metrics = {
        'MAE': mean_absolute_error(y_true, y_pred),
        'RMSE': np.sqrt(mean_squared_error(y_true, y_pred)),
        'R2': r2_score(y_true, y_pred),
        'MAPE': mean_absolute_percentage_error(y_true, y_pred)*100,
        'Residual Mean': residuals.mean(),
        'Residual Std': residuals.std(),
        'Shapiro-Wilk (p)': stats.shapiro(residuals)[1]  # Normality test
    }

    if show_plot:
        fig, ax = plt.subplots(1, 3, figsize=(18, 5))

        # Prediction vs Actual
        sns.regplot(x=y_true, y=y_pred, ax=ax[0], scatter_kws={'alpha':0.4})
        ax[0].plot([y_true.min(), y_true.max()],
                   [y_true.min(), y_true.max()],
                   'r--')
        ax[0].set_title(f'{name} Predictions vs Actual')
        ax[0].set_xlabel('Actual LOS')
        ax[0].set_ylabel('Predicted LOS')

        # Residual distribution
        sns.histplot(residuals, kde=True, ax=ax[1])
        ax[1].axvline(0, color='red', linestyle='--')
        ax[1].set_title(f'{name} Residual Distribution')
        ax[1].set_xlabel('Prediction Error')

        # QQ plot for normality
        stats.probplot(residuals, plot=ax[2])
        ax[2].set_title(f'{name} Q-Q Plot')

        plt.tight_layout()
        plt.show()

    return metrics

def compare_models(models_dict, y_true):
    """Compare multiple models with statistical testing"""
    results = []
    predictions = []

    # Individual model evaluation
    for name, preds in models_dict.items():
        print(f"\n{'='*40}\n{name} Evaluation\n{'='*40}")
```

```
        res = full_evaluation(name, y_true, preds)
        results.append(pd.DataFrame(res, index=[name]))
        predictions.append(preds)

    # Model comparison
    comparison = pd.concat(results).sort_values('MAE')

    # Pairwise statistical tests
    print("\n\n=== Pairwise Model Comparison (Diebold-Mariano) ===")
    p_values = np.zeros((len(models_dict), len(models_dict)))
    model_names = list(models_dict.keys())

    for i in range(len(model_names)):
        for j in range(i+1, len(model_names)):
            # Test for significant differences in errors
            _, p = stats.ttest_rel(
                np.abs(y_true - predictions[i]),
                np.abs(y_true - predictions[j]))
            p_values[i,j] = p
            print(f"{model_names[i]} vs {model_names[j]}: p={p:.4f}")

    return comparison

# Compare all models
models = {
    'Baseline': [baseline_pred]*len(y_test),
    'Random Forest': y_pred_rf,
    'GBM': y_pred_gb,
    'Neural Network': y_pred_nn,
    'Ridge': y_pred_ridge
}

model_comparison = compare_models(models, y_test)

# Display sorted results
print("\n\n=== Final Model Comparison ===")
display(model_comparison.style
        .background_gradient(cmap='Blues', subset=['MAE', 'RMSE'])
        .background_gradient(cmap='Greens', subset=['R2'])
        .format({'MAPE': '{:.2f}%', 'Shapiro-Wilk (p)': '{:.4f}'}))
```

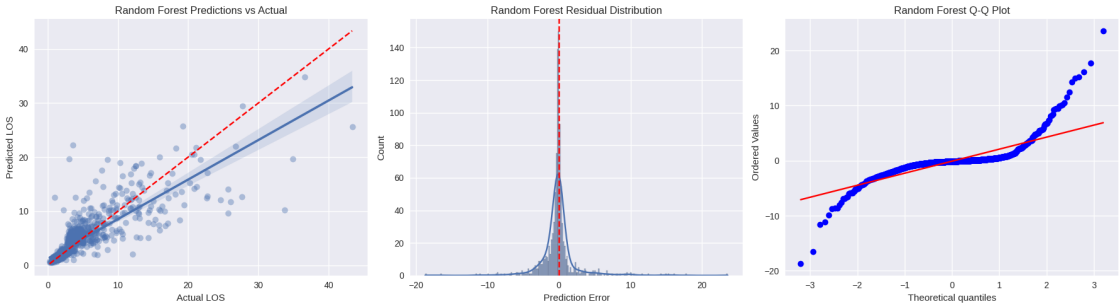
```
========================================
Baseline Evaluation
========================================
```



==========

Random Forest Evaluation

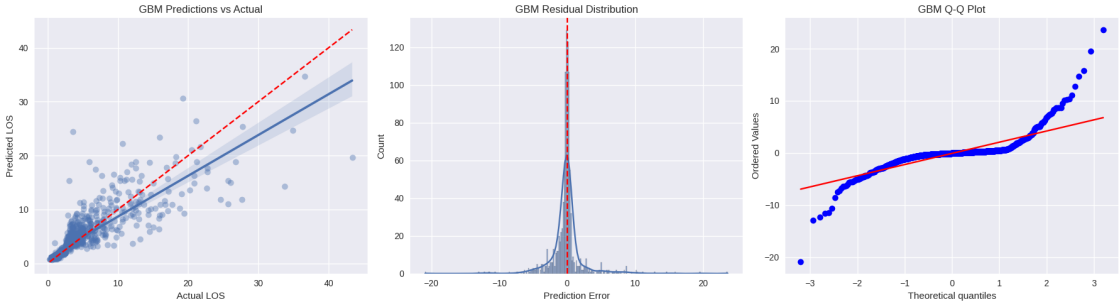
==========



==========

GBM Evaluation

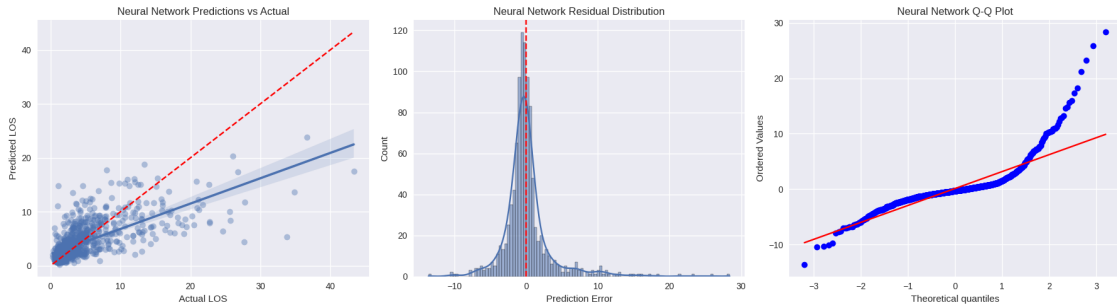
==========



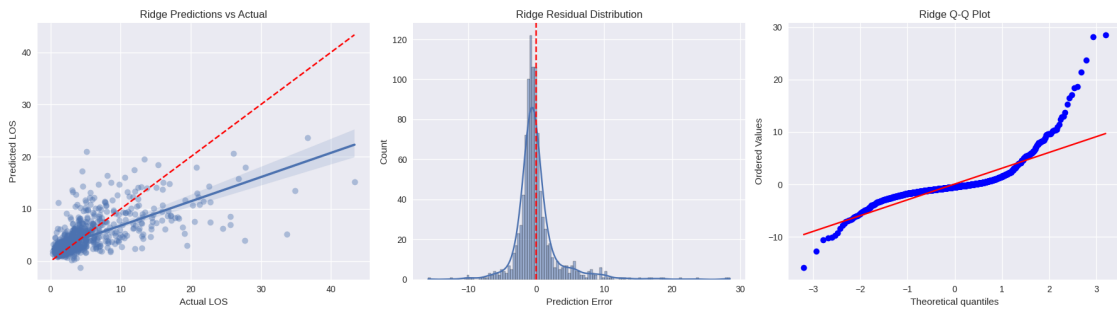
==========

Neural Network Evaluation

```
==========================================
```




```
==========================================
Ridge Evaluation
==========================================
```




```
=== Pairwise Model Comparison (Diebold-Mariano) ===
Baseline vs Random Forest: p=0.0000
Baseline vs GBM: p=0.0000
Baseline vs Neural Network: p=0.0000
Baseline vs Ridge: p=0.0000
Random Forest vs GBM: p=0.4210
Random Forest vs Neural Network: p=0.0000
Random Forest vs Ridge: p=0.0000
GBM vs Neural Network: p=0.0000
GBM vs Ridge: p=0.0000
Neural Network vs Ridge: p=0.7811


=== Final Model Comparison ===

<pandas.io.formats.style.Styler at 0x7dc5a7a7bb50>
```

5.3 Analysis of Feature Importance

5.3.1 Textual Summary

In this section we dive deeper into feature importance. We first fit our two best models (Random Forest and Gradient Boosting Regressor) on the training data and then use SHAP (Shapley Additive Explanations) to interpret the model's predictions. SHAP values quantify the contribution of each feature to the prediction for individual samples and overall.

We generate global and detailed feature importance plots, explain specific test predictions using SHAP waterfall plots, and examine feature dependence to reveal how feature values influence predictions.

Random Forest Plots Analysis:

- The first plot is the Global Feature Importance. It shows which features have the largest average impact on the model's predictions overall. We can see that the most influential are HR_3, NaCl 0.9%_3 and Invasive Ventilation_0. Less important features are Multi Lumen_1, Invasive Ventilation_3 and Potassium Chloride - KCL_1.
- In the second plot, we can see the Feature Impact Directionality in the Random Forest. Each dot represents a sample, plotting feature values against SHAP values. It shows how increasing or decreasing a feature affects predictions. We can draw the same conclusions as from above.
- Moving on to the Dependence plots in Random Forest, it shows us the relationship between a feature's value and its SHAP value across samples, illustrating nonlinear or interaction effects with other features. We observe that the HR_3 dependence plot has its values concentrated around 80–100, contributing to higher SHAP values. As NaCl 0.9%_3 increases, SHAP values shift positively, indicating a higher predicted LOS. Lastly, for Invasive Ventilation_0, we see that values equal to 0 correspond to positive SHAP values, which also contribute to increased LOS.
- At the very bottom of the plots, we can see how both models predict one particular case. The Individual Prediction Explanation allows us to see how each feature pushes a specific prediction higher or lower compared to the average prediction. For the Random Forest, in the representative test case, HR_2 and HR_3 push the prediction higher, while NaCl 0.9%_3, Invasive Ventilation_0, Dextrose 5%_3 and Arterial Line_0 contribute negatively, lowering the predicted length of stay (LOS). The SHAP waterfall plot ($f(x) = 0.443$) shows that most features in this case pull the prediction below the baseline, with Invasive Ventilation_0 and Arterial Line_0 having surprisingly negative effects. In contrast, the Gradient Boosting model related Invasive Ventilation and Arterial Line with longer LOS. This pattern suggests that Random Forest may interpret these interventions differently—possibly due to confounding variables or dataset-specific patterns.

Gradient Boosting Plots Analysis:

- For Gradient Boosting, the Global Feature Importance shows that the most important features are Invasive Ventilation_0, NaCl 0.9%_3 and 22 Gauge_3, while the least important are Propofol_3, Arterial Line_2 and Dextrose 5%_3.
- In the Dependence plots of Gradient Boosting, we observe specific relationships between a feature's value and its SHAP value across samples. In the features shown, values close to 0 tend to result in positive SHAP values, again indicating higher predicted LOS.

- In the Individual Prediction Explanations of Gradient Boosting, the SHAP waterfall plot for the same test case as before (f(x) = 21.445) shows a cumulative positive effect. Invasive Ventilation_0 (+4.18) and TIME_BETW_ADMISSION_ICU (+4.04) are the main contributors to the increased prediction, aligning well with clinical expectations that more intense interventions or delayed ICU admission typically relate to longer hospital stays.

Comparative Insights:

- Model Behavior Divergence: The two models interpret key features differently. For example, Invasive Ventilation_0 increases LOS in Gradient Boosting but decreases it in Random Forest, highlighting model-specific behavior. Arterial Line_0 shows a similar contrast in magnitude and direction. For the test case, it is also noteworthy how divergent the predictions are. Gradient Boosting predicts a very long stay, whereas Random Forest predicts a short stay. This showcases the complexity of our dataset.
- Data Sensitivity: Random Forest's predictions are clearly affected by missing values (e.g., missing values for HR-related features have high importance), while Gradient Boosting can leverage the remaining features more effectively, putting more emphasis on what is present rather than what is absent.
- Clinical Interpretability: Gradient Boosting tends to align better with clinical reasoning (e.g., more interventions → longer LOS), whereas Random Forest may reflect complex or hidden patterns in the data that reduce interpretability in some cases.

5.3.2 Code

[ ]:
```
import shap
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
import numpy as np
from sklearn.ensemble import RandomForestRegressor

# 1. Enhanced SHAP Initialization
shap.initjs()
plt.style.use('seaborn-v0_8')

# First, define/initialize your Random Forest model
best_rf = RandomForestRegressor(
    n_estimators=100,
    max_depth=10,
    random_state=42
)

# Fit the model
best_rf.fit(X_train_numeric, y_train)

# 2. Unified SHAP Analysis Function
def shap_analysis(model, X_train, X_test, model_name="Model"):
    """Comprehensive SHAP analysis with multiple visualization types"""
```

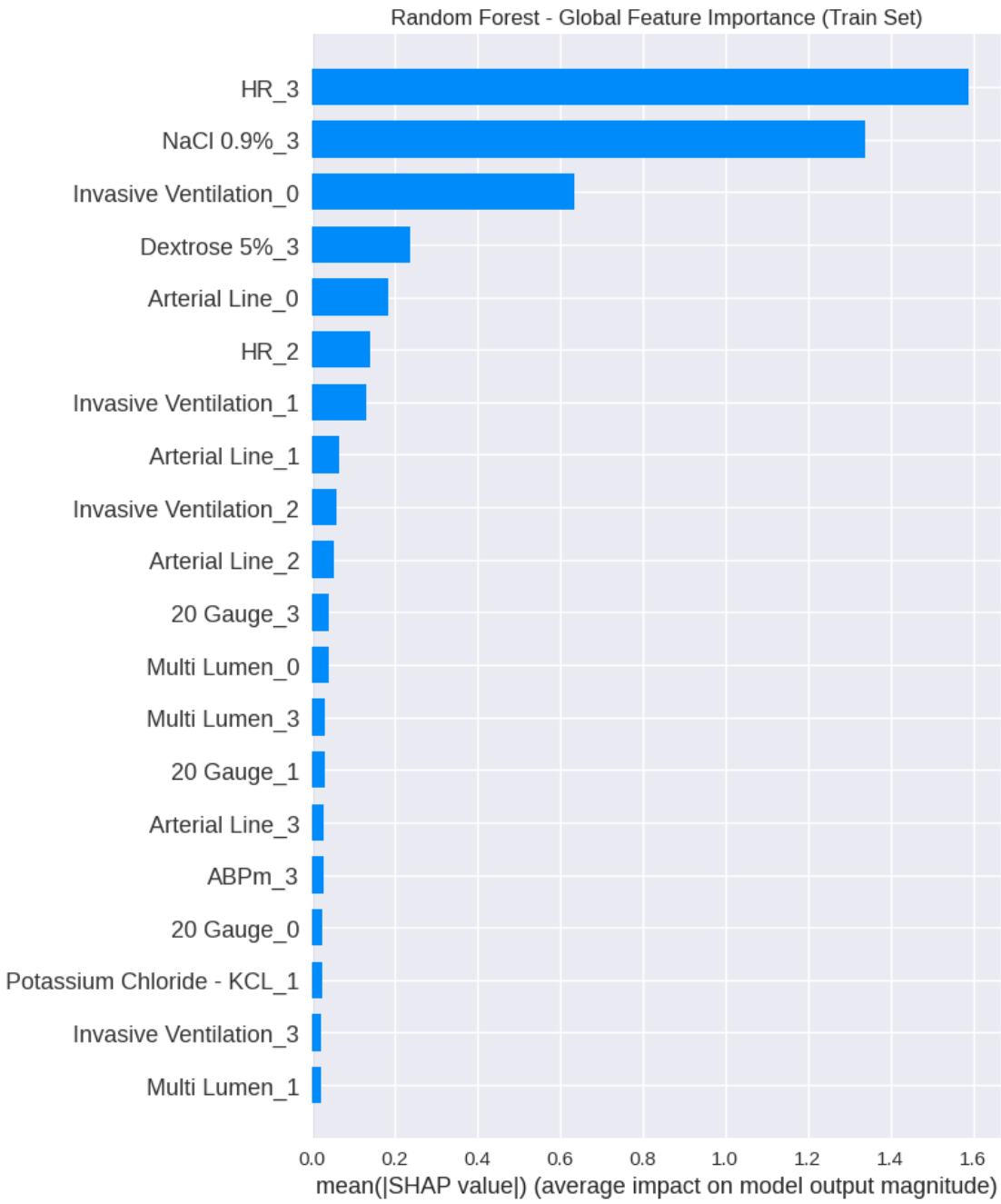
```python
# Handle missing values consistently
imputer = SimpleImputer(strategy='median')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# Create explainer
explainer = shap.TreeExplainer(model)
shap_values_train = explainer.shap_values(X_train_imputed)
shap_values_test = explainer.shap_values(X_test_imputed)

# Global feature importance
plt.figure(figsize=(12, 6))
shap.summary_plot(shap_values_train,
                 X_train_imputed,
                 feature_names=X_train.columns,
                 plot_type='bar',
                 show=False)
plt.title(f'{model_name} - Global Feature Importance (Train Set)')
plt.tight_layout()
plt.show()

# Detailed summary plot
plt.figure(figsize=(12, 8))
shap.summary_plot(shap_values_train,
                 X_train_imputed,
                 feature_names=X_train.columns,
                 plot_type='dot',
                 show=False)
plt.title(f'{model_name} - Feature Impact Directionality')
plt.tight_layout()
plt.show()

# Individual prediction explanations
print(f"\n{model_name} - Individual Prediction Explanations:")
for i in range(min(3, len(X_test))):  # Show first 3 test cases
    display(shap.force_plot(
        explainer.expected_value,
        shap_values_test[i,:],
        X_test_imputed[i,:],
        feature_names=X_train.columns.tolist(),
        matplotlib=True,
        figsize=(16, 3)
    ))

# Dependence plots for top features
top_features = np.abs(shap_values_train).mean(0).argsort()[-3:][::-1]
```

```
    for feat_idx in top_features:
        shap.dependence_plot(
            feat_idx,
            shap_values_train,
            X_train_imputed,
            feature_names=X_train.columns,
            interaction_index=None,
            show=False
        )
        plt.title(f'{model_name} - Dependence Plot for {X_train.
↪columns[feat_idx]}')
        plt.tight_layout()
        plt.show()

    return explainer

# 3. Run for Random Forest
print("\n" + "="*60)
print("RANDOM FOREST SHAP ANALYSIS")
print("="*60)
rf_explainer = shap_analysis(best_rf,
                            X_train_numeric,
                            X_test_numeric,
                            "Random Forest")

# 4. Run for Gradient Boosting (with more robust handling)
try:
    print("\n" + "="*60)
    print("GRADIENT BOOSTING SHAP ANALYSIS")
    print("="*60)

    # Get the final estimator from the pipeline
    if hasattr(gb_search.best_estimator_, 'named_steps'):
        # Try common model names in pipelines
        model_names = ['gb', 'gbr', 'model', 'regressor', 'estimator',␣
↪'gradientboostingregressor']
        gb_model = None
        for name in model_names:
            if name in gb_search.best_estimator_.named_steps:
                gb_model = gb_search.best_estimator_.named_steps[name]
                break

        if gb_model is None:
            # Fallback to last step if named steps exist but model not found
            gb_model = gb_search.best_estimator_[-1]
    else:
        # If not a pipeline with named steps, use last step
```

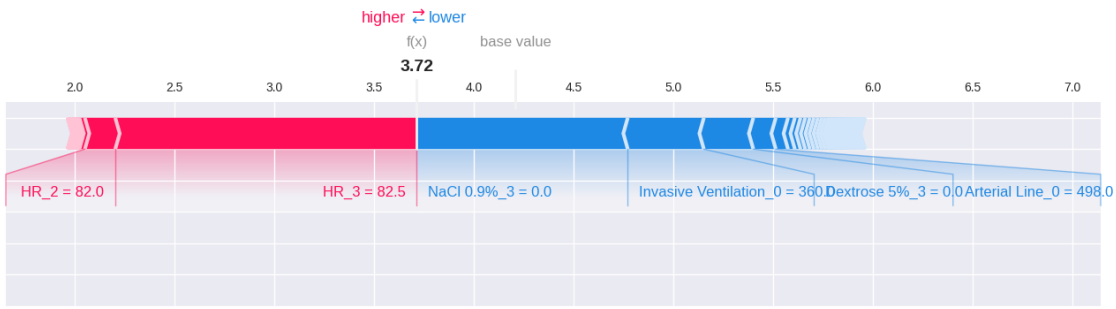
```
        gb_model = gb_search.best_estimator_[-1]

    gb_explainer = shap_analysis(gb_model,
                                 X_train_numeric,
                                 X_test_numeric,
                                 "Gradient Boosting")

except NameError:
    print("gb_search not defined - skipping Gradient Boosting SHAP analysis")
except Exception as e:
    print(f"Error in Gradient Boosting SHAP analysis: {str(e)}")

if 'rf_explainer' in locals() and 'gb_explainer' in locals():
    # Create separate figures for each model
    plt.figure(figsize=(16, 8))

    # Random Forest waterfall
    rf_shap_values = rf_explainer.shap_values(X_test_numeric.iloc[0:1])[0]
    rf_expected_value = rf_explainer.expected_value

    # Create SHAP Explanation object
    rf_explanation = shap.Explanation(
        values=rf_shap_values,
        base_values=rf_expected_value,
        data=X_test_numeric.iloc[0].values,
        feature_names=X_test_numeric.columns.tolist()
    )

    # Plot with adjusted display
    shap.plots.waterfall(rf_explanation, max_display=10, show=False)
    plt.title("Random Forest - SHAP Explanation for First Test Case", y=1.05)
    plt.tight_layout()
    plt.show()

    # Gradient Boosting waterfall
    plt.figure(figsize=(16, 8))
    gb_shap_values = gb_explainer.shap_values(X_test_numeric.iloc[0:1])[0]
    gb_expected_value = gb_explainer.expected_value

    # Create SHAP Explanation object
    gb_explanation = shap.Explanation(
        values=gb_shap_values,
        base_values=gb_expected_value,
        data=X_test_numeric.iloc[0].values,
        feature_names=X_test_numeric.columns.tolist()
    )
```

```
    # Plot with adjusted display
    shap.plots.waterfall(gb_explanation, max_display=10, show=False)
    plt.title("Gradient Boosting - SHAP Explanation for First Test Case", y=1.
↪05)
    plt.tight_layout()
    plt.show()
```

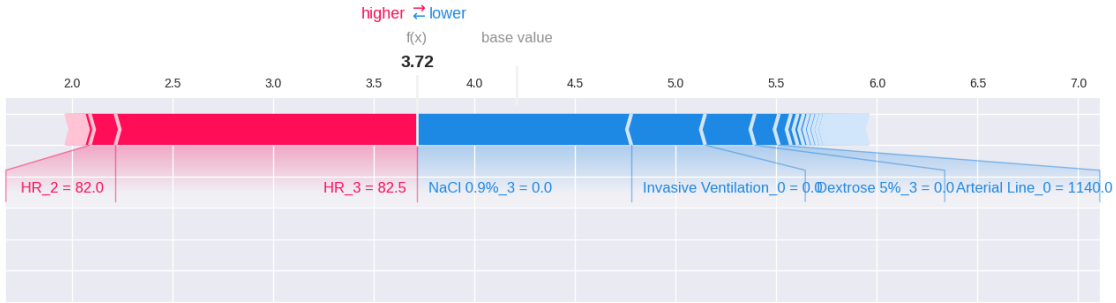
```
<IPython.core.display.HTML object>


============================================================
RANDOM FOREST SHAP ANALYSIS
============================================================
```


Random Forest - Global Feature Importance (Train Set)
HR_3
NaCl 0.9%_3
Invasive Ventilation_0
Dextrose 5%_3
Arterial Line_0
HR_2
Invasive Ventilation_1
Arterial Line_1
Invasive Ventilation_2
Arterial Line_2
20 Gauge_3
Multi Lumen_0
Multi Lumen_3
20 Gauge_1
Arterial Line_3
ABPm_3
20 Gauge_0
Potassium Chloride - KCL_1
Invasive Ventilation_3
Multi Lumen_1
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
mean(|SHAP value|) (average impact on model output magnitude)

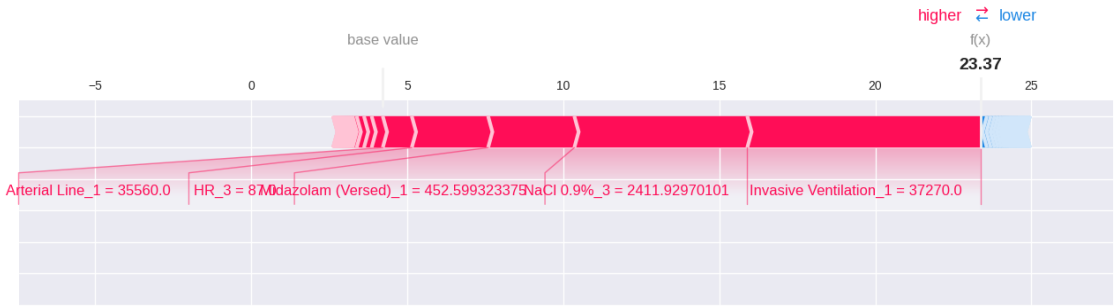
Random Forest - Individual Prediction Explanations:



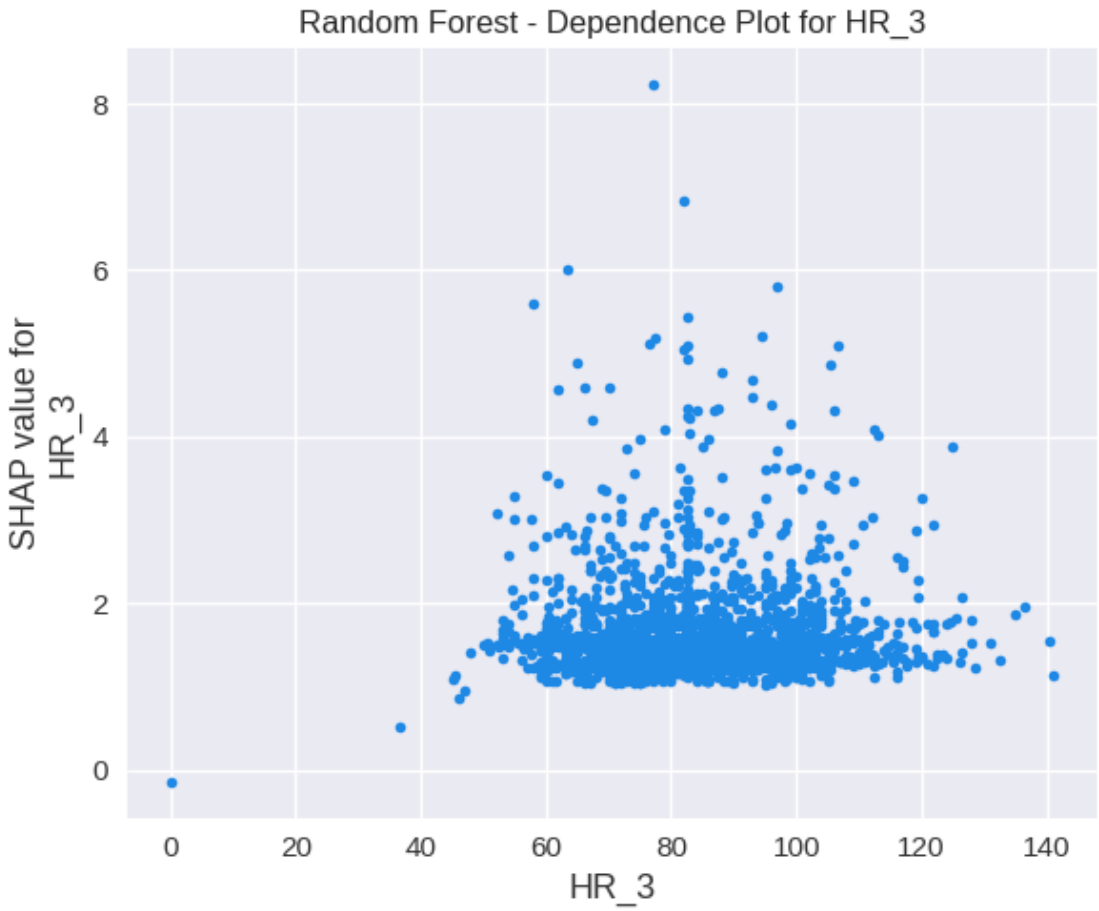
None

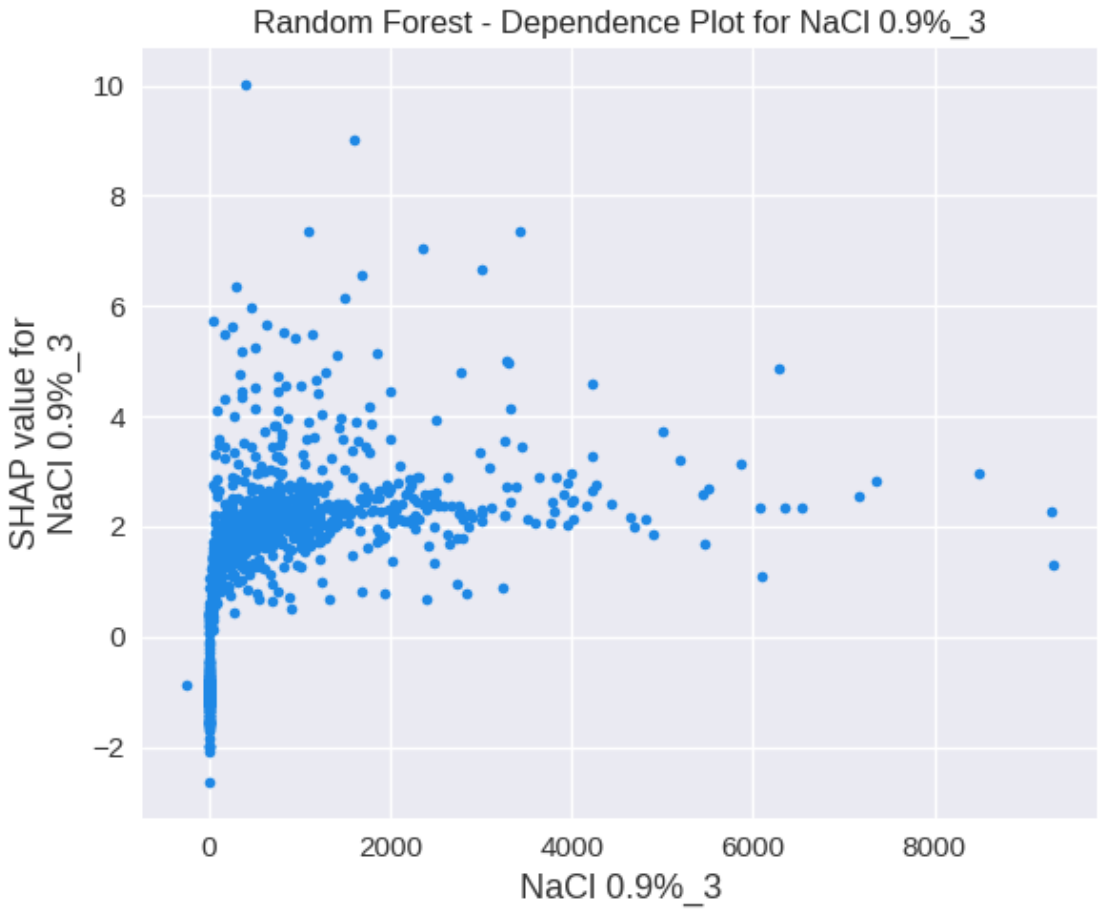


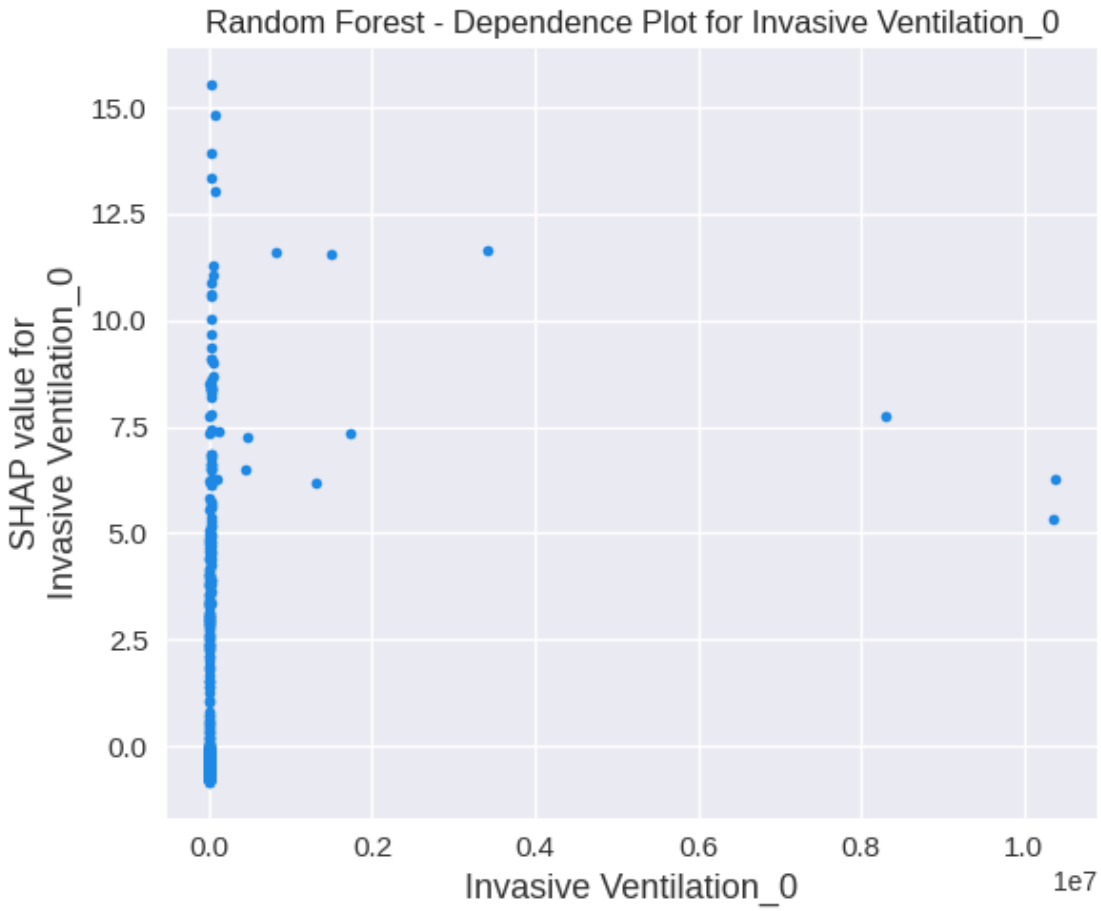
None



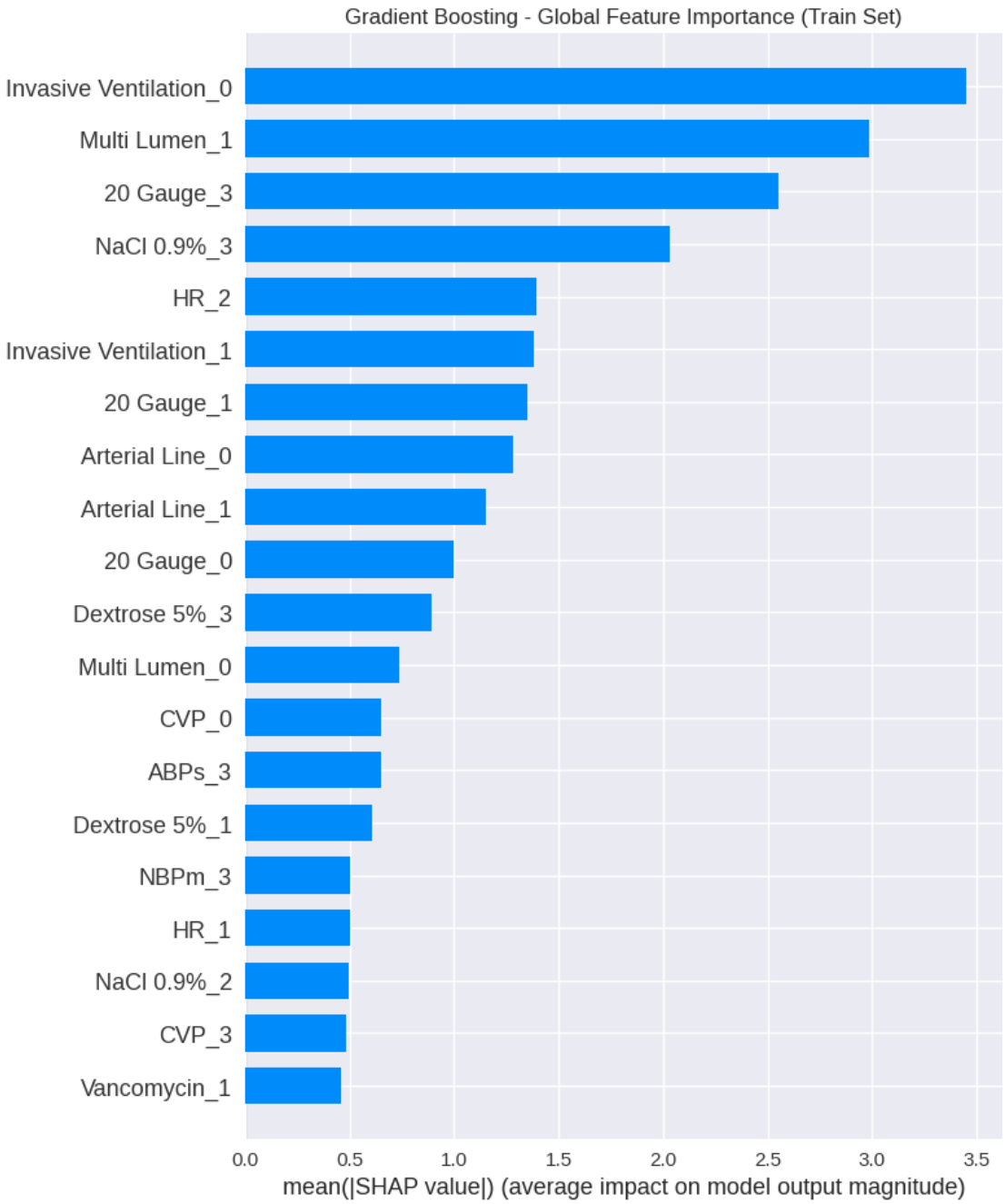
None

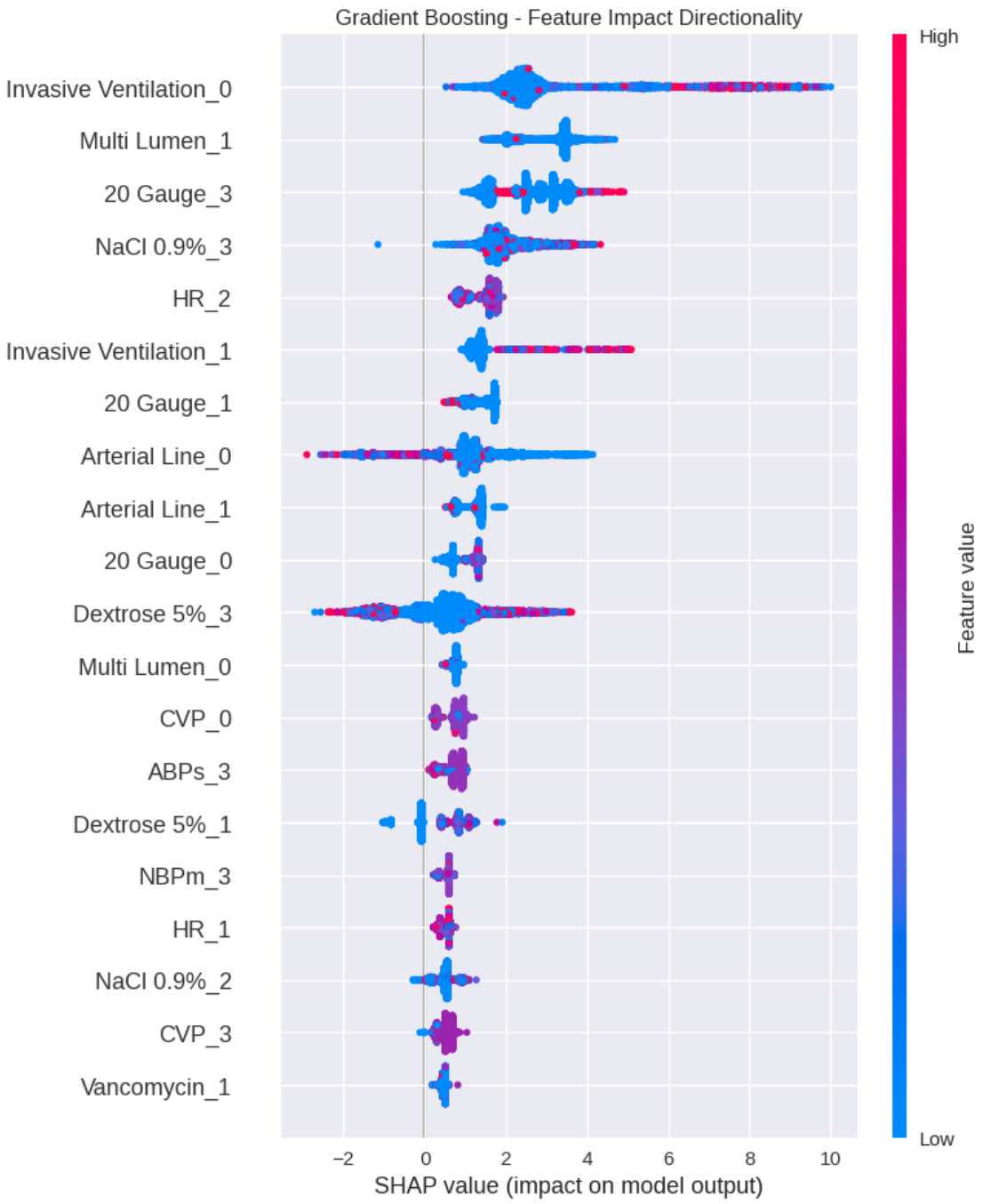

Random Forest - Dependence Plot for HR_3
SHAP value for HR_3
HR_3
0
2
4
6
8
0
20
40
60
80
100
120
140


Random Forest - Dependence Plot for NaCl 0.9%_3
SHAP value for
NaCl 0.9%_3
10
8
6
4
2
0
−2
0
2000
4000
6000
8000
NaCl 0.9%_3

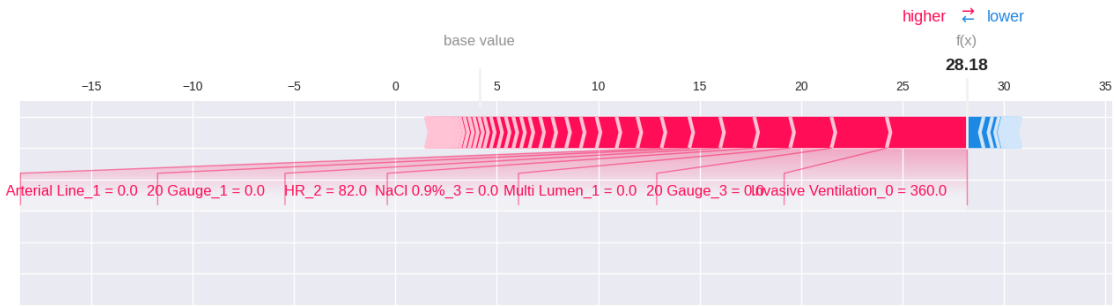



```
============================================================
GRADIENT BOOSTING SHAP ANALYSIS
============================================================
```

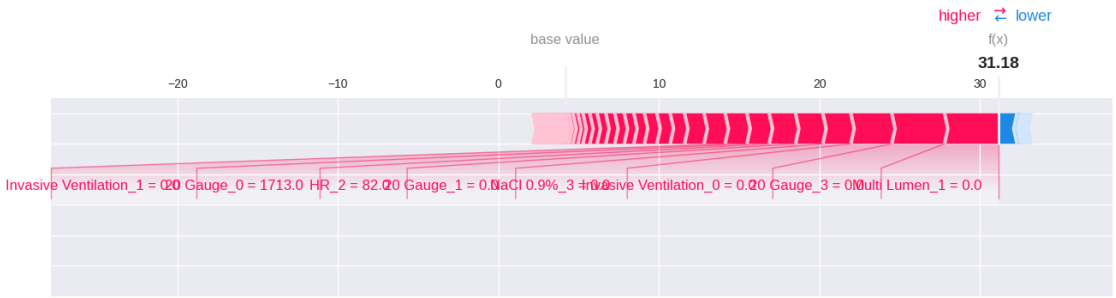

Gradient Boosting - Global Feature Importance (Train Set)
Invasive Ventilation_0
Multi Lumen_1
20 Gauge_3
NaCl 0.9%_3
HR_2
Invasive Ventilation_1
20 Gauge_1
Arterial Line_0
Arterial Line_1
20 Gauge_0
Dextrose 5%_3
Multi Lumen_0
CVP_0
ABPs_3
Dextrose 5%_1
NBPm_3
HR_1
NaCl 0.9%_2
CVP_3
Vancomycin_1
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
mean(|SHAP value|) (average impact on model output magnitude)



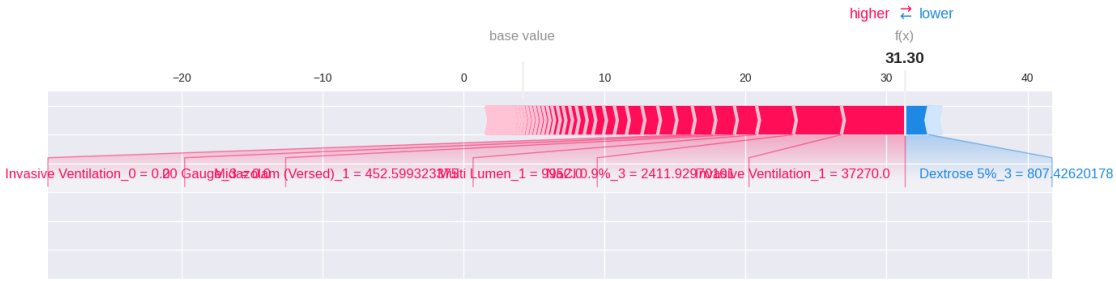
Gradient Boosting - Individual Prediction Explanations:



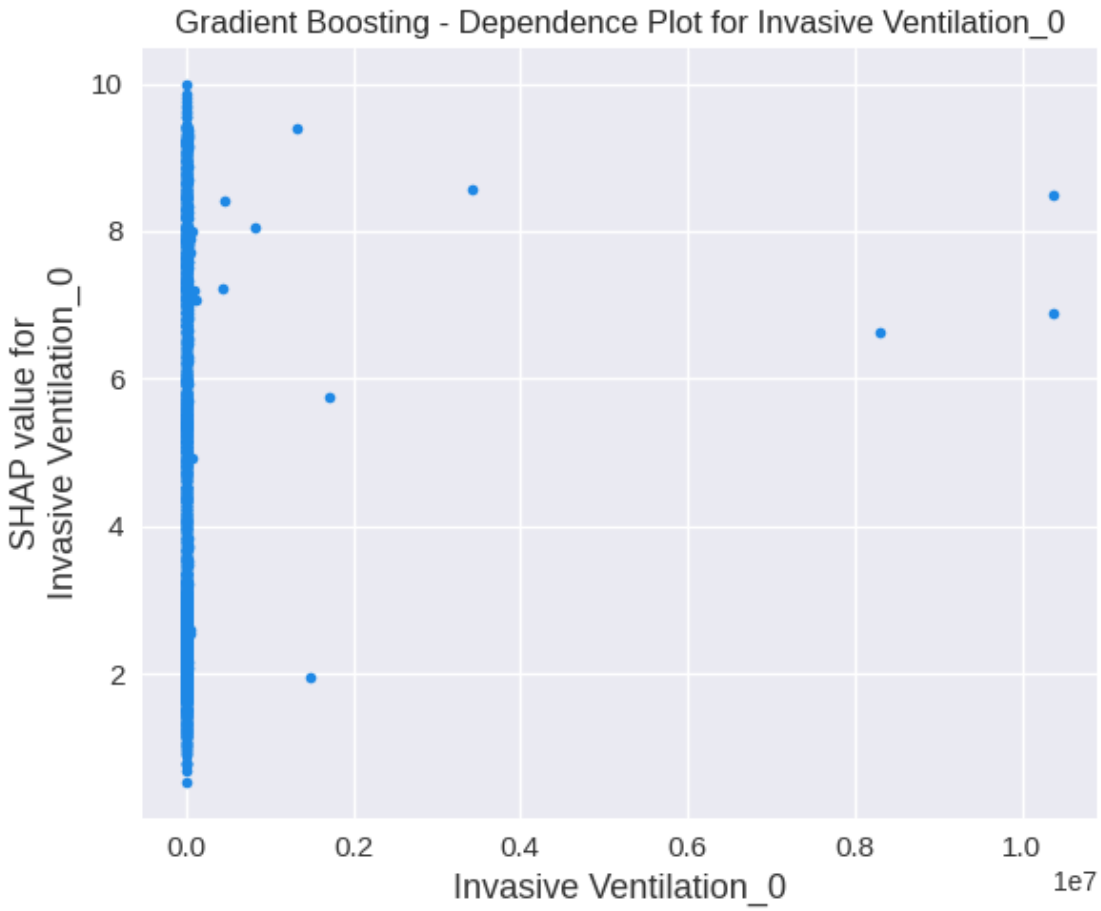
None

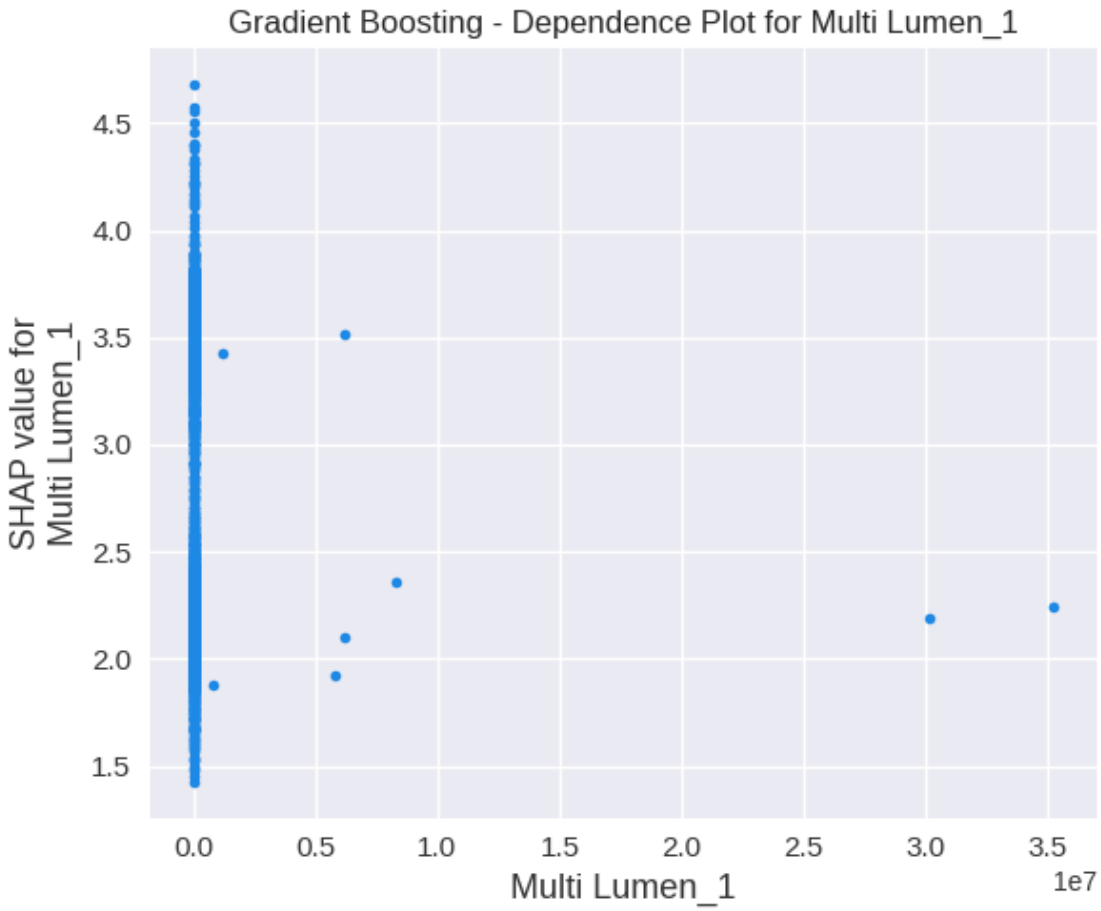


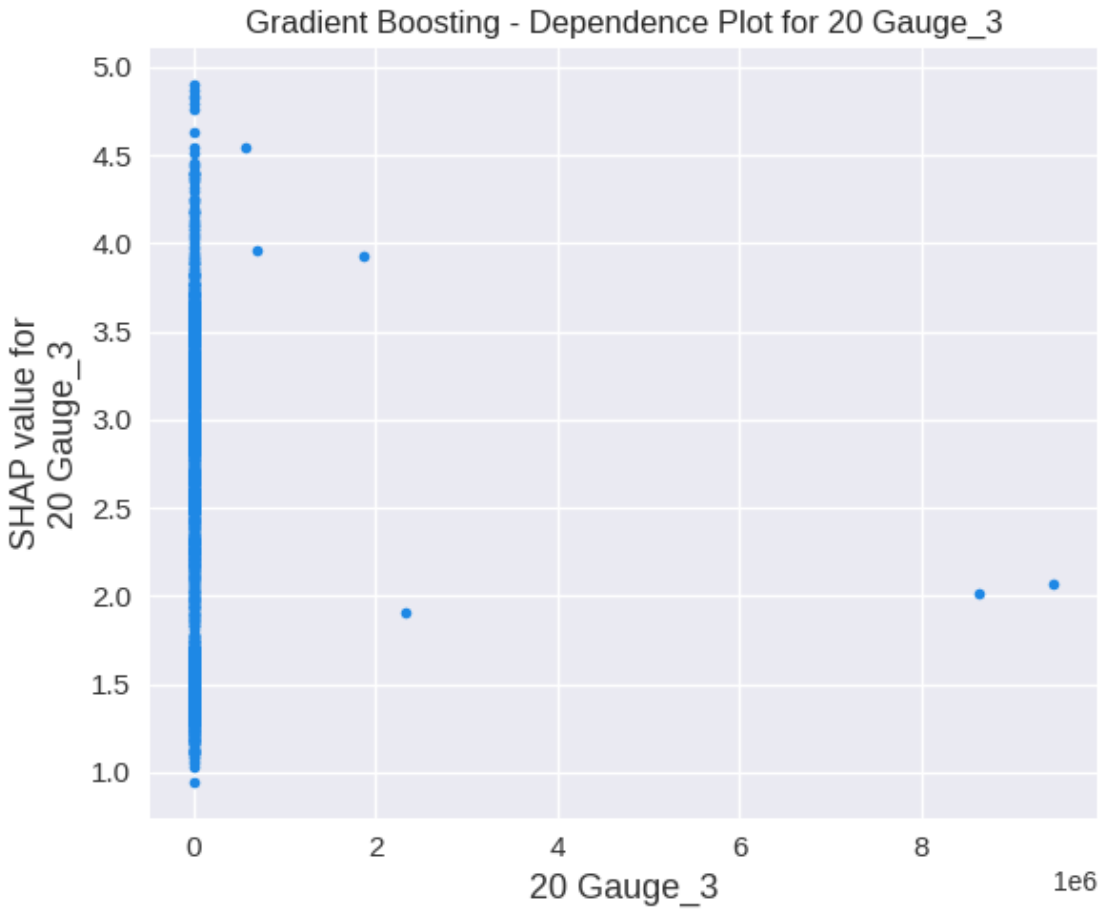
None

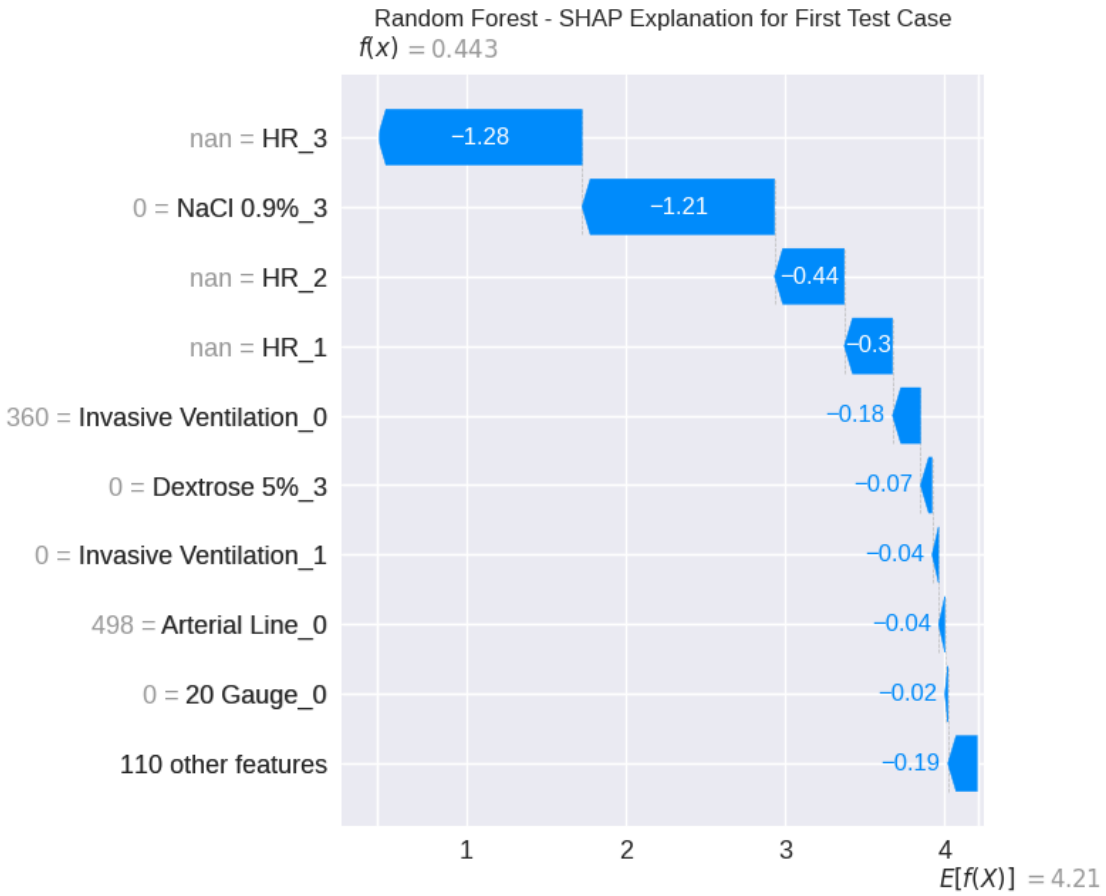


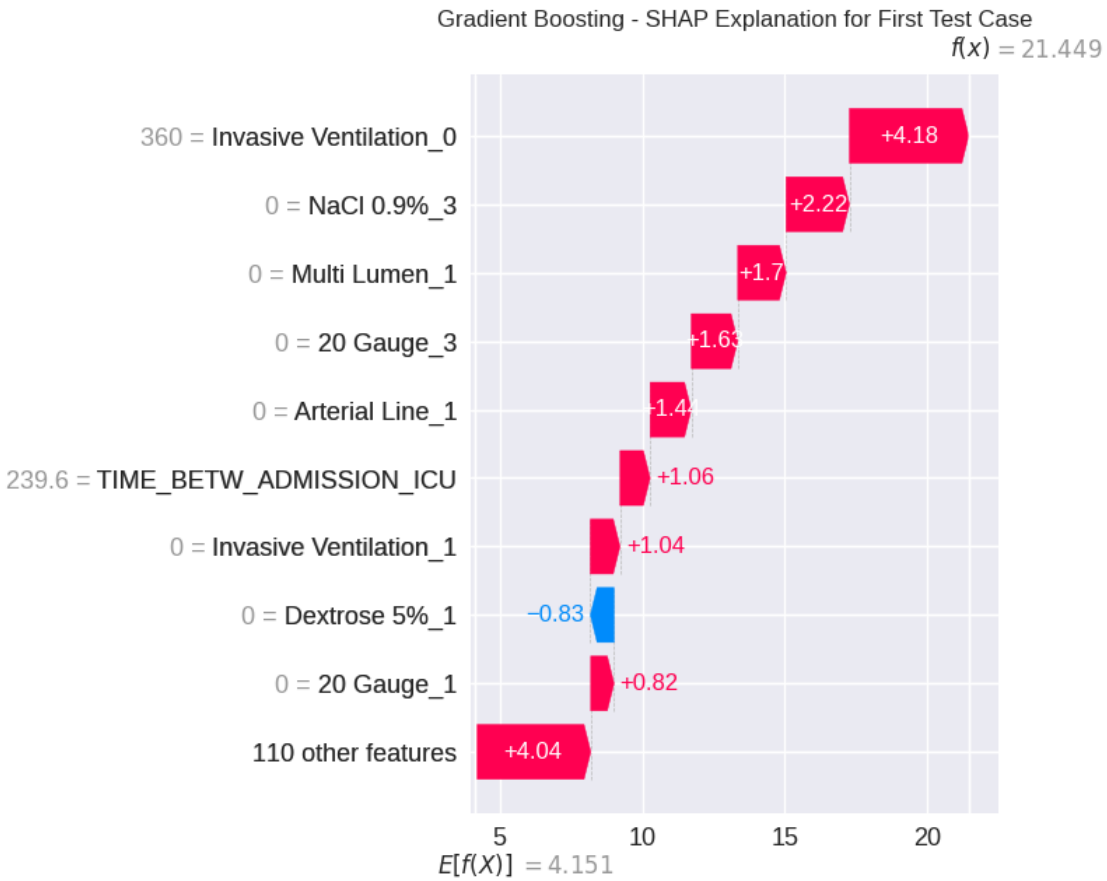
None


Gradient Boosting - Dependence Plot for Invasive Ventilation_0
SHAP value for
Invasive Ventilation_0
10
8
6
4
2
0.0
0.2
0.4
0.6
0.8
1.0
1e7
Invasive Ventilation_0


Gradient Boosting - Dependence Plot for Multi Lumen_1
SHAP value for
Multi Lumen_1
4.5
4.0
3.5
3.0
2.5
2.0
1.5
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
1e7
Multi Lumen_1


Gradient Boosting - Dependence Plot for 20 Gauge_3
SHAP value for
20 Gauge_3
5.0
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0
2
4
6
8
1e6
20 Gauge_3


Random Forest - SHAP Explanation for First Test Case
f(x) = 0.443
nan = HR_3
0 = NaCl 0.9%_3
nan = HR_2
nan = HR_1
360 = Invasive Ventilation_0
0 = Dextrose 5%_3
0 = Invasive Ventilation_1
498 = Arterial Line_0
0 = 20 Gauge_0
110 other features
−1.28
−1.21
−0.44
−0.3
−0.18
−0.07
−0.04
−0.04
−0.02
−0.19
1
2
3
4
E[f(X)] = 4.21



6 Conclusion

In this project, we explored the MIMIC-III ICU dataset with the goal of predicting the Length of Stay (LOS) for patients using a range of machine learning models. After selecting the heart failure diagnosis (ICD-9: 4280), we conducted thorough preprocessing, feature engineering, statistical analysis, and model evaluation. The data preprocessing included transforming time series data from several table into meaningful features for Machine Learning models learning from tabular data. Therefore, events were summarized on a daily basis. Moreover, we visualized the stay of an individual patient to get more insight into a common recovery timeline.

Model	Test MAE	Test RMSE	Test R^2
Baseline (Median)	2.68	5.05	-0.11
Random Forest + MICE	1.27	2.65	0.69
Gradient Boosting + KNN	1.38	2.77	0.67
HistGradientBoosting	1.24	2.59	0.71
Neural Network (MLP)	1.98	3.49	0.47
Ridge Regression	1.97	3.49	0.47

We could conclude that Tree-based models (Random Forest and Gradient Boosting) consistently outperformed other methods, with HistGradientBoosting achieving the best test R^2 of 0.71, indicating it explained 71% of the variance in LOS. The second best performer was the Random Forest. Both of these methods showed stable and competitive results across both cross-validation and test set. In contrast, the Neural Network and Ridge Regression performed notably worse, likely due to the data's high dimensionality and non-linear interactions that linear models cannot capture. Their shortcoming was especially evident in cross-validation, where they showed a tremendous variance and instability depending on the fold. Lastly, Baseline performance confirmed the value of using machine learning, as all models significantly improved over the median-only prediction.

An in-depth evaluation of the residuals confirmed that the errors of the well-performing models are within a reasonable range and follow a non-skewed distribution, although deviating from a normal distribution as indicated by the Shapiro Wilk Test. We could see that the predictions were especially non-accurate for very high LOS values, as longer stays are relatively rare, limiting the models' ability to learn their patterns.

Using SHAP values, we finally identified key clinical features that strongly influence LOS. These are HR_3, NaCl 0.9%_3, Invasive Ventilation_0, and 22 Gauge_3.

Possible explanations for their importance could be the following. For HR_3, abnormal heart rate on day 3 may indicate ongoing physiological instability, which can prolong recovery and hospital stay. Regarding NaCl 0.9%_3, the administration of normal saline on day 3 suggests persistent fluid management needs, reflecting patient severity. For Invasive Ventilation_0, early use of invasive ventilation often signals acute respiratory failure, typically associated with longer hospital stays. As of 22 Gauge_3, the use of a small-gauge IV catheter on day 3 may reflect limited vascular access or ongoing IV therapy needs, which can be linked to more complex care and longer LOS.

This project demonstrates the effectiveness of ensemble models for predicting ICU Length of Stay from complex clinical data. Tree-based models like HistGradientBoosting are not only accurate but also robust to missing data and interpretable via SHAP. The project also highlighted the importance of group-aware validation to prevent data leakage, careful feature preprocessing (imputation, encoding, scaling), and understanding model behavior through visual diagnostics.

For future work, several directions could enhance the predictive performance and clinical utility of the LOS model. First, incorporating time-dependent modeling approaches, such as recurrent neural networks (RNNs) or temporal attention mechanisms, may better capture the dynamic nature of patient trajectories compared to static daily summaries. Additionally, expanding the feature set to include more granular clinical notes could uncover subtler patterns associated with prolonged stays. Addressing the prediction challenges for long LOS cases through techniques such as quantile regression, outlier-aware modeling, or data augmentation could help mitigate the underperformance on rare but critical cases. Finally, integrating model predictions into a clinical decision support system (CDSS) and evaluating their impact in a prospective or simulated clinical setting would bridge the gap between predictive modeling and real-world ICU care.